



CODIGO DE IURIS

Código base – capa aplicación – proyecto
IURIS.MOVIL

DESCRIPCIÓN BREVE

Se muestra el código del proyecto IURIS.MOVIL que representa la capa aplicación, donde podemos interactuar con una interfaz gráfica quien obtendrá los elementos necesarios para poder trabajar de las capas COMMON, DAL Y BIZ, esta capa esta dirigida al usuario final, quien podrá interactuar con las leyes.

Juan Carlos Ortega Santiago

Desarrollador

Contenido

Archivo App	5
App.xaml.....	5
App.xaml.cs.....	7
Archivo FirtsView	9
FirtsView.xaml	9
FirtsView.xaml.cs	9
Apartado MainPage	10
MainPage.xaml	10
MainPage.xaml.cs	11
Archivo PageCrearCuenta	13
PageCrearCuenta.xaml	13
PageCrearCuenta.xaml.cs	15
Archivo PageInicioDeSesion	19
PageInicioDeSesion.xaml.....	19
PageInicioDeSesion.xaml.....	20
Archivo PageNuevoUser.....	32
PageNuevoUser.xaml	32
PageNuevoUser.xaml.cs	34
Archivo PagePrimeravista	35
PagePrimeravista.xaml	35
PagePrimeravista.xaml.cs.....	35
Archivo PageRecuperarCuenta	38
PageRecuperarCuenta.xaml	38
PageRecuperarCuenta.xaml.cs	39
Apartado Detail.....	41
PageMotrarCapitulosConCodigos.xaml.....	41
PageMotrarCapitulosConCodigos.xaml.cs	52
ViewDetail.xaml	70
ViewDetail.xaml.cs	74
Apartado Fonts	85
Archivo lucida_handwriting.ttf	85
Apartado Master.....	85

ViewMaster.xaml	85
ViewMaster.xaml.cs	87
Apartado Modelos y clases	89
TitulosDeMaster.cs	89
Carpeta BanerClass	89
AboutViewModel.cs	89
BaseViewModel.cs	89
IDataStore.cs	90
Item.cs	91
ItemDetailViewModel.cs	91
ItemsViewModel.cs	92
LoginViewModel.cs	94
MockDataStore.cs	94
NewItemViewModel.cs	96
Carpeta CustomRender	97
CustomEditor.cs	97
CustomEntry.cs	97
CustomLabel.cs	98
Carpeta DB_Local	98
Database.cs	98
LocalDataUltimoUser.cs	100
LocalSaveUser.cs	101
SaveMyClasificacionDeLey.cs	104
UltimoUserLocal.cs	105
Subcarpeta COMMON	106
MyClasificacion.cs	106
MyClasificacionDeLey.cs	106
MyDatosSobreUsuarioParaLey.cs	107
MyLey.cs	107
MyUser.cs	108
UltimoUser.cs	109
Carpeta Tools	109
ByteArrayToImageSourceConverter.cs	109

ClassAnuncio.cs	110
HerramientasGenerales.cs	112
Utils	114
GetClasificaiciones.cs	114
Settings.cs	116
Apartado Views.....	120
Carpeta Ajustes.....	120
PageAjustes.xaml	120
PageAjustes.xaml.cs	120
Carpeta Baners	121
baner.xaml.....	121
baner.xaml.cs.....	122
Carpeta ViewPayPal	127
PayPalPage.xaml	127
PayPalPage.xaml.cs	127
Carpeta ViewsApuntes	129
MisApuntes.xaml	129
MisApuntes.xaml.cs	130
ViewsMisApuntes.xaml	131
ViewsMisApuntes.xaml.cs	132
Carpeta ViewsCargarLey.....	134
ViewCargarLey.xaml	134
ViewCargarLey.xaml.cs.....	135
Subcarpeta TabbPage.....	139
TabbedPageCargar_CambiarLey.xaml.....	139
TabbedPageCargar_CambiarLey.xaml.cs.....	140
Subcarpeta ViewClasificaciones.....	140
ViewOfClasificaciones.xaml	140
ViewOfClasificaciones.xaml.cs	142
Sub-subcarpeta ViewPageLeyes.....	143
ViewLeyes.xaml	143
ViewLeyes.xaml.cs	144
Carpeta ViewsClasificaciones	146

ViewMiClasificacionPersonalizada.xaml.....	146
ViewMiClasificacionPersonalizada.xaml.cs.....	148
ViewsMisClasificaciones.xaml.....	150
ViewsMisClasificaciones.xaml.cs.....	151
Carpeta ViewsVentanasEmergentes.....	154
PageVistaSinInternet.xaml.....	154
PageVistaSinInternet.xaml.cs	154
WindowAlert.xaml	155
WindowAlert.xaml.cs	156
WindowListadoDeClasificaciones.xaml	158
WindowListadoDeClasificaciones.xaml.cs	159
WindowOfComprarEspacio.xaml	161
WindowOfComprarEspacio.xaml.cs	161
WindowOfEmergencyCrearNota.xaml	162
WindowOfEmergencyCrearNota.xaml.cs	163
WindowOfEmergencyNombreDeApunte.xaml.....	165
WindowOfEmergencyNombreDeApunte.xaml.cs.....	166
WindowOfEmergencyNuevaClasificacion.xaml	167
WindowOfEmergencyNuevaClasificacion.xaml.cs	168
WindowOfMenuAccion.xaml.....	170
WindowOfMenuAccion.xaml.cs.....	171
WindowOpcionDeCompra.xaml	173
WindowOpcionDeCompra.xaml.cs	174

Archivo App

App.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<Application xmlns="http://xamarin.com/schemas/2014/forms"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:d="http://xamarin.com/schemas/2014/forms/design"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
    mc:Ignorable="d"
    x:Class="IURIS.MOVIL.App">
    <Application.Resources>
        <ResourceDictionary>
            <Color x:Key="PrimaryColor">#640404</Color>
            <Color x:Key="ColorTexto">#640404</Color>
            <Color x:Key="SecondaryColor">#c40505</Color>

            <Style x:Key="TextoAMostrarColorBlanco" TargetType="Label">
                <Setter Property="TextColor" Value="White"/>
                <Setter Property="FontSize" Value="15"/>
            </Style>

            <Style x:Key="TextoAMostrarColorRojo" TargetType="Label">
                <Setter Property="TextColor" Value="{StaticResource
ColorTexto}"/>
            </Style>

            <Style x:Key="buttonPrincipal" TargetType="Button">
                <Setter Property="TextColor" Value="White"/>
                <Setter Property="BackgroundColor" Value="{StaticResource
PrimaryColor}"/>
                <Setter Property="BorderColor" Value="White"/>
                <Setter Property="BorderWidth" Value="2"/>
            </Style>

            <Style x:Key="buttonSecundario" TargetType="Button">
                <Setter Property="TextColor" Value="White"/>
                <Setter Property="BackgroundColor" Value="{StaticResource
SecondaryColor}"/>
                <Setter Property="BorderColor" Value="White"/>
                <Setter Property="BorderWidth" Value="2"/>
            </Style>

            <Style TargetType="FlexLayout" >
                <Setter Property="Direction" Value="Column"/>
                <Setter Property="AlignItems" Value="Stretch"/>
                <Setter Property="JustifyContent" Value="SpaceEvenly"/>
            </Style>

            <Style x:Key="EntryCrearCuenta" TargetType="Entry">
                <Setter Property="TextColor" Value="White"/>
                <Setter Property="FontSize" Value="25"/>
                <Setter Property="PlaceholderColor" Value="White"/>
                <Setter Property="VerticalOptions" Value="Center"/>
                <Setter Property="HorizontalOptions" Value="FillAndExpand"/>
            </Style>
        </ResourceDictionary>
    </Application.Resources>
</Application>
```

```

</Style>

<Style x:Key="EntryCrearUsuario" TargetType="Entry">
    <Setter Property="TextColor" Value="White"/>
    <Setter Property="FontSize" Value="30"/>
    <Setter Property="PlaceholderColor" Value="White"/>
    <Setter Property="VerticalOptions" Value="Center"/>
    <Setter Property="HorizontalOptions" Value="FillAndExpand"/>
</Style>

<Style x:Key="LabelCrearCuenta" TargetType="Label">
    <Setter Property="TextColor" Value="White"/>
    <Setter Property="FontSize" Value="Large"/>
    <Setter Property="HorizontalTextAlignment" Value="Center"/>
    <Setter Property="HorizontalOptions" Value="Center"/>
</Style>

<Style x:Key="LblMostrarLeyTitulo" TargetType="Label">
    <Setter Property="TextColor" Value="Black"/>
    <Setter Property="HorizontalOptions" Value="Center"/>
    <Setter Property="HorizontalTextAlignment" Value="Center"/>
    <Setter Property="FontSize" Value="Title"/>
</Style>

<Style x:Key="LblMostrarLeyCapitulo" TargetType="Label">
    <Setter Property="TextColor" Value="Black"/>
    <Setter Property="VerticalOptions" Value="Center"/>
    <Setter Property="HorizontalOptions" Value="StartAndExpand"/>
    <Setter Property="VerticalTextAlignment" Value="Center"/>
    <Setter Property="FontSize" Value="Large"/>
</Style>

<Style TargetType="CollectionView" x:Key="collection">
    <Setter Property="VisualStateManager.VisualStateGroups">
        <VisualStateGroupList>
            <VisualStateGroup x:Name="CommonStates">
                <VisualState x:Name="Normal" />
                <VisualState x:Name="Selected">
                    <VisualState.Setters>
                        <Setter Property="BackgroundColor"
                            Value="{StaticResource PrimaryColor}"/>
                        <Setter Property="Opacity" Value="0,7"/>
                    </VisualState.Setters>
                </VisualState>
            </VisualStateGroup>
        </VisualStateGroupList>
    </Setter>
</Style>

<Style TargetType="ContentView" x:Name="VisualStateSelected">
    <Setter Property="VisualStateManager.VisualStateGroups">
        <VisualStateGroupList>
            <VisualStateGroup x:Name="CommonStates">
                <VisualState x:Name="Normal" />
                <VisualState x:Name="Selected">
                    <VisualState.Setters>
                        <Setter Property="BackgroundColor"
Value="LightBlue" />

```

```

                <Setter Property="Opacity" Value="0.8"/>
            </VisualState.Setters>
        </VisualState>
    </VisualStateGroup>
</VisualStateGroupList>
</Setter>
</Style>

</ResourceDictionary>

<ResourceDictionary>
    <Color x:Key="NavigationPrimary">#640404</Color>
    <Style TargetType="NavigationPage">
        <Setter Property="BarBackgroundColor" Value="{StaticResource
NavigationPrimary}" />
        <Setter Property="BarTextColor" Value="White" />
    </Style>
</ResourceDictionary>
</Application.Resources>
</Application>

```

App.xaml.cs

```

using IURIS.DAL;
using IURIS.MOVIL.Modelos_y_clases;
using IURIS.MOVIL.Modelos_y_clases.DB_Local;
using IURIS.MOVIL.Modelos_y_clases.Utills;
using IURIS.MOVIL.Views.Baners;
using IURIS.MOVIL.Views.ViewPayPal;
using Matcha.BackgroundService;
using System;
using System.IO;
using Xamarin.Forms;
using Xamarin.Forms.Xaml;

[assembly: ExportFont("lucida_handwriting.ttf")]
namespace IURIS.MOVIL
{
    public partial class App : Application
    {
        public static MasterDetailPage masterDetail { get; set; }

        public static MyUser MyUser { get; set; }

        static Database database;
        static LocalDataUltimoUser UltimoUser;
        static SaveMyClasificacionDeLey SaveMyClasificacionDeLey;

        public static Database Database
        {
            get
            {
                if (database == null)
                {
                    database = new
Database(Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.Local
ApplicationData), "MyUserData.Iuris"));

```



```

        }
        return database;
    }
}

public static LocalDataUltimoUser ultimoUser
{
    get
    {
        if (UltimoUser == null)
        {
            UltimoUser = new
LocalDataUltimoUser(Path.Combine(Environment.GetFolderPath(Environment.SpecialF
older.LocalApplicationData), "MyUserData.Iuris"));
        }
        return UltimoUser;
    }
}

public static SaveMyClasificacionDeLey MyClasificacionDeLey
{
    get
    {
        if (SaveMyClasificacionDeLey == null)
        {
            SaveMyClasificacionDeLey = new
SaveMyClasificacionDeLey(Path.Combine(Environment.GetFolderPath(Environment.Spe
cialFolder.LocalApplicationData), "MyUserData.Iuris"));
        }
        return SaveMyClasificacionDeLey;
    }
}

public App()
{
    InitializeComponent();
    //Device.SetFlags(new[] { "RadioButton_Experimental" });
    //Device.SetFlags(new[] { "Expander_Experimental" });

    var pagina = new NavigationPage(new PagePrimeravista());
    pagina.BackgroundColor =
(Color)App.Current.Resources["PrimaryColor"];
    pagina.BarBackgroundColor =
(Color)App.Current.Resources["PrimaryColor"];
    MainPage = pagina;
}

protected override void OnStart()
{
    //Register Periodic Tasks
    BackgroundAggregatorService.Add(() => new GetClasificaiciones(24));
    //BackgroundAggregatorService.Add(() => new PeriodicCall2(4));

    //Start the background service
    BackgroundAggregatorService.StartBackgroundService();
}

protected override void OnSleep()

```

```

    {
        BackgroundAggregatorService.StopBackgroundService();
    }

    protected override void OnResume()
    {
        try
        {
            BackgroundAggregatorService.StartBackgroundService();
        }
        catch (Exception)
        {
            return;
        }
    }
}
}

```

Archivo FirtsView

FirtsView.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<MasterDetailPage xmlns="http://xamarin.com/schemas/2014/forms"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:d="http://xamarin.com/schemas/2014/forms/design"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
    mc:Ignorable="d"
    x:Class="IURIS.MOVIL.FirtsView"
    NavigationPage.HasNavigationBar="False" >
    <!--<ContentPage.Content>
        <StackLayout>
            <Label Text="Welcome to Xamarin.Forms!"
                VerticalOptions="CenterAndExpand"
                HorizontalOptions="CenterAndExpand" />
        </StackLayout>
    </ContentPage.Content>-->
</MasterDetailPage>

```

FirtsView.xaml.cs

```

using IURIS.COMMON.Entidades.UsuariosDeAplicacion;

```

```

using IURIS.MOVIL.Detail;
using IURIS.MOVIL.Master;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class FirtsView : MasterDetailPage
    {
        public FirtsView()
        {
            InitializeComponent();

            this.Master = new ViewMaster();
            this.Detail = new NavigationPage(new ViewDetail());
            //Detail = new NavigationPage(new LoginRegister() {
            BarBackgroundColor = Color.LimeGreen, BarTextColor = Color.White });

            App.masterDetail = this;
        }
    }
}

```

Apartado MainPage

MainPage.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:d="http://xamarin.com/schemas/2014/forms/design"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
    mc:Ignorable="d"
    x:Class="IURIS.MOVIL.MainPage"
    NavigationPage.HasNavigationBar="False">

    <ScrollView Padding="0,30">
        <FlexLayout>

            <Label FlexLayout.Shrink="0" FlexLayout.Basis=

                "auto" Text="IURIS" FontAttributes="Bold"
HorizontalTextAlignment="Center" FontSize="40" TextColor="White"/>

            <Label FlexLayout.Shrink="0" Padding="0,0,0,5"
HorizontalTextAlignment="Center" Text="Bienvenido" FontAttributes="Italic"
FontSize="30" TextColor="White"/>

```

```

        <StackLayout Margin="0,5" >

            <Image FlexLayout.Shrink="0" Source="Logo.png" Aspect="Fill"
HorizontalOptions="CenterAndExpand" VerticalOptions="CenterAndExpand"/>
        </StackLayout>

        <Label FlexLayout.Shrink="0" Text="From Semper Papyrus"
HorizontalTextAlignment="Center" FontSize="20" TextColor="White"/>

        <Button FlexLayout.Shrink="0" Margin="80,5" x:Name="BtnEntrar"
Clicked="BtnEntrar_Clicked" FontSize="Large" Style="{StaticResource
buttonSecundario}" Text="Entrar"/>

        <Button FlexLayout.Shrink="0" Text="Crear cuenta" Margin="80,5"
x:Name="btnCrearCuenta" Clicked="BtnCrearCuenta_Clicked" FontSize="Large"
Style="{StaticResource buttonSecundario}"/>

        <Label FlexLayout.Shrink="0" Text="Codigo de usuario"
HorizontalTextAlignment="Center" FontSize="15" TextColor="White" />

        <Label FlexLayout.Shrink="0" x:Name="lblCodigoUsuario"
Margin="0,0,0,5" HorizontalTextAlignment="Center" Text="000000" FontSize="15"
TextColor="White"/>

        <!--<Polyline Points="4 4,20 10,4 20"
Stroke="DarkBlue"
StrokeThickness="7"
StrokeLineJoin="Round"/>-->

    </FlexLayout>

</ScrollView>

</ContentPage>

```

MainPage.xaml.cs

```

using Acr.UserDialogs;
using IURIS.MOVIL.Modelos_y_clases;
using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using IURIS.MOVIL.Utills;
using IURIS.MOVIL.Views.ViewsVentanasEmergentes;
using Rg.Plugins.Popup.Services;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Xamarin.Essentials;
using Xamarin.Forms;

namespace IURIS.MOVIL
{

```

```

// Learn more about making custom code visible in the Xamarin.Forms
previewer
// by visiting https://aka.ms/xamarinforms-previewer
[DesignTimeVisible(false)]
public partial class MainPage : ContentPage
{
    int Intentos = 0;
    ClassAnuncio Anuncio = new ClassAnuncio();
    public MainPage()
    {
        DatosAValidar();
        InitializeComponent();
        HabilitarBotones(false);

        DatosAIniciar();
    }

    private async void DatosAValidar()
    {
        UserDialogs.Instance.ShowLoading("Validando\npor favor espere.",
MaskType.None);
        await Task.Delay(500);

        MainThread.BeginInvokeOnMainThread(async () =>
        {
            await Task.Delay(100);

            var ultimoUsers = await App.ultimoUser.GetUltimoUserAsync();
            UltimoUser _UltimoUser = ultimoUsers.FirstOrDefault();
            if (_UltimoUser!=null)
            {

                var usuarios = await App.Database.GetPeopleAsync();
                App.MyUser = usuarios.Find(r => r.Id ==
_UltimoUser.IdUser);
                if (App.MyUser != null)
                {
                    Intentos = 0;
                    await Navigation.PopAsync();
                    await Navigation.PushAsync(new FirtsView(), false);

                    UserDialogs.Instance.HideLoading();
                    return;
                }
            }
            else
            {
                HabilitarBotones(true);
            }
        });

        UserDialogs.Instance.HideLoading();
    }

    private void HabilitarBotones(bool v)
    {
        btnCrearCuenta.IsEnabled = v;
        BtnEntrar.IsEnabled = v;
    }
}

```

```

    }

    private void DatosAlIniciar()
    {
        if(Settings.NumUsuario != "") lblCodigoUsuario.Text =
Settings.NumUsuario;

        Anuncio.MostrarAnuncioPantalla();
    }

    private async void BtnCrearCuenta_Clicked(object sender, EventArgs e)
    {
        Intentos++;
        if (Intentos != 1) return;

        await Navigation.PushAsync(new PageCrearCuenta(), false);
        Intentos = 0;
    }

    private async void BtnEntrar_Clicked(object sender, EventArgs e)
    {
        Intentos++;
        if (Intentos != 1) return;

        await Navigation.PushAsync(new PageInicioDeSesion(), false);
        Intentos = 0;
    }
}
}

```

Archivo PageCrearCuenta

PageCrearCuenta.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:d="http://xamarin.com/schemas/2014/forms/design"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006" xmlns:local="clr-namespace:IURIS.MOVIL.Modelos_y_clases"
    mc:Ignorable="d"
    x:Class="IURIS.MOVIL.PageCrearCuenta" Title="Crear Cuenta"
    BackgroundColor="{StaticResource SecondaryColor}" >
    <ScrollView Padding="0,5">

        <FlexLayout>

            <local:CustomEntry x:Name="EntryNombre" Margin="15,0"
Placeholder="Nombre..." ClearButtonVisibility="WhileEditing" ReturnType="Next"
Style="{StaticResource EntryCrearCuenta}" IsTextPredictionEnabled="True"/>

```

```

        <local:CustomEntry x:Name="EntryApellidoPaterno" Margin="15,0"
Placeholder="Apellido paterno..." ClearButtonVisibility="WhileEditing"
ReturnType="Next" Style="{StaticResource EntryCrearCuenta}"
IsTextPredictionEnabled="True"/>

        <local:CustomEntry x:Name="EntryApellidoMaterno" Margin="15,0"
Placeholder="Apellido materno..." ClearButtonVisibility="WhileEditing"
ReturnType="Next" Style="{StaticResource EntryCrearCuenta}"
IsTextPredictionEnabled="True"/>

        <StackLayout Orientation="Vertical" VerticalOptions="Center"
Margin="15,0" >
            <local:CustomEntry x:Name="EntryCorreoElectronico"
TextChanged="EntryCorreoElectronico_TextChanged" Placeholder="Correo
electrónico..." ClearButtonVisibility="WhileEditing" ReturnType="Next"
Style="{StaticResource EntryCrearCuenta}" IsTextPredictionEnabled="True"/>
            <Label x:Name="lblCorreoExistente" TextColor="White"
FontSize="Medium" Text="El correo ingresado ya existe" IsVisible="False"/>
            <Label x:Name="lblFaltantesDeCorreo" TextColor="White"
FontSize="Medium" Text="Faltan componentes al correo" IsVisible="False"/>
        </StackLayout>

        <StackLayout Orientation="Vertical" VerticalOptions="Center"
Margin="15,0" >
            <local:CustomEntry x:Name="EntryContrasenia"
TextChanged="EntryContrasenia_TextChanged" MaxLength="4" Keyboard="Numeric"
Placeholder="Contraseña..." ClearButtonVisibility="WhileEditing"
ReturnType="Next" Style="{StaticResource EntryCrearCuenta}"/>
            <Label x:Name="lblTamañoDeContrasenia" TextColor="White"
FontSize="Medium" Text="Contraseña muy corta" IsVisible="False"/>
        </StackLayout>

        <StackLayout Orientation="Vertical" VerticalOptions="Center"
Margin="15,0" >
            <local:CustomEntry x:Name="EntryConfirmarContrasenia"
TextChanged="EntryConfirmarContrasenia_TextChanged" MaxLength="4"
Keyboard="Numeric" Placeholder="Confirmar contraseña..."
ClearButtonVisibility="WhileEditing" Completed="BtnOK_Clicked"
Style="{StaticResource EntryCrearCuenta}" />
            <Label x:Name="lblContraseniaNoCoincide" TextColor="White"
FontSize="Medium" Text="Las contraseñas no coinciden" IsVisible="False"/>
        </StackLayout>

        <Grid ColumnDefinitions="*,*" ColumnSpacing="-1" Margin="-2,10,-
2,0">
            <Button Grid.Column="0" Text="Ok" FontSize="Large"
x:Name="btnOK" Clicked="BtnOK_Clicked" Style="{StaticResource
buttonPrincipal}"/>
            <Button Grid.Column="1" FontSize="Large" Text="Cancelar"
x:Name="btnCanselar" Clicked="BtnCanselar_Clicked" Style="{StaticResource
buttonPrincipal}"/>
        </Grid>

        <Frame BorderColor="White"
Padding="0,2"
HasShadow="True"
Margin="-2,-1.5"
IsClippedToBounds="True"

```

```

        BackgroundColor="{StaticResource PrimaryColor}"
        x:Name="stackCodigoDeUsuario"
        IsVisible="false">

        <StackLayout Orientation="Vertical">
            <Label Text="Codigo de usuario" Style="{StaticResource
LabelCrearCuenta}"/>
            <Label x:Name="lblCodigoUsuario" Style="{StaticResource
LabelCrearCuenta}"/>
        </StackLayout>
    </Frame>

    <StackLayout Padding="10">
        <Image FlexLayout.Shrink="0" Source="Logo.png" Aspect="Fill"
HorizontalOptions="CenterAndExpand" VerticalOptions="CenterAndExpand"/>
    </StackLayout>

</FlexLayout>

</ScrollView>
</ContentPage>

```

PageCrearCuenta.xaml.cs

```

using Acr.UserDialogs;
using IURIS.BIZ;
using IURIS.COMMON.Constantes;
using IURIS.COMMON.Entidades.Ley;
using IURIS.COMMON.Entidades.Ley.ComponentesDeLey;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion.ComponentesDeUsuario;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion.ComponentesDeUsuario.DatosCriticos;
using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using IURIS.MOVIL.Utills;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;

using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class PageCrearCuenta : ContentPage
    {
        IManejadorDeUsuarioAplicacion manejadorDeUsuarioAplicacion;
        IManejadorDeLeyes manejadorDeLeyes;
        bool CorreoCorrecto = false;
        private bool TamanoDeContraseniaCorrecta = false;
    }
}

```



```

private bool ContraseniasIguales = false;
private string CorreoSinEspacios = "";
Const _Const = new Const();
//List<Usuarios> Usuarios;
public PageCrearCuenta()
{
    InitializeComponent();

    manejadorDeUsuarioAplicacion = new ManejadorDeUsuarioAplicacion(new
RepositorioGenerico<Usuarios>(true));
}

private async void BtnOK_Clicked(object sender, EventArgs e)
{
    UserDialogs.Instance.ShowLoading("Creando usuario\npor favor
esperar");
    await Task.Delay(200);
    try
    {
        if (string.IsNullOrEmpty(EntryCorreoElectronico.Text) ||
string.IsNullOrEmpty(EntryApellidoMaterno.Text) ||
string.IsNullOrEmpty(EntryApellidoPaterno.Text) ||
string.IsNullOrEmpty(EntryNombre.Text) ||
string.IsNullOrEmpty(EntryConfirmarContrasenia.Text) ||
string.IsNullOrEmpty(EntryContrasenia.Text))
        {
            await DisplayAlert("Nuevo registro", "Faltan datos por
llenar", "OK");
            return;
        }

        if (!CorreoCorrecto || !TamanoDeContraseniaCorrecta) return;

        //if
(manejadorDeUsuarioAplicacion.ExisteCorreo(EntryCorreoElectronico.Text))
        //{
            //    lblCorreoExistente.IsVisible = true;
            //    return;
            //}

        lblCorreoExistente.IsVisible = false;

        if (!ContraseniasIguales) return;

        int numUsuario = manejadorDeUsuarioAplicacion.Listar.Count + 1;
        //Usuarios usuarios = new Usuarios()
        //{
            //    Nombre = EntryNombre.Text,
            //    ApellidoPaterno = EntryApellidoPaterno.Text,
            //    ApellidoMaterno = EntryApellidoMaterno.Text,
            //    Correo = CorreoSinEspacios,
            //    IdApp = numUsuario,
            //    Contrasenia = int.Parse(EntryContrasenia.Text),
            //    Apuntes = new List<Apunte>(),
        //};
    }
}

```

```

        //manejadorDeLeyes = new ManejadorDeLeyes(new
RepositorioGenerico<Leyes>());
        //Leyes leyes = manejadorDeLeyes.Listar.Where(w =>
w.CodigoLey.ToUpper() == _Const.MiLeyPrincipal.ToUpper()).FirstOrDefault();
        //leyes.Clasificaciones = new List<ClasificacionPUusuario>();

        //List<Leyes> Mleyes = new List<Leyes>();
        //Mleyes.Add(leyes);
        //usuarios.MisLeyes = Mleyes;
        //usuarios.MiUltimaLeyCargada = leyes.CodigoLey;

        //usuarios.DatosSobreUsuario = new DatosSobreUsuarioParaLey();
        //usuarios.DatosSobreUsuario.NumLeyesPermitidas = 4;

        //Settings.LastCode = usuarios.MiUltimaLeyCargada;
        //Settings.NumAleatorio = 1;
        //Settings.CountParaNumAleatorio = "1";

        //if (!manejadorDeUsuarioAplicacion.AGREGAR(usuarios))
        //{
        //    await DisplayAlert("Crear cuenta", "No se puede efectuar
por el momento\n intente mas tarde", "OK");
        //    return;
        //}

        //stackCodigoDeUsuario.IsVisible = true;
        //lblCodigoUsuario.Text = usuarios.IdApp.ToString();

        //UserDialogs.Instance.HideLoading();

        //await DisplayAlert("Usuario creado", "Su registro fue
exitoso", "OK");

        await Navigation.PushAsync(new PageInicioDeSesion(), true);
    }
    catch (Exception ex)
    {
        UserDialogs.Instance.HideLoading();
        await DisplayAlert("Error", "Error: " + ex.Message, "ok");
        return;
    }
}

private async void BtnCanselar_Clicked(object sender, EventArgs e)
{
    await Navigation.PopAsync();
}

private bool email_bien_escrito(String email)
{
    String expresion;
    expresion = "\\w+([-+.']\\w+)*@\\w+([-.']\\w+)*\\.\\w+([-.']\\w+)*";

    if (Regex.IsMatch(email, expresion)) return Regex.Replace(email,
expresion, String.Empty).Length == 0 ? true : false;
    else return false;
}

```

```

        private void EntryCorreoElectronico_TextChanged(object sender,
TextChangedEventArgs e)
        {
            char space = ' ';
            CorreoSinEspacios=EntryCorreoElectronico.Text.TrimStart(space);
            CorreoSinEspacios =CorreoSinEspacios.TrimEnd(space);

            if (email_bien_escrito(CorreoSinEspacios))
            {
                CorreoCorrecto = true;
                lblFaltantesDeCorreo.IsVisible = false;
            }
            else
            {
                CorreoCorrecto = false;
                lblFaltantesDeCorreo.IsVisible = true;
            }
        }

        private void EntryContrasenia_TextChanged(object sender,
TextChangedEventArgs e)
        {
            lblTamanoDeContrasenia.IsVisible = EntryContrasenia.Text.Length <
4 ? true : false;
            if (!lblTamanoDeContrasenia.IsVisible)
            TamanoDeContraseniaCorrecta = true;

            lblContraseniaNoCoinside.IsVisible = EntryConfirmarContrasenia.Text
!= EntryContrasenia.Text ? true : false;
            if (!lblContraseniaNoCoinside.IsVisible) ContraseniasIguales =
true;
        }

        private void EntryConfirmarContrasenia_TextChanged(object sender,
TextChangedEventArgs e)
        {
            lblContraseniaNoCoinside.IsVisible = EntryConfirmarContrasenia.Text
!= EntryContrasenia.Text ? true : false;
            if (!lblContraseniaNoCoinside.IsVisible) ContraseniasIguales =
true;
        }
    }
}

```

Archivo PageInicioDeSesion

PageInicioDeSesion.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:d="http://xamarin.com/schemas/2014/forms/design"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006" xmlns:Local="clr-namespace:IURIS.MOVIL.Modelos_y_clases"
    mc:Ignorable="d"
    x:Class="IURIS.MOVIL.PageInicioDeSesion"
    NavigationPage.HasNavigationBar="False" >
    <ScrollView Padding="0,5,0,20">

        <FlexLayout>

            <Label FlexLayout.Shrink="0" Text="IURIS"
HorizontalTextAlignment="Center" FontAttributes="Bold" FontSize="40"
TextColor="White"></Label>

            <StackLayout Margin="0,20,0,0" FlexLayout.Shrink="0">

                <Image FlexLayout.Shrink="0" Source="Logo.png" Aspect="Fill"
HorizontalOptions="CenterAndExpand" VerticalOptions="CenterAndExpand"/>
            </StackLayout>

            <Label FlexLayout.Shrink="0" Text="From Semper Papyrus"
FontSize="30" HorizontalTextAlignment="Center" TextColor="White"></Label>

            <Label FlexLayout.Shrink="0" Margin="0,-15" Text="Inicio de sesión"
TextColor="White" HorizontalTextAlignment="Center" FontSize="Large"
VerticalOptions="End" VerticalTextAlignment="End"
HorizontalOptions="Center"></Label>

            <Grid FlexLayout.Shrink="0" Margin="0,-20"
FlexLayout.AlignSelf="Stretch" ColumnDefinitions="0.6*,*" ColumnSpacing="5">
                <Label Grid.Column="0" x:Name="lblcorreo" FlexLayout.Grow="20"
Text="Correo:" HorizontalTextAlignment="End" FontSize="Title"
Style="{StaticResource TextoAMostrarColorBlanco}"/>
                <Local:CustomEntry Grid.Column="1" FlexLayout.Grow="1"
x:Name="EntryCorreo" Placeholder="Ejemplo1234@gmail.com" ReturnType="Next"
Style="{StaticResource EntryCrearCuenta}" HorizontalTextAlignment="Center"
IsTextPredictionEnabled="False" FontSize="Medium"/>
            </Grid>

            <Grid FlexLayout.Shrink="0" Margin="0,-20"
FlexLayout.AlignSelf="Auto" ColumnDefinitions="0.6*,*" ColumnSpacing="5">
                <Label Grid.Column="0" x:Name="lblcontra" Text="Contraseña:"
HorizontalTextAlignment="End" FontSize="Title" Style="{StaticResource
TextoAMostrarColorBlanco}"/></Label>
                <Local:CustomEntry Grid.Column="1" MaxLength="4"
Keyboard="Numeric" Placeholder="****" Style="{StaticResource EntryCrearCuenta}"
HorizontalTextAlignment="Center" IsTextPredictionEnabled="False"
IsPassword="True" FontSize="Medium" x:Name="EntryPasswor"
Completed="BtnAceptar_Clicked"/>
            </Grid>

        </FlexLayout>

    </ScrollView>
</ContentPage>
```

```

        <StackLayout Orientation="Horizontal" FlexLayout.Shrink="0"
FlexLayout.AlignSelf="Center" Margin="15,-15" >
            <CheckBox Grid.Column="0" x:Name="CheckRecuerdame"
Color="White" CheckedChanged="CheckRecuerdame_CheckedChanged"
HorizontalOptions="End" ></CheckBox>
            <Label Text="Recordar usuario" Margin="5,0,0,0"
VerticalOptions="Center" HorizontalTextAlignment="End" FontSize="20"
Style="{StaticResource TextoAMostrarColorBlanco}"></Label>
            <StackLayout.GestureRecognizers>
                <TapGestureRecognizer
Tapped="TapGestureRecognizer_Tapped_1"/>
            </StackLayout.GestureRecognizers>
        </StackLayout>

        <Grid FlexLayout.Shrink="0" ColumnDefinitions="*,*" Margin="20,-
10">
            <Button Grid.Column="0" Text="Aceptar" x:Name="btnAceptar"
Clicked="BtnAceptar_Clicked" Style="{StaticResource buttonPrincipal}"
></Button>
            <Button Grid.Column="1" Text="Cancelar" x:Name="btnCanselar"
Clicked="BtnCanselar_Clicked" Style="{StaticResource
buttonPrincipal}"></Button>
        </Grid>

        <Label FlexLayout.Shrink="0" TextColor="White"
HorizontalTextAlignment="Center" FontSize="Medium"
x:Name="LblNoRecuerdoMiContrasenia" Margin="0,-10">
            <Label.FormattedText>
                <FormattedString>
                    <Span Text="No recuerdo mi contraseña"
TextDecorations="Underline"/>
                </FormattedString>
            </Label.FormattedText>
            <Label.GestureRecognizers>
                <TapGestureRecognizer
Tapped="TapGestureRecognizer_Tapped"/>
            </Label.GestureRecognizers>
        </Label>

    </FlexLayout>

</ScrollView>

</ContentPage>

```

PageInicioDeSesion.xaml

```

//using Xamarin.Forms.PlatformConfiguration.Android.Widget;

using Acr.UserDialogs;

using IURIS.BIZ;

```

```
using IURIS.COMMON.Entidades.Ley;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion.ComponentesDeUsuario.DatosCriticos;
using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using IURIS.MOVIL.Modelos_y_clases.DB_Local;
using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using IURIS.MOVIL.Modelos_y_clases.Tools;
using IURIS.MOVIL.Utills;
using MarcTron.Plugin.Controls;
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Xamarin.Essentials;
```

```
using Xamarin.Forms;
using Xamarin.Forms.Xaml;
```

```
namespace IURIS.MOVIL
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class PageInicioDeSesion : ContentPage
    {
        int Intentos = 0;
```

```
        IManejadorDeUsuarioAplicacion manejadorDeUsuarioAplicacion;
```

```

IManejadorDeLeyes manejadorDeLeyes;

Usuarios _User;

bool Recuerdame = false;

List<Leyes> LeyesParaActualizar=new List<Leyes>();

List<Leyes> LeyesActualizadas;

public PageInicioDeSesion()
{
    InitializeComponent();

    manejadorDeUsuarioAplicacion = new ManejadorDeUsuarioAplicacion(new
RepositorioGenerico<Usuarios>(true));

    //manejadorDeLeyes = new ManejadorDeLeyes(new RepositorioGenerico<Leyes>());

    //MainThread.BeginInvokeOnMainThread(async () => {

    //    await Task.Delay(5000);

    //    //LeyesActualizadas = manejadorDeLeyes.Listar.Where(x => x.EsModificacion ==
true).ToList();

    //    if (Settings.LastCode != "Last_LeyCodigo_key")

    //    {

    //        _leyGuardada = manejadorDeLeyes.Listar.Find(x=>x.CodigoLey==Settings.LastCode);

    //        Debug.WriteLine("[ID]--Ley guardad {" + _leyGuardada.id.ToString());

    //    }

    //});

    DatosAlIniciar();

}

private void DatosAlIniciar()
{

```

```
if (!Settings.Recuerdame) return;
```

```
EntryCorreo.Text = Settings.Email != "" ? Settings.Email : null;
```

```
EntryPasswor.Text = Settings.Contrasenia != "" ? Settings.Contrasenia : null;
```

```
CheckRecuerdame.IsChecked = Settings.Recuerdame;
```

```
}
```

```
private async void BtnCanselar_Clicked(object sender, EventArgs e)
```

```
{
```

```
    Intentos++;
```

```
    if (Intentos != 1) return;
```

```
    await Navigation.PushAsync(new MainPage(), false);
```

```
    Intentos = 0;
```

```
}
```

```
private void TapGestureRecognizer_Tapped(object sender, EventArgs e)
```

```
{
```

```
    Intentos++;
```

```
    if (Intentos != 1) return;
```

```
    LblNoRecuerdoMiContrasenia.TextColor = Color.CadetBlue;
```

```
    Navigation.PushAsync(new PageRecuperarCuenta(), false);
```

```
    LblNoRecuerdoMiContrasenia.TextColor = Color.White;
```

```
    Intentos = 0;
```

```
}
```



```

private void BtnAceptar_Clicked(object sender, EventArgs e)
{
    MainThread.BeginInvokeOnMainThread(async () =>
    {
        await Task.Delay(1000);

        UserDialogs.Instance.ShowLoading("Iniciando sesión", MaskType.None);

        await Task.Delay(200);

        Intentos++;

        if (Intentos != 1) return;

        try
        {
            var ultimoUsers= await App.ultimoUser.GetUltimoUserAsync();
            UltimoUser _UltimoUser = ultimoUsers.FirstOrDefault();
            if (_UltimoUser != null)
            {

                var usuarios = await App.Database.GetPeopleAsync();
                App.MyUser = usuarios.Find(r => r.Id==_UltimoUser.IdUser);
                if (App.MyUser != null)
                {
                    Intentos = 0;

                    await Navigation.PushAsync(new FirtsView(), false);

                    UserDialogs.Instance.HideLoading();

                    return;
                }
            }
        }
    }
}

```

```
}
```

```
if (string.IsNullOrEmpty(EntryPasswor.Text) ||  
string.IsNullOrEmpty(EntryCorreo.Text)) return;
```

```
string CorreoSinEspacios;
```

```
CorreoSinEspacios = EntryCorreo.Text.TrimStart();
```

```
CorreoSinEspacios = CorreoSinEspacios.TrimEnd();
```

```
//_User = manejadorDeUsuarioAplicacion.EncontrarUsuario(CorreoSinEspacios,  
int.Parse(EntryPasswor.Text));
```

```
//if (_User != null)
```

```
//{
```

```
//  Settings.Recuerdame = Recuerdame;
```

```
//  if (Settings.Recuerdame)
```

```
//  {
```

```
//    Settings.Email = _User.Correo;
```

```
//    Settings.Contrasenia = _User.Contrasenia.ToString();
```

```
//  }
```

```
//  Settings.NumUsuario = _User.IdApp.ToString();
```

```
//  Settings.LastCode = _User.MiUltimaLeyCargada;
```

```
//  //HerramientasGenerales herramientasGenerales = new  
HerramientasGenerales(_User);
```

```
//  //herramientasGenerales.TipoDeAlmacenamiento();
```

```
//  LocalSaveUser localSaveUser = new LocalSaveUser(_User);
```

```

// if (!await localSaveUser.ExisteUsuario()) localSaveUser.Save();

// UltimoUserLocal _Ultimo = new UltimoUserLocal();

// if (_UltimoUser != null)
// {

//     _UltimoUser.IdUser = App.MyUser.Id;
//     await _Ultimo.Update(_UltimoUser);
// }
// else
// {
//     _Ultimo.Save(App.MyUser.Id);
// }

// //if (HayActualizacion()) Actualizaeyes();

// await Navigation.PushAsync(new FirtsView(), false);
//}
//else
//{
//     await DisplayAlert("Error de usuario", "Por favor verifica los datos ingresados",
"OK");

//}

```

```

        //Intentos = 0;
    }
    catch (TimeoutException)
    {
        Intentos = 0;
        await Task.Delay(200);
        UserDialogs.Instance.HideLoading();
        await DisplayAlert("Error", "No tienes conexión a internet", "ok");

        return;
    }

    catch (Exception ex)
    {
        Intentos = 0;
        await Task.Delay(200);
        UserDialogs.Instance.HideLoading();
        await DisplayAlert("Error", "Error:" + ex.Message, "ok");
        return;
    }
    await Task.Delay(200);
    UserDialogs.Instance.HideLoading();
});
}

private async void Actualizaeyes()
{
    foreach (var LeyParaActualizar in LeyesParaActualizar)
    {

```

```

foreach (var MiLey in _User.MisLeyes)
{
    if (MiLey.id == LeyParaActualizar.id)
    {

        if (_User.MiUltimaLeyCargada == MiLey.CodigoLey)

            _User.MiUltimaLeyCargada = MiLey.CodigoLey != LeyParaActualizar.CodigoLey ?
LeyParaActualizar.CodigoLey : _User.MiUltimaLeyCargada;

            MiLey.EsModificacion = MiLey.EsModificacion != LeyParaActualizar.EsModificacion ?
LeyParaActualizar.EsModificacion : MiLey.EsModificacion;

            //MiLey.EsModificacion = false;

            MiLey.NombreLey = MiLey.NombreLey != LeyParaActualizar.NombreLey ?
LeyParaActualizar.NombreLey : MiLey.NombreLey;

            MiLey.UltimaFechaDeModificacion = MiLey.UltimaFechaDeModificacion !=
LeyParaActualizar.UltimaFechaDeModificacion ? LeyParaActualizar.UltimaFechaDeModificacion :
MiLey.UltimaFechaDeModificacion;

            MiLey.CodigoLey = MiLey.CodigoLey != LeyParaActualizar.CodigoLey ?
LeyParaActualizar.CodigoLey : MiLey.CodigoLey;

            //MiLey.ListaDeTitulos[0] = null;

            foreach (var Titulo in MiLey.ListaDeTitulos)
            {

                foreach (var TituloAAActualizar in LeyParaActualizar.ListaDeTitulos)
                {

                    if (Titulo.id == TituloAAActualizar.id)
                    {

```

```
Titulo.NumTitulo = Titulo.NumTitulo != TituloAAActualizar.NumTitulo ?  
TituloAAActualizar.NumTitulo : Titulo.NumTitulo;
```

```
Titulo.NombreTitulo = Titulo.NombreTitulo != TituloAAActualizar.NombreTitulo  
? TituloAAActualizar.NombreTitulo : Titulo.NombreTitulo;
```

```
foreach (var _Capitulo in Titulo.ListaCapitulos)  
{  
    foreach (var _CapituloAAActualizar in TituloAAActualizar.ListaCapitulos)  
    {  
        if (_Capitulo.id == _CapituloAAActualizar.id)  
        {  
            _Capitulo.NumCapitulo = _Capitulo.NumCapitulo !=  
_CapituloAAActualizar.NumCapitulo ? _CapituloAAActualizar.NumCapitulo : _Capitulo.NumCapitulo;  
            _Capitulo.NombreCapitulo = _Capitulo.NombreCapitulo !=  
_CapituloAAActualizar.NombreCapitulo ? _CapituloAAActualizar.NombreCapitulo :  
_Capitulo.NombreCapitulo;
```

```
        foreach (var _Articulo in _Capitulo.ListaArticulos)  
        {  
            foreach (var _ArticuloAModificar in  
_CapituloAAActualizar.ListaArticulos)  
            {  
                if (_Articulo.id == _ArticuloAModificar.id)  
                {
```

```
                    _Articulo.Contenido = _Articulo.Contenido !=  
_ArticuloAModificar.Contenido ? _ArticuloAModificar.Contenido : _Articulo.Contenido;  
                    _Articulo.NombreArticulo = _Articulo.NombreArticulo !=  
_ArticuloAModificar.NombreArticulo ? _ArticuloAModificar.NombreArticulo :  
_Articulo.NombreArticulo;
```

```
        _Articulo.NumArticulo = _Articulo.NumArticulo !=  
_ArticuloAModificar.NumArticulo ? _ArticuloAModificar.NumArticulo : _Articulo.NumArticulo;  
    }  
  
    }  
  
    }  
  
    }  
  
    }  
  
    }
```

```
    }
```

```
    }
```

```
    }
```

```
    }
```

```
}
```

```
try
```

```
{
```

```
    if (_User.DatosSobreUsuario == null)
```

```
    {
```

```
        _User.DatosSobreUsuario = new DatosSobreUsuarioParaLey();
```

```
        _User.DatosSobreUsuario.NumLeyesPermitidas = 4;
```

```
    }
```

```
    if (manejadorDeUsuarioAplicacion.Modificar(_User))
```

```

        await DisplayAlert("", "la actualizacion fue exitosa", "OK");

    }
    catch (Exception)
    {

        throw;
    }
}

private void CheckRecuerdame_CheckedChanged(object sender, CheckedChangedEventArgs e)
{
    Recuerdame = CheckRecuerdame.IsChecked;
}

private void TapGestureRecognizer_Tapped_1(object sender, EventArgs e)
{
    CheckRecuerdame.IsChecked = CheckRecuerdame.IsChecked ? false : true;
    Recuerdame = CheckRecuerdame.IsChecked ? true : false;
}

public bool HayActualizacion()
{
    if (LeyesActualizadas.Count == 0)
        return false;

    foreach (var Ley in LeyesActualizadas)

```



```

    {
        foreach (var MiLey in _User.MisLeyes)
        {
            if (MiLey.id == Ley.id && Ley.UltimaFechaDeModificacion >
MiLey.UltimaFechaDeModificacion) //&& Ley.UltimaFechaDeModificacion >
MiLey.UltimaFechaDeModificacion //Poner que obtenga la hora y la fecha exactas
            {
                LeyesParaActualizar.Add(Ley);
            }
        }
    }

    return LeyesParaActualizar.Count == 0;
}
}
}

```

Archivo PageNuevoUser

PageNuevoUser.xaml

```

<?xml version="1.0" encoding="utf-8" ?>

<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"

    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml" xmlns:local="clr-
namespace:IURIS.MOVIL.Modelos_y_clases"

    x:Class="IURIS.MOVIL.PageNuevoUser" Title="Nuevo usuario"
BackgroundColor="{StaticResource PrimaryColor}" NavigationPage.HasBackButton="False">

    <StackLayout>

```

<StackLayout>

```
<local:CustomEntry x:Name="EntryNombre" Margin="15,5,0,0" Placeholder="Nombre..."
ClearButtonVisibility="WhileEditing" ReturnType="Next" Style="{StaticResource
EntryCrearUsuario}" IsTextPredictionEnabled="True"/>
```

```
<local:CustomEntry x:Name="EntryApellidoPaterno" Margin="15,5,0,0"
Placeholder="Apellido paterno..." ClearButtonVisibility="WhileEditing" ReturnType="Next"
Style="{StaticResource EntryCrearUsuario}" IsTextPredictionEnabled="True"/>
```

```
<local:CustomEntry x:Name="EntryApellidoMaterno" Margin="15,5,0,0"
Placeholder="Apellido materno..." ClearButtonVisibility="WhileEditing" ReturnType="Next"
Style="{StaticResource EntryCrearUsuario}" IsTextPredictionEnabled="True"/>
```

</StackLayout>

<Grid ColumnDefinitions="*,*" ColumnSpacing="-1" Margin="-2,30,-2,50">

```
<Button Grid.Column="0" Text="Ok" FontSize="Large" x:Name="btnOK"
Style="{StaticResource buttonPrincipal}" Clicked="btnOK_Clicked"/>
```

```
<Button Grid.Column="1" FontSize="Large" Text="Cancelar" x:Name="btnCanselar"
Style="{StaticResource buttonPrincipal}"/>
```

</Grid>

<StackLayout Padding="10">

```
<Image FlexLayout.Shrink="0" Source="Logo.png" Aspect="Fill"
HorizontalOptions="CenterAndExpand" VerticalOptions="CenterAndExpand"/>
```

</StackLayout>

</StackLayout>

</ContentPage>

PageNuevoUser.xaml.cs

```
using Acr.UserDialogs;
using IURIS.MOVIL.Modelos_y_clases.DB_Local;
using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Xamarin.Essentials;
using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class PageNuevoUser : ContentPage
    {
        public PageNuevoUser()
        {
            InitializeComponent();
        }

        private async void btnOK_Clicked(object sender, EventArgs e)
        {
            UserDialogs.Instance.ShowLoading("Creando\npor favor espere.",
MaskType.None);
            await Task.Delay(500);

            MainThread.BeginInvokeOnMainThread(async () =>
            {
                await Task.Delay(1000);

                if (string.IsNullOrEmpty(EntryNombre.Text) &&
string.IsNullOrEmpty(EntryApellidoPaterno.Text) &&
string.IsNullOrEmpty(EntryApellidoMaterno.Text)) return;

                LocalSaveUser localSaveUser = new LocalSaveUser();
                if (localSaveUser.SaveUser(EntryNombre.Text,
EntryApellidoPaterno.Text, EntryApellidoMaterno.Text))
                {
                    await Navigation.PushAsync(new FirtsView(), false);
                    UserDialogs.Instance.HideLoading();
                }
                else
                {
                    await DisplayAlert("", "Error no se ha podido creare el
usuario, intenta mas tarde", "Aceptar");
                    await Navigation.PopAsync();
                }
            });
        }
    }
}
```

Archivo PagePrimeravista

PagePrimeravista.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="IURIS.MOVIL.PagePrimeravista"
              BackgroundColor="{StaticResource PrimaryColor}"
              NavigationPage.HasNavigationBar="False">
    <ContentPage.Content>
        <StackLayout HorizontalOptions="Center" VerticalOptions="Center">

            <Image Source="Logo.png" Aspect="Fill" HorizontalOptions="Center"
              VerticalOptions="Center" Margin="0,0,0,20"/>
            <Label x:Name="lblEntrar" Text="Entrar" Style="{StaticResource
              TextoAMostrarColorBlanco}" FontSize="50" FontAttributes="Italic"
              HorizontalOptions="Center" IsVisible="false"/>

            <StackLayout.GestureRecognizers>
                <TapGestureRecognizer Tapped="TapGestureRecognizer_Tapped"/>
            </StackLayout.GestureRecognizers>
        </StackLayout>
    </ContentPage.Content>
</ContentPage>
```

PagePrimeravista.xaml.cs

```
using Acr.UserDialogs;
using IURIS.BIZ;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion.ComponentesDeUsuario.DatosCriticos;
using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using IURIS.MOVIL.Views.ViewsVentanasEmergentes;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Net.NetworkInformation;
using System.Text;
using System.Threading.Tasks;
using Xamarin.Essentials;
using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class PagePrimeravista : ContentPage
    {
    }
```

```

IManejadorDeUsuarioAplicacion manejadorDeUsuarioAplicacion;
public PagePrimeravista()
{
    InitializeComponent();
    DatosAValidar();
}

public static bool HayConexion()
{
    try
    {
        string huesped = $"8.8.8.8";

        return new Ping().Send(huesped).Status == IPStatus.Success;
    }
    catch
    {
        return false;
    }
}

private async void DatosAValidar()
{
    if (!HayConexion())
    {
        await Navigation.PopAsync(false);
        await Navigation.PushAsync(new PageVistaSinInternet("Revisa tu
conexión\na internet"), false);
        return;
    }
    UserDialogs.Instance.ShowLoading("Validando\npor favor
espere.", MaskType.None);
    await Task.Delay(500);

    MainThread.BeginInvokeOnMainThread(async () =>
    {
        await Task.Delay(1000);

        try
        {
            manejadorDeUsuarioAplicacion = new
ManejadorDeUsuarioAplicacion(new RepositorioGenerico<Usuarios>());

            var usuarios = await App.Database.GetPeopleAsync();
            App.MyUser = usuarios.FirstOrDefault();

            if (App.MyUser != null)
            {
                if (App.MyUser.IdApp == 0)

```

```

        {
            try
            {
                Usuarios user = new Usuarios()
                {
                    ApellidoMaterno =
App.MyUser.ApellidoMaterno,
                    ApellidoPaterno =
App.MyUser.ApellidoPaterno,
                    DatosSobreUsuario = new
DatosSobreUsuarioParaLey() { NumLeyesPermitidas =
App.MyUser.DatosSobreUsuario.NumLeyesPermitidas },
                    Nombre = App.MyUser.Nombre,
                    IdApp =
manejadorDeUsuarioAplicacion.Listar.Count + 1
                };
                if
(manejadorDeUsuarioAplicacion.AGREGAR(user))
                {
                    App.MyUser.IdApp = user.IdApp;
                    await
App.Database.UpdateUserAsync(App.MyUser);
                }
            }
            catch (Exception)
            {
            }
        }

        Usuarios User = await
manejadorDeUsuarioAplicacion.ConsultUsuario(App.MyUser.IdApp.ToString(),
App.MyUser.Nombre, App.MyUser.ApellidoPaterno, App.MyUser.ApellidoMaterno);

        if (User.DatosSobreUsuario.NumLeyesPermitidas !=
App.MyUser.DatosSobreUsuario.NumLeyesPermitidas)
        {
            App.MyUser.DatosSobreUsuario.NumLeyesPermitidas
= User.DatosSobreUsuario.NumLeyesPermitidas;
            await App.Database.UpdateUserAsync(App.MyUser);
        }

        await Navigation.PushModalAsync(new FirtsView(),
false);
    }
    else { lblEntrar.IsVisible = true; }

    UserDialogs.Instance.HideLoading();
    await Navigation.PopAsync(false);
}
catch (Exception)
{
    UserDialogs.Instance.HideLoading();

    await Navigation.PushAsync(new
PageVistaSinInternet("Ocurrio un error\nintente mas tarde"), false);
}

```

```

        }

        });

    }

    private async void TapGestureRecognizer_Tapped(object sender, EventArgs
e)
    {
        await Navigation.PushAsync(new PageNuevoUser(), false);
    }
}

```

Archivo PageRecuperarCuenta

PageRecuperarCuenta.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:d="http://xamarin.com/schemas/2014/forms/design"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006" xmlns:local="clr-namespace:IURIS.MOVIL.Modelos_y_clases"
    mc:Ignorable="d"
    x:Class="IURIS.MOVIL.PageRecuperarCuenta" Title="Recuperar
cuenta">
    <ContentPage.Content>
        <ScrollView>
            <Grid>
                <Grid.RowDefinitions>
                    <RowDefinition Height="10"/>
                    <RowDefinition Height="auto"/>
                    <RowDefinition Height="auto"/>
                    <RowDefinition Height="auto"/>
                    <RowDefinition Height="auto"/>
                    <RowDefinition Height="auto"/>
                    <RowDefinition Height="50"/>
                </Grid.RowDefinitions>

                <StackLayout Orientation="Horizontal" VerticalOptions="Center"
Grid.Row="1" Margin="10,0,15,0" >
                    <local:CustomEntry x:Name="EntryNombre"
Placeholder="Nombre..." Style="{StaticResource EntryCrearCuenta}"
Margin="5,0,0,0" ReturnType="Next" ></local:CustomEntry>
                </StackLayout>

                <StackLayout Orientation="Horizontal" VerticalOptions="Center"
Grid.Row="2" Margin="10,0,15,0" >
                    <local:CustomEntry x:Name="EntryApellidoPaterno"
Placeholder="Apellido Paterno..." Style="{StaticResource EntryCrearCuenta}"
Margin="5,0,0,0" ReturnType="Next" ></local:CustomEntry>
                </StackLayout>
            </Grid>
        </ScrollView>
    </ContentPage.Content>
</ContentPage>

```

```

        <StackLayout Orientation="Horizontal" VerticalOptions="Center"
Grid.Row="3" Margin="10,0,15,0" >
            <local:CustomEntry x:Name="EntryApellidoMaterno"
Placeholder="Apellido Materno..." Style="{StaticResource EntryCrearCuenta}"
Margin="5,0,0,0" ReturnType="Next" ></local:CustomEntry>
        </StackLayout>

        <StackLayout Orientation="Horizontal" VerticalOptions="Center"
Grid.Row="4" Margin="10,0,15,0" >
            <local:CustomEntry x:Name="EntryCorreoElectronico"
Placeholder="Correo electrónico..." Style="{StaticResource EntryCrearCuenta}"
Margin="5,0,0,0" Completed="btnBuscar_Clicked" ></local:CustomEntry>
        </StackLayout>

        <!--<StackLayout Orientation="Horizontal"
VerticalOptions="Center" Grid.Row="5" Margin="10,0,15,0" >
            <Entry x:Name="EntryNumeroDeTelefono" MaxLength="10"
Keyboard="Numeric" Placeholder="Numero de telefono..." Style="{StaticResource
EntryCrearCuenta}" Margin="5,0,0,0"></Entry>
        </StackLayout-->

        <StackLayout VerticalOptions="Center"
HorizontalOptions="Center" Grid.Row="5" Margin="10,0,15,0">

            <Frame
                Grid.Column="0"
                BorderColor="White"
                Padding="1,1,1,1"
                HasShadow="True"
                IsClippedToBounds="True">

                <Button Text="Buscar" x:Name="btnBuscar"
Style="{StaticResource buttonSecundario}" WidthRequest="200"
Clicked="btnBuscar_Clicked"></Button>
            </Frame>
        </StackLayout>

    </Grid>
</ScrollView>
</ContentPage.Content>
</ContentPage>

```

PageRecuperarCuenta.xaml.cs

```

using IURIS.BIZ;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Xamarin.Essentials;
using Xamarin.Forms;

```



```

using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class PageRecuperarCuenta : ContentPage
    {
        IManejadorDeUsuarioAplicacion manejadorDeUsuarioAplicacion;
        public Usuarios User=null;
        public PageRecuperarCuenta()
        {
            InitializeComponent();
            manejadorDeUsuarioAplicacion = new ManejadorDeUsuarioAplicacion(new
RepositorioGenerico<Usuarios>(true));
        }

        private async void btnBuscar_Clicked(object sender, EventArgs e)
        {
            if(string.IsNullOrEmpty(EntryApellidoMaterno.Text) ||
string.IsNullOrEmpty(EntryApellidoPaterno.Text) ||
string.IsNullOrEmpty(EntryNombre.Text) ||
string.IsNullOrEmpty(EntryCorreoElectronico.Text))
            {
                await DisplayAlert("Error", "Faltan datos por ingresar",
"Aceptar");
                return;
            }

            //User =
manejadorDeUsuarioAplicacion.NoRecuerdoMiContrasenias(EntryNombre.Text,
EntryApellidoPaterno.Text, EntryApellidoMaterno.Text,
EntryCorreoElectronico.Text);

            //if ( User!= null)
            //{
                //    string Nombre = string.Format("{0} {1}\ntu contraseña es:
{2}", EntryNombre.Text, EntryApellidoPaterno.Text,
User.Contrasenia.ToString());
                //    await DisplayAlert("Usuario", Nombre, "Aceptar");
                //    await Navigation.PopAsync();
                //    await Navigation.PushAsync(new PageInicioDeSesion(), true);
                //}
            //else
            //{
                //    await DisplayAlert("Usuario", "Usuario no encontrado\n por
favor verifique sus datos", "Aceptar");
                //}

        }
    }
}

```

Apartado Detail

PageMotrarCapitulosConCodigos.xaml

```
<?xml version="1.0" encoding="utf-8" ?>

<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:d="http://xamarin.com/schemas/2014/forms/design"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    xmlns:local="clr-namespace:IURIS.MOVIL.Modelos_y_clases"
    xmlns:pvc="clr-namespace:Xamarin.Forms.PancakeView;assembly=Xamarin.Forms.PancakeView"
    mc:Ignorable="d"
    x:Class="IURIS.MOVIL.Detail.PageMotrarCapitulosConCodigos"
    xmlns:controls="clr-namespace:MarcTron.Plugin.Controls;assembly=Plugin.MtAdmob"
    xmlns:extensions="http://xamarin.com/schemas/2020/toolkit">

    <ContentPage.Resources>
        <ResourceDictionary>
            <local:ByteArrayToImageSourceConverter x:Key="ByteArrayToImage" />
        </ResourceDictionary>
    </ContentPage.Resources>

    <NavigationPage.TitleView >
        <FlexLayout Direction="Row">

            <local:CustomEntry

                x:Name="SearchViewDetailTitle"

                VerticalOptions="Center"
```

```
HorizontalOptions="Start"
HorizontalTextAlignment="Start"
Placeholder="Titulo, palabras clave..."
PlaceholderColor="White"
TextColor="White"
ClearButtonVisibility="WhileEditing"
BackgroundColor="Transparent"
TextChanged="SearchViewDetailTitle_TextChanged"
Completed="SearchViewDetailTitle_SearchButtonPressed"
ReturnType="Search"
IsVisible="false"
IsSpellCheckEnabled="True"/>
<!--Margin="-50,0,0,0"-->
```

```
<Label x:Name="lblNumResultados" HorizontalOptions="End"
HorizontalTextAlignment="End" Margin="0,0,10,0" IsVisible="false" Style="{StaticResource
TextoAMostrarColorBlanco}" VerticalOptions="Center" VerticalTextAlignment="Center"
FontSize="Body"></Label>
```

```
<Grid ColumnDefinitions="*,Auto" ColumnSpacing="5"
x:Name="GridTituloEIMGBuscador">
```

```
<FlexLayout Grid.Column="0" Direction="Column" AlignContent="Center"
AlignItems="Center" Margin="0,0,0,5">
```

```
<Label x:Name="lblTitle" MaxLines="1" LineBreakMode="MiddleTruncation"
LineHeight="10" Style="{StaticResource TextoAMostrarColorBlanco}"
HorizontalTextAlignment="Center" HorizontalOptions="Center">
```

```
<Label.GestureRecognizers>
```

```
<TapGestureRecognizer Tapped="TapGestureRecognizer_Tapped_7"/>
```

```
</Label.GestureRecognizers>
```

```
</Label>
```

```
<Label x:Name="lblCodigo" Style="{StaticResource TextoAMostrarColorBlanco}"
FontSize="Small" VerticalOptions="End" VerticalTextAlignment="End" HorizontalOptions="Center"
HorizontalTextAlignment="Center"/>
```

```
</FlexLayout>
```

```
<Image Grid.Column="1" Source="Buscador.png" WidthRequest="20"
HorizontalOptions="End" HeightRequest="20" Margin="10,0,20,0">
```

```
<Image.GestureRecognizers>
```

```
<TapGestureRecognizer Tapped="TapGestureRecognizer_Tapped"/>
```

```
</Image.GestureRecognizers>
```

```
</Image>
```

```
</Grid>
```

```
</FlexLayout>
```

```
</NavigationPage.TitleView>
```

```
<Grid>
```

```
<FlexLayout AlignItems="Start" JustifyContent="Start" >
```

```
<StackLayout FlexLayout.Shrink="1.5" FlexLayout.Grow="1" FlexLayout.Basis="40%">
```

```
<CarouselView x:Name="cIltionCapitulos" ItemsSource="{Binding ListaCapitulos}"
IndicatorView="indicatorView" Margin="10,5,10,0"
PositionChanged="cIltionCapitulos_PositionChanged" BindingContext="{Binding .}" >
```

```
<CarouselView.EmptyView>
```

```
<StackLayout>
```

```
<Label FontAttributes="Bold" FontSize="Large" HorizontalOptions="Center"
Style="{StaticResource TextoAMostrarColorRojo}">No se encontraron datos</Label>
```

```
</StackLayout>
```

```
</CarouselView.EmptyView>
```

```
<CarouselView.ItemsLayout>
```

```

        <LinearLayout Orientation="Horizontal" SnapPointsAlignment="Center"
SnapPointsType="Mandatory" />

    </CarouselView.ItemsLayout>

    <CarouselView.ItemTemplate>

        <DataTemplate >

            <ContentView >

                <FlexLayout JustifyContent="Center" AlignContent="Center"
AlignItems="Center" >

                    <Label Text="{Binding NumCapitulo}" FontSize="17" Style="{StaticResource
TextoAMostrarColorRojo}" FlexLayout.Shrink="0" HorizontalTextAlignment="Center"></Label>

                    <Label Text="{Binding NombreCapitulo}" FontSize="Subtitle"
Style="{StaticResource TextoAMostrarColorRojo}" MaxLines="4"
HorizontalTextAlignment="Center"></Label>

                </FlexLayout>

                <ContentView.GestureRecognizers>

                    <TapGestureRecognizer Tapped="TapGestureRecognizer_Tapped_3"
BindingContext="{Binding .}" />

                </ContentView.GestureRecognizers>

            </ContentView>

        </DataTemplate>

    </CarouselView.ItemTemplate>

</CarouselView>

<IndicatorView x:Name="indicatorView"

    IndicatorColor="{StaticResource SecondaryColor}"

    SelectedIndicatorColor="{StaticResource PrimaryColor}"

    HorizontalOptions="Center"

        VerticalOptions="Start" MaximumVisible="3"

/>

</StackLayout>

```

```
<RefreshView IsRefreshing="{Binding IsRefreshing}" Command="{Binding RefreshCommand}" FlexLayout.Shrink="2.2" FlexLayout.Grow="1" Margin="0,0,0,60">
```

```
<CollectionView x:Name="cllctionArticulos" ItemsSource="{Binding ListaArticulos}" Margin="10">
```

```
<CollectionView.EmptyView>
```

```
<StackLayout>
```

```
<Label FontAttributes="Bold" FontSize="Large" HorizontalOptions="Center" Style="{StaticResource TextoAMostrarColorRojo}">No se encontraron datos</Label>
```

```
</StackLayout>
```

```
</CollectionView.EmptyView>
```

```
<CollectionView.ItemTemplate>
```

```
<DataTemplate>
```

```
<Grid RowDefinitions="auto,auto,auto" RowSpacing="0" >
```

```
<StackLayout Grid.Row="0" Orientation="Horizontal" VerticalOptions="Start" Spacing="5">
```

```
<Image Source="LapizNota.png" WidthRequest="25" HeightRequest="25" IsVisible="{Binding NotaAdjunta}" BindingContext="{Binding .}" VerticalOptions="Center" HorizontalOptions="Start">
```

```
<Image.GestureRecognizers>
```

```
<TapGestureRecognizer Tapped="TapGestureRecognizer_Tapped_2"/>
```

```
</Image.GestureRecognizers>
```

```
</Image>
```

```
<local:CustomLabeJustify FontSize="17" HorizontalTextAlignment="Start" Style="{StaticResource TextoAMostrarColorRojo}" FontAttributes="Bold" HorizontalOptions="StartAndExpand" VerticalOptions="Center">
```

```
<local:CustomLabeJustify.Text>
```

```
<MultiBinding StringFormat="{0} {1}">
```

```
<Binding Path="NumArticulo" />
```

```

        <Binding Path="NombreArticulo" />

    </MultiBinding>

    </local:CustomLabelJustify.Text>

</local:CustomLabelJustify>

    <Image Source="Tres_Puntos_Rojos.png" WidthRequest="25"
HeightRequest="25" BindingContext="{Binding .}" VerticalOptions="Center"
HorizontalOptions="End">

        <Image.GestureRecognizers>

            <TapGestureRecognizer Tapped="TapGestureRecognizer_Tapped_4"/>

        </Image.GestureRecognizers>

    </Image>

</StackLayout>

    <local:CustomLabelJustify BindingContext="{Binding .}" FontSize="Subtitle"
Grid.Row="1" Margin="0,10, 0,0" Style="{StaticResource TextoAMostrarColorRojo}">

        <local:CustomLabelJustify.Triggers>

            <DataTrigger TargetType="local:CustomLabelJustify" Binding="{Binding
TieneColorDeTexto}" Value="true">

                <Setter Property="IsVisible" Value="false"/>

            </DataTrigger>

        </local:CustomLabelJustify.Triggers>

        <local:CustomLabelJustify.FormattedText>

            <FormattedString>

                <Span Text="{Binding Contenido}"/>

            </FormattedString>

        </local:CustomLabelJustify.FormattedText>

    </local:CustomLabelJustify>

```

```
<StackLayout BindableLayout.ItemsSource="{Binding Subrayados}"
BindingContext="{Binding .}" Grid.Row="1" Margin="0,10, 0,0" IsVisible="{Binding
TieneColorDeTexto}" Spacing="-1" >
```

```
<BindableLayout.ItemTemplate>
```

```
<DataTemplate>
```

```
<local:CustomLabeJustify BindingContext="{Binding .}"
FontSize="Subtitle" Style="{StaticResource TextoAMostrarColorRojo}" >
```

```
<local:CustomLabeJustify.FormattedText>
```

```
<FormattedString>
```

```
<Span Text="{Binding TextoContenidoAnterior}" />
```

```
<Span Text="{Binding TextoContenidoSeleccionado}"
BackgroundColor="{Binding ColorTextoHex}" TextColor="Black" />
```

```
<Span Text="{Binding TextoContenidoDespues}" />
```

```
</FormattedString>
```

```
</local:CustomLabeJustify.FormattedText>
```

```
</local:CustomLabeJustify>
```

```
</DataTemplate>
```

```
</BindableLayout.ItemTemplate>
```

```
</StackLayout>
```

```
<CarouselView ItemsSource="{Binding Fotografia}" Grid.Row="2"
HorizontalOptions="Center" IsVisible="{Binding FotoAdjunta}" FlexLayout.Basis="20%">
```

```
<CarouselView.ItemTemplate>
```

```
<DataTemplate >
```

```
<!--<StackLayout Orientation="Vertical" HorizontalOptions="Center"
VerticalOptions="Start" Padding="50,10">-->
```

```
<FlexLayout JustifyContent="Start" AlignContent="Center"
AlignItems="Center" Padding="50,10" >
```



```
        <Label Text="{Binding Descripcion}" Style="{StaticResource
TextoAMostrarColorRojo}" HorizontalOptions="Center" HorizontalTextAlignment="Center"
FlexLayout.Shrink="0"></Label>
```

```
        <Image Source="{Binding Foto, Converter={StaticResource
ByteArrayToImage}}" HorizontalOptions="Center" FlexLayout.Shrink="1"/>
```

```
    </FlexLayout>
```

```
  </DataTemplate>
```

```
</CarouselView.ItemTemplate>
```

```
</CarouselView>
```

```
</Grid>
```

```
</DataTemplate>
```

```
</CollectionView.ItemTemplate>
```

```
</CollectionView>
```

```
</RefreshView>
```

```
</FlexLayout>
```

```
<pv:PancakeView HorizontalOptions="FillAndExpand" VerticalOptions="End">
```

```
  <pv:PancakeView.BackgroundGradientStops>
```

```
    <pv:GradientStopCollection>
```

```
      <pv:GradientStop Color="Transparent" Offset="0"/>
```

```
      <pv:GradientStop Color="Transparent" Offset="1"/>
```

```
    </pv:GradientStopCollection>
```

```
  </pv:PancakeView.BackgroundGradientStops>
```

```
  <Grid RowDefinitions="auto,auto" RowSpacing="2.5">
```

```
    <StackLayout Grid.Row="0" x:Name="stakColores" Orientation="Horizontal"
HorizontalOptions="Start" VerticalOptions="End" Margin="15,0,0,0" IsVisible="false">
```

```
<Ellipse Fill="#F7FE2E" Aspect="Uniform"
WidthRequest="40"
HeightRequest="40">
    <Ellipse.GestureRecognizers>
        <TapGestureRecognizer Tapped="EllipceColor"/>
    </Ellipse.GestureRecognizers>
</Ellipse>
```

```
<Ellipse Fill="#60FB36" Aspect="Uniform"
WidthRequest="40"
HeightRequest="40">
    <Ellipse.GestureRecognizers>
        <TapGestureRecognizer Tapped="EllipceColor"/>
    </Ellipse.GestureRecognizers>
</Ellipse>
```

```
<Ellipse Fill="#38E9FB" Aspect="UniformToFill"
WidthRequest="40"
HeightRequest="40">
    <Ellipse.GestureRecognizers>
        <TapGestureRecognizer Tapped="EllipceColor"/>
    </Ellipse.GestureRecognizers>
</Ellipse>
```

```
<Ellipse Fill="#FAC437" Aspect="UniformToFill"
WidthRequest="40"
HeightRequest="40">
    <Ellipse.GestureRecognizers>
```

```

        <TapGestureRecognizer Tapped="EllipceColor"/>
    </Ellipse.GestureRecognizers>
</Ellipse>

<Ellipse Fill="#FB39EB" Aspect="UniformToFill"
WidthRequest="40"
HeightRequest="40">
    <Ellipse.GestureRecognizers>
        <TapGestureRecognizer Tapped="EllipceColor"/>
    </Ellipse.GestureRecognizers>
</Ellipse>

</StackLayout>

<extensions:Expander Grid.Row="0" x:Name="expandr" IsEnabled="False"
Rotation="180" HorizontalOptions="End" VerticalOptions="End" State="Expanding"
    ExpandAnimationEasing="{x:Static Easing.SinOut}"
ExpandAnimationLength="1"
CollapseAnimationEasing="{x:Static Easing.SinOut}"
CollapseAnimationLength="1">
    <extensions:Expander.Header >
        <Image Source="Mas.png" WidthRequest="35" HeightRequest="35">
            <Image.GestureRecognizers>
                <TapGestureRecognizer Tapped="ExpaderForPlus"/>
            </Image.GestureRecognizers>
        </Image>
    </extensions:Expander.Header>
    <extensions:Expander.ContentTemplate>
        <DataTemplate>

```

```

        <ContentView Margin="0,15,0,0">
            <StackLayout x:Name="expanding" >
                <Image Source="Save.png" HeightRequest="35" WidthRequest="35"
Rotation="180">
                    <Image.GestureRecognizers>
                        <TapGestureRecognizer Tapped="TapGestureRecognizer_Tapped_5"/>
                    </Image.GestureRecognizers>
                </Image>
                <Image Source="eraser.png" HeightRequest="35" WidthRequest="35"
Rotation="180">
                    <Image.GestureRecognizers>
                        <TapGestureRecognizer Tapped="TapGestureRecognizer_Tapped_6"/>
                    </Image.GestureRecognizers>
                </Image>
            </StackLayout>
        </ContentView>
    </DataTemplate>
</extensions:Expander.ContentTemplate>
</extensions:Expander>

<controls:MTAdView Grid.Row="1" x:Name="myAds" IsVisible="false"
    AdId="ca-app-pub-2336879831564913/3076562549"
    VerticalOptions="End" HorizontalOptions="FillAndExpand" Margin="0,0,0,5">
    <controls:MTAdView.HeightRequest>
        <x:OnIdiom>
            <x:OnIdiom.Phone>50</x:OnIdiom.Phone>
            <x:OnIdiom.Tablet>90</x:OnIdiom.Tablet>
            <x:OnIdiom.Desktop>90</x:OnIdiom.Desktop>
        </x:OnIdiom>
    </controls:MTAdView.HeightRequest>

```

```
</controls:MTAdView>

</Grid>
</pv:PancakeView>
</Grid>
</ContentPage>
```

[PageMotrarCapitulosConCodigos.xaml.cs](#)

```
using IURIS.COMMON.Entidades.Ley;
using IURIS.COMMON.Entidades.Ley.ComponentesDeLey;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
using IURIS.MOVIL.Views;
using IURIS.MOVIL.Views.ViewsVentanasEmergentes;
using IURIS.MOVIL.Utills;
using Rg.Plugins.Popup.Services;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Input;
using Xamarin.Forms;
using Xamarin.Forms.Xaml;
using IURIS.MOVIL.Modelos_y_clases;
using System.IO;
using Xamarin.Forms.Shapes;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion.ComponentesDeUsuario;
using IURIS.COMMON.Interfaces;
```

```

using IURIS.DAL;

using IURIS.BIZ;

using Plugin.Clipboard;

using Acr.UserDialogs;

using MarcTron.Plugin.Controls;

using Xamarin.Essentials;

using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;


namespace IURIS.MOVIL.Detail
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class PageMotrarCapitulosConCodigos : ContentPage
    {
        readonly Titulo _titulo;

        MyLey _MyLey;

        public bool _ArticuloSeleccionado;

        static int _NumRecultadosEncontrados = 0;

        public Capitulo _Capitulo;

        public Capitulo _CapituloFind;

        public Articulo _Articulo;

        public Articulo _ArticuloFin;

        bool isRefreshing;

        private string ColorHex;

        Ellipse Ellipse_Cargado = null;

        int numToques = 0;

        ClassAnuncio Anuncio = new ClassAnuncio();


        public PageMotrarCapitulosConCodigos(Titulo titulo, MyLey ley, MTAdView adView, Capitulo
CapAlnciar,Articulo articulo)

```

```

{
    InitializeComponent();
    _MyLey = ley;
    BindingContext = this;
    MainThread.BeginInvokeOnMainThread(async () =>
    {
        await Task.Delay(1500);
        myAds.IsVisible = true;
    });

    _titulo = titulo;
    _CapituloFind = CapAlniciar;
    _ArticuloFin = articulo;
    DatosAlnicializar();

}

```

```

public bool IsRefreshing
{
    get => isRefreshing;
    set
    {
        isRefreshing = value;
        OnPropertyChanged(nameof(IsRefreshing));
    }
}

```

```

public ICommand RefreshCommand
{
    get
    {
        return new Command(async () =>
        {
            try
            {
                if (_Capitulo!=null) ActualizarDatosArticulo(_Capitulo.ListaArticulos);
                else ActualizarDatosCapitulo(_titulo.ListaCapitulos);
                //await Task.Delay(1500); // Only to demonstrate refresh views..
            }
            finally
            {
                IsRefreshing = false;
            }
        });
    }
}

```

```

private void DatosAlInicializar()
{

```

```

    Anuncio.MostrarAnuncioPantallaParaTitulos();

```

```

    ActualizarDatosCapitulo(_titulo.ListaCapitulos);

```

```

    lblTitle.Text = _titulo.NombreTitulo;

```

```

    lblCodigo.Text = _MyLey.CodigoLey;

```

```

    clIctionArticulos.ItemsSource = null;

```



```

        if (_CapituloFind != null)
        {

            clIctionArticulos.ItemsSource = _titulo.ListaCapitulos.Find(s =>
s.id.Equals(_CapituloFind.id)).ListaArticulos;

            clIctionCapitulos.Position = _titulo.ListaCapitulos.IndexOf(_CapituloFind);

            if (_ArticuloFin != null)
            {
                List<Articulo> ar = new List<Articulo>() { _ArticuloFin };
                clIctionArticulos.ItemsSource = ar;
                clIctionArticulos.ScrollTo(_ArticuloFin, position: ScrollToPosition.Start);
            }
        }
    else
    {

        clIctionArticulos.ItemsSource =
_titulo.ListaCapitulos.FirstOrDefault().ListaArticulos;//modificar
        _Capitulo = _titulo.ListaCapitulos.FirstOrDefault();
    }

}

private void ActualizarDatosCapitulo(List<Capitulo> listaCapitulos)
{
    clIctionCapitulos.ItemsSource = null;

    clIctionCapitulos.ItemsSource = listaCapitulos;
}

```

```

private void ActualizarDatosArticulo(List<Articulo> listaArticulos)
{
    clIctionArticulos.SelectedItem = null;

    clIctionArticulos.ItemsSource = null;
    clIctionArticulos.ItemsSource = listaArticulos;
}

private void TapGestureRecognizer_Tapped(object sender, EventArgs e)
{
    MostrarSearch(true);
}

private void BuscarTexto(TextChangedEventArgs TextChange)
{
    //Revisar el por que los resultados se muestran en formato horizontal
    if (SearchViewDetailTitle.Text != null)
    {
        List<Articulo> _ListaArticulos= new List<Articulo>();
        List<Articulo> _ListaArticulosGenerico= new List<Articulo>();

        List<Capitulo> _ListaCapitulos = new List<Capitulo>();

        foreach (var item in _titulo.ListaCapitulos)
        {
            _ListaArticulosGenerico = item.ListaArticulos.ToList().Where(e =>
e.NumArticulo.ToUpper().Contains(TextChange.NewTextValue.ToUpper())) == true ||

```

```
e.NombreArticulo.ToUpper().Contains(TextChange.NewTextValue.ToUpper()) == true ||  
e.Contenido.ToUpper().Contains(TextChange.NewTextValue.ToUpper()) == true).ToList();
```

```
foreach (var ati in _ListaArticulosGenerico)  
{  
    _ListaArticulos.Add(ati);  
}  
}
```

```
_ListaCapitulos = _titulo.ListaCapitulos.ToList().Where(e =>  
e.NumCapitulo.ToUpper().Contains(TextChange.NewTextValue.ToUpper()) == true ||  
e.NombreCapitulo.ToUpper().Contains(TextChange.NewTextValue.ToUpper()) == true).ToList();
```

```
ActualizarDatosCapitulo(_ListaCapitulos);  
ActualizarDatosArticulo(_ListaArticulos);
```

```
_NumRecultadosEncontrados = _ListaArticulos.Count + _ListaCapitulos.Count;
```

```
IblNumResultados.Text = TextChange.NewTextValue == ""? "" :  
_NumRecultadosEncontrados.ToString();  
}
```

```
}
```

```
private void TapGestureRecognizer_Tapped_1(object sender, EventArgs e)  
{  
    MostrarSearch(false);  
}
```

```

private void SearchViewDetailTitle_TextChanged(object sender, TextChangedEventArgs e)
{
    BuscarTexto(e);
}

private void SearchViewDetailTitle_SearchButtonPressed(object sender, EventArgs e)
{
    MostrarSearch(false);
    SearchViewDetailTitle.Text = null;
}

private void MostrarSearch(bool v)
{
    SearchViewDetailTitle.IsVisible = v;
    lblNumResultados.IsVisible = v;
    lblNumResultados.Text = null;
    GridTituloEIMGBuscador.IsVisible = !v;
}

private async void TapGestureRecognizer_Tapped_2(object sender, EventArgs e)
{
    var articulo = ((Image)sender).BindingContext as Articulo;
    if (articulo == null) return;

    await PopupNavigation.Instance.PushAsync(new WindowOfEmergencyCrearNota(_titulo,
    articulo, _MyLey, _Capitulo), false);
}

```

```

private void clItionCapitulos_PositionChanged(object sender, PositionChangedEventArgs e)
{
    var Cap = ((CarouselView)sender).CurrentItem as Capitulo;
    if (!(Cap is Capitulo)) return;

    ActualizarDatosArticulo(Cap.ListaArticulos);
    _Capitulo = Cap;

    Anuncio.MostrarAnuncioPantallaParaTitulos();

}

```

```

private void TapGestureRecognizer_Tapped_3(object sender, EventArgs e)
{
    var Cap = ((ContentView)sender).BindingContext as Capitulo;

    if (!(Cap is Capitulo)) return;

    ActualizarDatosArticulo(Cap.ListaArticulos);
    _Capitulo = Cap;

    Anuncio.MostrarAnuncioPantalla();

}

```

```

private async void TapGestureRecognizer_Tapped_4(object sender, EventArgs e)
{
    var articulo = ((Image)sender).BindingContext as Articulo;
    if (articulo == null) return;

```

```

        _Articulo = articulo;

        await PopupNavigation.Instance.PushAsync(new WindowOfMenuAccion(_titulo, articulo,
        _MyLey, _Capitulo), false);

        expandr.IsEnabled = true;
    }

```

```

private void TapGestureRecognizer_Tapped_5Fotos(object sender, EventArgs e)

```

```

{
    var Fot = ((Image)sender).BindingContext as Fotografia;

    if (!(Fot is Fotografia)) return;

    //Stream stream = new MemoryStream(Fot.Foto);
    //image.Source = ImageSource.FromStream(stream);
    //((Image)sender).Source = ImageSource.FromStream(() => { return stream; });
    ((Image)sender).Margin= new Thickness(20,20,20,20);
    ((Image)sender).Source = ImageSource.FromStream(() => new MemoryStream(Fot.Foto));

}

```

```

private void ArticulosConFoto_PositionChanged(object sender, PositionChangedEventArgs e)

```

```

{
    var Fot = ((CarouselView)sender).BindingContext as Fotografia;

    if (!(Fot is Fotografia)) return;

}

```

```

private void EllipceColor(object sender, EventArgs e)

```

```

{
    if (Ellipse_Cargado != null)
    {
        Ellipse_Cargado.StrokeThickness = 0;
        Ellipse_Cargado.Stroke = Brush.Transparent;
    }
    ColorHex = ((SolidColorBrush)(((Ellipse)sender).Fill)).Color.ToHex();
    Ellipse_Cargado = ((Ellipse)sender);
    ((Ellipse)sender).Stroke = Brush.Black;
    ((Ellipse)sender).StrokeThickness = 4;
}

```

```

private async void ExpaderForPlus(object sender, EventArgs e)

```

```

{
    numToques += 1;
    if (numToques == 2)
    {
        expandr.IsEnabled = false;
        numToques = 0;
        _Articulo = null;
    }
    stakColores.IsVisible = stakColores.IsVisible==false? true:false;
    try
    {
        if (!expandr.IsExpanded)
        {

```

```

            await PopupNavigation.Instance.PushAsync(new WindowAlert("Selecciona tu color
favorito y guardalo/eliminalo"), false);

```

```

        await Task.Delay(4000);

        await PopupNavigation.Instance.PopAsync(false);
    }
}

catch (Exception ex)
{
    return;
}

}

private async void TapGestureRecognizer_Tapped_5(object sender, EventArgs e)
{
    try
    {

        //if (_Articulo == null) return;

        string txt;

        txt = await CrossClipboard.Current.GetTextAsync();

        if (_Articulo.Contenido.ToUpper().Contains(txt)) return;


        //var cap=_Capitulo.ListaArticulos.Where(r =>
r.Contenido.Contains(txt)).SingleOrDefault();

        if (ColorHex == null) return;

        if(!_Articulo.TieneColorDeTexto) _Articulo.TieneColorDeTexto=true;

        if (_Articulo.Subrayados == null) _Articulo.Subrayados = new List<Subrayado>();
    }
}

```



```
//if (_Articulo.Subrayados.Find(w => w.ColorTextoHex == ColorHex) != null)
//{
//    await DisplayAlert("Error", "Ya tienes ese color por favor elije otro", "ok");
//    return;
//}
```

Subrayado subrayado;

Subrayado subrayadoTemp;

int Items = _Articulo.Subrayados.Count();

if (Items < 1)

{

int x = _Articulo.Contenido.IndexOf(txt);

int y = txt.Length;

subrayado = new Subrayado()

{

ColorTextoHex = ColorHex,

TextoContenidoAnterior = _Articulo.Contenido.Substring(0, x),

TextoContenidoSeleccionado = txt,

TextoContenidoDespues = ""

};

subrayadoTemp = new Subrayado()

{

TextoContenidoAnterior = _Articulo.Contenido.Substring(x + y),

```

        ColorTextoHex = null,
        TextoContenidoDespues = "",
        TextoContenidoSeleccionado = ""
    };
}
else
{

    Subrayado sub = _Articulo.Subrayados[Items - 1];
    //if (_Articulo.Subrayados.Where(w => w.ColorTextoHex == ColorHex).Count() >= 1)
    //{

    //}

    int x = sub.TextoContenidoAnterioror.IndexOf(txt);
    int y = txt.Length;

    subrayado = new Subrayado()
    {
        ColorTextoHex = ColorHex,
        TextoContenidoAnterioror = sub.TextoContenidoAnterioror.Substring(0, x),
        TextoContenidoSeleccionado = txt,
        TextoContenidoDespues = ""
    };

    subrayadoTemp = new Subrayado()
    {

        TextoContenidoAnterioror = sub.TextoContenidoAnterioror.Substring(x + y),

```

```

        ColorTextoHex = null,
        TextoContenidoDespues = "",
        TextoContenidoSeleccionado = ""
    };

    _Articulo.Subrayados.Remove(sub);
}

_Articulo.Subrayados.Add(subrayado);
_Articulo.Subrayados.Add(subrayadoTemp);

if (await App.Database.UpdateUserAsync(App.MyUser))
{
    IsRefreshing = true;
    return;
}
else
{
    try
    {
        await PopupNavigation.Instance.PushAsync(new WindowAlert("Error", "No se han guardado los cambios", "Ok"), false);
        await Task.Delay(3000);
        await PopupNavigation.Instance.PopAsync(false);
    }
    catch (Exception)
    {
        return;
    }
}

```

```

        }
    }
}
catch (Exception ex)
{
    //await DisplayAlert("Error", "Error: " + ex.Message, "ok");

    try
    {
        await PopupNavigation.Instance.PushAsync(new
WindowAlert("Error",ex.Message,"Ok"), false);

        await Task.Delay(5000);

        await PopupNavigation.Instance.PopAsync(false);
    }
    catch (Exception)
    {

        return;
    }
    return;
}
}

```

```

private async void TapGestureRecognizer_Tapped_6(object sender, EventArgs e)
{
    try
    {

        if (ColorHex == null)

            return;
    }
}

```

```

if (!_Articulo.TieneColorDeTexto)

    return;

string txtABorrar= await CrossClipboard.Current.GetTextAsync();

Subrayado subrayado = _Articulo.Subrayados.Where(r =>
r.TextoContenidoSeleccionado.ToUpper().Contains(txtABorrar.ToUpper()))).First();

if (subrayado == null)

    return;

int Index = _Articulo.Subrayados.IndexOf(subrayado);

if (_Articulo.Subrayados.Where(r => r.ColorTextoHex != null).Count() == 1)
{
    _Articulo.TieneColorDeTexto = false;
    _Articulo.Subrayados = null;

}
else
{
    _Articulo.Subrayados[Index + 1].TextoContenidoAnterior = string.Format("{0}{1}{2}",
_Articulo.Subrayados[Index].TextoContenidoAnterior,
_Articulo.Subrayados[Index].TextoContenidoSeleccionado, _Articulo.Subrayados[Index +
1].TextoContenidoAnterior);

    _Articulo.Subrayados[Index + 1].TextoContenidoDespues = "";

    if (_Articulo.Subrayados[Index + 1].TextoContenidoSeleccionado == "" &&
_Articulo.Subrayados[Index + 1].ColorTextoHex == null)

    {

        _Articulo.Subrayados[Index + 1].TextoContenidoSeleccionado = "";

        _Articulo.Subrayados[Index + 1].ColorTextoHex = null;
    }
}

```

```

    }

    _Articulo.Subrayados.Remove(subrayado);
}

if (await App.Database.UpdateUserAsync(App.MyUser))
    IsRefreshing = true;
}

catch (Exception)
{
    //await DisplayAlert("Error", "No se ha podido borrar el subrayado,\npor favor intente
mas tarde", "ok");

    try
    {
        await PopupNavigation.Instance.PushAsync(new WindowAlert("Error", "No se ha
podido borrar el subrayado,\npor favor intente mas tarde", "Ok"), false);

        await Task.Delay(5000);

        await PopupNavigation.Instance.PopAsync(false);
    }

    catch (Exception)
    {

        return;
    }
}
}
}

```

```

private async void TapGestureRecognizer_Tapped_7(object sender, EventArgs e)
{
    try
    {

```

```

        await PopupNavigation.Instance.PushAsync(new WindowAlert(_titulo.NombreTitulo),
false);

        await Task.Delay(2500);

        await PopupNavigation.Instance.PopAsync(false);
    }
    catch (Exception)
    {
        return;
    }
}
}
}
}

```

ViewDetail.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:d="http://xamarin.com/schemas/2014/forms/design"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006" xmlns:pv="clr-
namespace:Xamarin.Forms.PancakeView;assembly=Xamarin.Forms.PancakeView"
    xmlns:Local="clr-namespace:IURIS.MOVIL.Modelos_y_clases" xmlns:controls="clr-
namespace:MarcTron.Plugin.Controls;assembly=Plugin.MtAdmob"
    mc:Ignorable="d"
    x:Class="IURIS.MOVIL.Detail.ViewDetail" >

    <!--<VisualStateManager.VisualStateGroups>
        <VisualStateGroup x:Name="CommonStates">
            <VisualState x:Name="Normal"/>
            <VisualState x:Name="Selected">
                <VisualState.Setters>
                    <Setter Property="BackgroundColor" Value="{StaticResource
PrimaryColor}" />
                    <Setter Property="Opacity" Value="0,2"/>
                </VisualState.Setters>
            </VisualState>
        </VisualStateGroup>
    </VisualStateManager.VisualStateGroups>-->

    <ContentPage.Resources>
        <Style TargetType="ContentView">
            <Setter Property="VisualStateManager.VisualStateGroups">
                <VisualStateGroupList>

```

```

        <VisualStateGroup x:Name="CommonStates">
            <VisualState x:Name="Normal" />
            <VisualState x:Name="Selected">
                <VisualState.Setters>
                    <Setter Property="BackgroundColor"
Value="LightBlue" />
                    <Setter Property="Opacity" Value="0.8"/>
                </VisualState.Setters>
            </VisualState>
        </VisualStateGroup>
    </VisualStateGroupList>
</Setter>
</Style>
</ContentPage.Resources>

<NavigationPage.TitleView>
    <FlexLayout Direction="Row" Wrap="Wrap">

        <Local:CustomEntry

            x:Name="SearchViewDetailTitle"
            VerticalOptions="Center"
            HorizontalOptions="StartAndExpand"
            Placeholder="Titulo, palabras clave..."
            PlaceholderColor="White"
            TextColor="White"
            ClearButtonVisibility="WhileEditing"
            BackgroundColor="Transparent"
            TextChanged="SearchViewDetailTitle_TextChanged"
            Completed="SearchViewDetailTitle_SearchButtonPressed"
            ReturnType="Search"
            IsSpellCheckEnabled="True"
            IsVisible="false"
        />

        <Label x:Name="lblNumResultados" HorizontalOptions="End"
Margin="0,0,10,3" IsVisible="false" Style="{StaticResource
TextoAMostrarColorBlanco}" VerticalOptions="Center"
VerticalTextAlignment="Center" FontSize="Body"></Label>

        <Grid ColumnDefinitions="*,Auto" ColumnSpacing="5" >
            <FlexLayout Grid.Column="0" x:Name="tituloLey"
Direction="Column" AlignItems="Center" AlignContent="Center" Margin="0,0,0,5">
                <Label x:Name="lblTitle" MaxLines="1"
LineBreakMode="MiddleTruncation" LineHeight="10" Style="{StaticResource
TextoAMostrarColorBlanco}" HorizontalTextAlignment="Center"
HorizontalOptions="Center">
                    <Label.GestureRecognizers>
                        <TapGestureRecognizer
Tapped="TapGestureRecognizer_Tapped_1"/>
                    </Label.GestureRecognizers>
                </Label>
                <Label x:Name="lblCodigo" Style="{StaticResource
TextoAMostrarColorBlanco}" FontSize="Small" VerticalOptions="End"
VerticalTextAlignment="End" HorizontalOptions="Center"
HorizontalTextAlignment="Center" Margin="0,2,0,10"/>
            </FlexLayout>

```



```

                <Image x:Name="IMGBuscador" Grid.Column="1"
Source="Buscador.png" WidthRequest="20" HorizontalOptions="End"
HeightRequest="20" Margin="10,0,20,0">
                <Image.GestureRecognizers>
                <TapGestureRecognizer
Tapped="TapGestureRecognizer_Tapped"/>
                </Image.GestureRecognizers>
            </Image>
        </Grid>
    </FlexLayout>
</NavigationPage.TitleView>

<ContentPage.Content>
    <Grid>

        <CollectionView x:Name="ClItionTitulos" SelectionMode="Single"
SelectionChanged="ClItionTitulos_SelectionChanged" Margin="0,0,0,60">
            <CollectionView.EmptyView>
                <StackLayout>
                    <Label FontAttributes="Bold" FontSize="Large"
HorizontalOptions="Center" Style="{StaticResource TextoAMostrarColorRojo}">No
se encontraron datos</Label>
                </StackLayout>
            </CollectionView.EmptyView>
            <CollectionView.ItemsLayout>
                <LinearItemsLayout Orientation="Vertical" ItemSpacing="10"
SnapPointsAlignment="Center"/>
            </CollectionView.ItemsLayout>
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <ContentView >
                        <StackLayout Orientation="Horizontal"
HorizontalOptions="CenterAndExpand" VerticalOptions="Center" Margin="10,5" >
                            <Label Style="{StaticResource
TextoAMostrarColorRojo}" >
                                <VisualStateManager.VisualStateGroups>
                                    <VisualStateGroup
x:Name="CommonStates">
                                        <VisualState x:Name="Normal"/>
                                        <VisualState x:Name="Selected">
                                            <VisualState.Setters>
                                                <Setter
Property="TextColor" Value="White" />
                                            </VisualState.Setters>
                                        </VisualState>
                                    </VisualStateGroup>
                                </VisualStateManager.VisualStateGroups>
                                <Label.Text>
                                    <MultiBinding StringFormat="{{0}} -
{1}">
                                        <Binding Path="NumTitulo" />
                                        <Binding Path="NombreTitulo" />
                                    </MultiBinding>
                                </Label.Text>
                            </Label>
                        </StackLayout>
                    </ContentView>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </Grid>
</ContentPage.Content>

```

```

        </CollectionView.ItemTemplate>
    </CollectionView>

    <CollectionView x:Name="ClktionDarEncontrados"
SelectionMode="Single" Style="{StaticResource collection}" Margin="0,0,0,60"
IsVisible="false" SelectionChanged="ClktionDarEncontrados_SelectionChanged">
        <CollectionView.EmptyView>
            <StackLayout>
                <Label FontAttributes="Bold" FontSize="Large"
HorizontalOptions="Center" Style="{StaticResource TextoAMostrarColorRojo}">No
se encontraron datos</Label>
            </StackLayout>
        </CollectionView.EmptyView>
        <CollectionView.ItemsLayout>
            <LinearItemsLayout Orientation="Vertical" ItemSpacing="10"
SnapPointsAlignment="Center"/>
        </CollectionView.ItemsLayout>
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <ContentView >
                    <StackLayout BindableLayout.ItemsSource="{Binding
Subrayados}" BindingContext="{Binding .}" Margin="0,10, 0,0" Spacing="-1" >
                        <BindableLayout.ItemTemplate>
                            <DataTemplate>
                                <Label BindingContext="{Binding .}"
FontSize="Subtitle" Style="{StaticResource TextoAMostrarColorRojo}" >
                                    <Label.FormattedText>
                                        <FormattedString>
                                            <Span Text="{Binding
TextoContenidoAnterior}" />
                                                <Span Text="{Binding
TextoContenidoSeleccionado}" BackgroundColor="{Binding ColorTextoHex}"
TextColor="Black" />
                                                    <Span Text="{Binding
TextoContenidoDespues}" />
                                                        </FormattedString>
                                                    </Label.FormattedText>
                                                </Label>
                                            </DataTemplate>
                                        </BindableLayout.ItemTemplate>
                                    </StackLayout>
                                </ContentView>
                            </DataTemplate>
                        </CollectionView.ItemTemplate>
                    </CollectionView>

                    <controls:MTAdView x:Name="myAds" IsVisible="false"
                        AdsId="ca-app-pub-
2336879831564913/3076562549"
                        VerticalOptions="End"
HorizontalOptions="EndAndExpand">
                        <controls:MTAdView.HeightRequest>
                            <x:OnIdiom>
                                <x:OnIdiom.Phone>50</x:OnIdiom.Phone>
                                <x:OnIdiom.Tablet>90</x:OnIdiom.Tablet>
                                <x:OnIdiom.Desktop>90</x:OnIdiom.Desktop>

```

```

        </x:OnIdiom>
        </controls:MTAdView.HeightRequest>
    </controls:MTAdView>

    </Grid>
</ContentPage.Content>

</ContentPage>

```

ViewDetail.xaml.cs

```

using Acr.UserDialogs;

using IURIS.COMMON.Entidades.Ley;

using IURIS.COMMON.Entidades.Ley.ComponentesDeLey;

using IURIS.COMMON.Entidades.UsuariosDeAplicacion;

using IURIS.MOVIL.Modelos_y_clases;

using IURIS.MOVIL.Modelos_y_clases.DB_Local;

using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;

using IURIS.MOVIL.Modelos_y_clases.Tools;

using IURIS.MOVIL.Utills;

using IURIS.MOVIL.Views;

using IURIS.MOVIL.Views.ViewsVentanasEmergentes;

using MarcTron.Plugin;

using Rg.Plugins.Popup.Services;

using System;

using System.Collections.Generic;

using System.Collections.ObjectModel;

using System.Linq;

using System.Text;

using System.Threading.Tasks;

using Xamarin.Essentials;

using Xamarin.Forms;

using Xamarin.Forms.Xaml;

```

```

namespace IURIS.MOVIL.Detail
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class ViewDetail : ContentPage
    {

        protected override bool OnBackButtonPressed()
        {
            if (ClItionDarEncontrados.IsVisible)
            {
                ClItionDarEncontrados.IsVisible = false;
                ClItionTitulos.IsVisible = true;
                return true;
            }
            else
            {
                System.Diagnostics.Process.GetCurrentProcess().CloseMainWindow();
                return false;
            }
        }

        ClassAnuncio Anuncio = new ClassAnuncio();
        MyLey _MyLey;

        public ViewDetail()
        {
            InitializeComponent();

```

```
DatosAlInicializar();
```

```
MainThread.BeginInvokeOnMainThread(async () =>
```

```
{  
    await Task.Delay(1500);  
    myAds.IsVisible = true;  
});
```

```
Anuncio.MostrarAnuncioPantalla();
```

```
var test = CrossMTAdmob.Current.IsInterstitialLoaded().ToString();
```

```
CrossMTAdmob.Current.ShowInterstitial();
```

```
CrossMTAdmob.Current.LoadInterstitial("ca-app-pub-2336879831564913/4601945943");
```

```
}
```

```
private void DatosAlInicializar()
```

```
{  
    _MyLey = App.MyUser.MisLeyes.Where(e => e.CodigoLey ==  
App.MyUser.MiUltimaLeyCargada).SingleOrDefault();
```

```
lblTitle.Text = _MyLey.NombreLey;
```

```
lblCodigo.Text = _MyLey.CodigoLey;
```

```
ActualizarDatos(_MyLey.ListaDeTitulos);
```

```
ClItionTitulos.SelectedItem = null;
```

```
}
```

```
private void ActualizarDatos(List<Titulo> _MiLista)
```

```
{  
    ClItionTitulos.ItemsSource = null;
```

```

        ClItionTitulos.ItemsSource = _MiLista;
    }

    private void MostrarSearch(bool v)
    {
        SearchViewDetailTitle.IsVisible = v;
;        tituloLey.IsVisible = !v;
        IMGBuscador.IsVisible = !v;
        lblNumResultados.IsVisible = v;
        lblNumResultados.Text = null;
    }

    private async void ClItionTitulos_SelectionChanged(object sender,
    SelectionChangedEventArgs e)
    {
        Titulo titulo = ClItionTitulos.SelectedItem as Titulo;
        if (titulo == null || titulo.ListaCapitulos == null || titulo.ListaCapitulos.Count == 0)
        {
            ClItionTitulos.SelectedItem = null;
            return;
        }
        MostrarSearch(false);

        await Navigation.PushAsync(new PageMotrarCapitulosConCodigos(titulo, _MyLey,
        myAds,null,null), false);

        ClItionTitulos.SelectedItem = null;
    }

```

```

private void TapGestureRecognizer_Tapped(object sender, EventArgs e)
{
    MostrarSearch(true);
}

private void BuscarTexto(TextChangedEventArgs TextChange)
{
    if (SearchViewDetailTitle.Text == null || TextChange.NewTextValue == "")
    {
        ClItionDarEncontrados.IsVisible = false;
        ClItionTitulos.IsVisible = true;
        return;
    }

    List<DatosDeSubrayado> _DatosEncontrados = new List<DatosDeSubrayado>();

    foreach (var item in _MyLey.ListaDeTitulos)
    {
        if (item.NumTitulo.ToUpper().Contains(TextChange.NewTextValue.ToUpper()) ||
            item.NombreTitulo.ToUpper().Contains(TextChange.NewTextValue.ToUpper()))
        {
            string cadena = string.Format("{0} - {1}", item.NumTitulo, item.NombreTitulo);
            _DatosEncontrados.Add(new DatosDeSubrayado()
            {
                id = item.id,
                tipo = "Titulo",
                Subrayados = new List<Subrayado>() { Getsubrayado(cadena,
                    TextChange.NewTextValue) }
            });
        }
    }
}

```

```

    }

    foreach (var con in item.ListaCapitulos)
    {
        if (con.NombreCapitulo.ToUpper().Contains(TextChange.NewTextValue.ToUpper()) ||
            con.NumCapitulo.ToUpper().Contains(TextChange.NewTextValue.ToUpper()))
        {

            string cadena = string.Format("{0} - {1}", con.NumCapitulo, con.NombreCapitulo);

            _DatosEncontrados.Add(new DatosDeSubrayado()
            {
                id = con.id,
                tipo = "Capitulo",
                Subrayados = new List<Subrayado>() { Getsubrayado(cadena,
                    TextChange.NewTextValue) }
            });
        }

        foreach (var arti in con.ListaArticulos)
        {
            if (arti.NombreArticulo.ToUpper().Contains(TextChange.NewTextValue.ToUpper()) ||
                arti.NumArticulo.ToUpper().Contains(TextChange.NewTextValue.ToUpper()) ||
                arti.Contenido.ToUpper().Contains(TextChange.NewTextValue.ToUpper()))
            {

                string cadena = string.Format("{0} - {1}\n{2}", arti.NumArticulo,
                    arti.NombreArticulo, arti.Contenido);

                _DatosEncontrados.Add(new DatosDeSubrayado
                {
                    id = arti.id,
                    tipo = "Articulo",

```



```
        Subrayados = new List<Subrayado>() { Getsubrayado(cadena,
        TextChange.NewTextValue) },
```

```
    });
```

```
    }
```

```
    }
```

```
    }
```

```
}
```

```
ClItionDarEncontrados.ItemsSource = _DatosEncontrados;
```

```
ClItionDarEncontrados.IsVisible = true;
```

```
ClItionTitulos.IsVisible = false;
```

```
//List<Titulo> titulos = new List<Titulo>();
```

```
//titulos =
```

```
// _MyLey.ListaDeTitulos.Where(
```

```
//     e => e.NumTitulo.ToUpper().Contains(TextChange.NewTextValue.ToUpper())
```

```
//     ||
```

```
//     e.NombreTitulo.ToUpper().Contains(TextChange.NewTextValue.ToUpper())
```

```
//     ).ToList();
```

```
//ActualizarDatos(titulos);
```

```
    lblNumResultados.Text = TextChange.NewTextValue == "" ? "" :
    _DatosEncontrados.Count.ToString();
```

```
}
```

```
private Subrayado Getsubrayado(string cadena, string txt)
```

```
{
```

```
    try
```

```
{
```

```

        int x = cadena.ToUpper().IndexOf(txt.ToUpper());
        int y = txt.Length;

        return new Subrayado()
        {
            ColorTextoHex = "#eef21f",
            TextoContenidoAnterior = cadena.Substring(0, x),
            TextoContenidoSeleccionado = txt,
            TextoContenidoDespues = cadena.Substring(x + y)
        };
    }
    catch (Exception)
    {
        return null;
    }
}

private void SearchViewDetailTitle_TextChanged(object sender, TextChangedEventArgs e)
{
    BuscarTexto(e);
}

private void SearchViewDetailTitle_SearchButtonPressed(object sender, EventArgs e)
{
    MostrarSearch(false);
    SearchViewDetailTitle.Text = null;
}

```

```

private async void TapGestureRecognizer_Tapped_1(object sender, EventArgs e)
{
    try
    {
        await PopupNavigation.Instance.PushAsync(new WindowAlert(_MyLey.NombreLey),
false);
        await Task.Delay(2500);
        await PopupNavigation.Instance.PopAsync(false);
    }
    catch (Exception)
    {
        return;
    }
}

```

```

private class DatosDeSubrayado
{
    public string id { get; set; }
    public string tipo { get; set; }
    public List<Subrayado> Subrayados { get; set; }
}

```

```

private async void ClItionDarEncontrados_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    if (ClItionDarEncontrados.SelectedItem == null) return;

    var DatosSub = (DatosDeSubrayado)ClItionDarEncontrados.SelectedItem;

```

```

if (DatosSub.tipo.Equals("Titulo")){
    var d=_MyLey.ListaDeTitulos.Find(g => g.id == DatosSub.id);
    ClItionTitulos.SelectedItem = d;
}

if (DatosSub.tipo.Equals("Capitulo"))
{
    Titulo titu=null;
    Capitulo NumCap=null;
    foreach (var tit in _MyLey.ListaDeTitulos)
    {
        titu = tit;
        NumCap = tit.ListaCapitulos.Find(capi => capi.id.Equals(DatosSub.id));
        break;
    }

    ClItionTitulos.SelectedItem = titu;

    await Navigation.PushAsync(new PageMotrarCapitulosConCodigos(titu, _MyLey,
myAds,NumCap,null), false);

}

if (DatosSub.tipo.Equals("Articulo"))
{
    Titulo titu = null;
    Capitulo NumCap = null;
    Articulo Numarticulo = null;
    foreach (var titulo in _MyLey.ListaDeTitulos)
    {

```

```

        foreach (var capitulo in titulo.ListaCapitulos)
        {

            foreach (var aticulo in capitulo.ListaArticulos)
            {
                if (aticulo.id.Equals(DatosSub.id))
                {
                    titu = titulo;
                    NumCap = capitulo;
                    Numarticulo = aticulo;
                    break;
                }
            }

        }

        ClItionTitulos.SelectedItem = titu;

        await Navigation.PushAsync(new PageMotrarCapitulosConCodigos(titu, _MyLey, myAds,
        NumCap, Numarticulo), false);
    }

    ClItionTitulos.SelectedItem = null;
}
}
}

```

Apartado Fonts

Archivo lucida_handwriting.ttf

Apartado Master

ViewMaster.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:d="http://xamarin.com/schemas/2014/forms/design"
             xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006" xmlns:pv="clr-
namespace:Xamarin.Forms.PancakeView;assembly=Xamarin.Forms.PancakeView"
             mc:Ignorable="d"
             x:Class="IURIS.MOVIL.Master.ViewMaster" Title="Master">
    <ContentPage.Content>
        <Grid>

            <FlexLayout JustifyContent="Start">

                <StackLayout Orientation="Horizontal"
                    BackgroundColor="{StaticResource PrimaryColor}" FlexLayout.Basis="7%">
                    <Image Source="Logo.png" Margin="15,5"/>
                    <Label x:Name="lblIDUser" Style="{StaticResource
LabelCrearCuenta}" FontAttributes="Bold" FontSize="Title"
VerticalTextAlignment="Center"/>
                </StackLayout>

                <Label Style="{StaticResource TextoAMostrarColorRojo}"
                    Margin="15,10,0,5" HorizontalTextAlignment="Start" Text="Inicio"
                    FontSize="Large" >
                    <Label.GestureRecognizers>
                        <TapGestureRecognizer
Tapped="TapGestureRecognizer_Tapped_2" />
                    </Label.GestureRecognizers>
                </Label>

                <Label Style="{StaticResource TextoAMostrarColorRojo}"
                    Margin="15,5" HorizontalTextAlignment="Start" Text="Clasificación
personalizada" FontSize="Large" >
                    <Label.GestureRecognizers>
                        <TapGestureRecognizer Tapped="TapGestureRecognizer_Tapped"
/>
                    </Label.GestureRecognizers>
                </Label>

                <Label Style="{StaticResource TextoAMostrarColorRojo}"
                    Margin="15,5" HorizontalTextAlignment="Start" Text="Mis apuntes"
                    FontSize="Large" >
                    <Label.GestureRecognizers>
                        <TapGestureRecognizer
Tapped="TapGestureRecognizer_Tapped_1" />
                    </Label.GestureRecognizers>
                </Label>
            </FlexLayout>
        </Grid>
    </ContentPage.Content>
</ContentPage>
```

```

        <Label Style="{StaticResource TextoAMostrarColorRojo}"
Margin="15,5" HorizontalTextAlignment="Start" Text="Cargar / cambiar ley"
FontSize="Large" >
        <Label.GestureRecognizers>
            <TapGestureRecognizer
Tapped="TapGestureRecognizer_Tapped_3" />
        </Label.GestureRecognizers>
    </Label>

    <!--
    <Label Margin="15,5" Text="Ajustes" Style="{StaticResource
TextoAMostrarColorRojo}" FontSize="Large">
        <Label.GestureRecognizers>
            <TapGestureRecognizer
Tapped="TapGestureRecognizer_Tapped_6" />
        </Label.GestureRecognizers>
    </Label>
-->

    <!--<Label Style="{StaticResource TextoAMostrarColorRojo}"
Margin="15,5" HorizontalTextAlignment="Start" VerticalTextAlignment="End"
Text="Cerrar sesión" FontSize="Large" >
        <Label.GestureRecognizers>
            <TapGestureRecognizer
Tapped="TapGestureRecognizer_Tapped_5" />
        </Label.GestureRecognizers>
    </Label>-->

</FlexLayout>

    <pv:PancakeView HorizontalOptions="FillAndExpand"
VerticalOptions="End" CornerRadius="0,50,0,0">
        <pv:PancakeView.BackgroundGradientStops>
            <pv:GradientStopCollection>
                <pv:GradientStop Color="{StaticResource PrimaryColor}"
Offset="0"/>
                <pv:GradientStop Color="{StaticResource PrimaryColor}"
Offset="1"/>
            </pv:GradientStopCollection>
        </pv:PancakeView.BackgroundGradientStops>
        <Label Margin="10,15,10,20" Text=" para mas espacio "
Style="{StaticResource TextoAMostrarColorBlanco }" FontSize="Large"
Padding="15">
            <Label.FormattedText>
                <FormattedString>
                    <Span Text="Solo cuentas con 2 espacios para leyes
de forma gratuita. &#10;"></Span>
                    <Span Text="Para solicitar mas espacio ponte en
contacto con: "></Span>
                    <Span Text="iurisespacio@gmail.com"
TextDecorations="Underline" FontSize="Body" FontAttributes="Bold">
                        <!--<Span.GestureRecognizers>
                            <TapGestureRecognizer
                        </Span.GestureRecognizers>-->
                    </Span>

```

```

        </FormattedString>
    </Label.FormattedText>
    <!--<Label.GestureRecognizers>
        <TapGestureRecognizer
Tapped="TapGestureRecognizer_Tapped_4" />
    </Label.GestureRecognizers-->
</Label>
</pv:PancakeView>
</Grid>
</ContentPage.Content>
</ContentPage>

```

ViewMaster.xaml.cs

```

using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
using IURIS.MOVIL.Detail;
using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using IURIS.MOVIL.Utills;
using IURIS.MOVIL.Views;
using IURIS.MOVIL.Views.ViewsCargarLey;
using IURIS.MOVIL.Views.ViewsCargarLey.TabbPage;
using IURIS.MOVIL.Views.ViewsCargarLey.ViewClasificaciones;
using IURIS.MOVIL.Views.ViewsVentanasEmergentes;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Timers;
using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Master
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class ViewMaster : ContentPage
    {
        public ViewMaster()
        {
            InitializeComponent();

            lblIDUser.Text = String.Format("IURIS Id: {0}", App.MyUser.IdApp);
        }

        private void TapGestureRecognizer_Tapped(object sender, EventArgs e)
        {
            App.masterDetail.IsPresented = false;
            App.masterDetail.Detail = new NavigationPage(new
ViewsMisClasificaciones(null, App.MyUser.MisLeyes.Where(w=>w.CodigoLey==App.MyUs
er.MiUltimaLeyCargada).SingleOrDefault()));
        }

        private void TapGestureRecognizer_Tapped_1(object sender, EventArgs e)
        {
            App.masterDetail.IsPresented = false;
        }
    }
}

```



```

        App.masterDetail.Detail = new NavigationPage(new MisApuntes(null));
    }

    private void TapGestureRecognizer_Tapped_2(object sender, EventArgs e)
    {
        App.masterDetail.IsPresented = false;
        App.masterDetail.Detail = new NavigationPage(new ViewDetail());
    }

    private void TapGestureRecognizer_Tapped_3(object sender, EventArgs e)
    {
        App.masterDetail.IsPresented = false;
        //App.masterDetail.Detail = new NavigationPage(new
WindowDeCoppa());
        App.masterDetail.Detail = new NavigationPage(new
TabbedPageCargar_CambiarLey());
    }

    private void TapGestureRecognizer_Tapped_4(object sender, EventArgs e)
    {
        App.masterDetail.IsPresented = false;
        App.masterDetail.Detail = new NavigationPage(new
WindowOpcionDeCompra());
    }

    private async void TapGestureRecognizer_Tapped_5(object sender,
EventArgs e)
    {
        Settings.Recuerdame = false;
        Settings.Email = "";
        Settings.Contrasenia = "";
        var ultimosUsuarios = await App.ultimoUser.GetUltimoUserAsync();
        App.ultimoUser.DeleteUltimoUserAsync(ultimosUsuarios.FirstOrDefault());

        App.masterDetail.IsPresented = false;
        await Navigation.PushAsync(new PageInicioDeSesion(), false);
    }

    private void TapGestureRecognizer_Tapped_6(object sender, EventArgs e)
    {
    }
}
}

```

Apartado Modelos y clases

TitulosDeMaster.cs

```
using System;
using System.Collections.Generic;
using System.Text;

namespace IURIS.MOVIL.Modelos_y_clases
{
    public class TitulosDeMaster
    {
        public string NombreTitulo { get; set; }
    }
}
```

Carpeta BanerClass

AboutViewModel.cs

```
using System.Windows.Input;
using Xamarin.Essentials;
using Xamarin.Forms;

namespace IURIS.MOVIL.Modelos_y_clases
{
    public class AboutViewModel : BaseViewModel
    {
        public AboutViewModel()
        {
            Title = "About";
            OpenWebCommand = new Command(async () => await
Browser.OpenAsync("https://aka.ms/xamarin-quickstart"));
        }

        public ICommand OpenWebCommand { get; }
    }
}
```

BaseViewModel.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Runtime.CompilerServices;
using Xamarin.Forms;

namespace IURIS.MOVIL.Modelos_y_clases
{
    public class BaseViewModel : INotifyPropertyChanged
    {
        public IDataStore<Item> DataStore =>
DependencyService.Get<IDataStore<Item>>();
    }
}
```

```

    bool isBusy = false;
    public bool IsBusy
    {
        get { return isBusy; }
        set { SetProperty(ref isBusy, value); }
    }

    string title = string.Empty;
    public string Title
    {
        get { return title; }
        set { SetProperty(ref title, value); }
    }

    protected bool SetProperty<T>(ref T backingStore, T value,
        [CallerMemberName] string propertyName = "",
        Action onChanged = null)
    {
        if (EqualityComparer<T>.Default.Equals(backingStore, value))
            return false;

        backingStore = value;
        onChanged?.Invoke();
        OnPropertyChanged(propertyName);
        return true;
    }

    #region INotifyPropertyChanged
    public event PropertyChangedEventHandler PropertyChanged;
    protected void OnPropertyChanged([CallerMemberName] string propertyName
= "")
    {
        var changed = PropertyChanged;
        if (changed == null)
            return;

        changed.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }
    #endregion
}

```

IDataStore.cs

```

using System;
using System.Collections.Generic;
using System.Text;
using System.Threading.Tasks;

namespace IURIS.MOVIL.Modelos_y_clases
{
    public interface IDataStore<T>
    {
        Task<bool> AddItemAsync(T item);
    }
}

```

```

        Task<bool> UpdateItemAsync(T item);
        Task<bool> DeleteItemAsync(string id);
        Task<T> GetItemAsync(string id);
        Task<IEnumerable<T>> GetItemsAsync(bool forceRefresh = false);
    }
}

```

Item.cs

```

using System;
using System.Collections.Generic;
using System.Text;

namespace IURIS.MOVIL.Modelos_y_clases
{
    public class Item
    {
        public string Id { get; set; }
        public string Text { get; set; }
        public string Description { get; set; }
    }
}

```

ItemDetailViewModel.cs

```

using IURIS.MOVIL.Modelos_y_clases;
using System;
using System.Diagnostics;
using Xamarin.Forms;

namespace XamarinMarctronAdsApp.ViewModels
{
    [QueryProperty(nameof(ItemId), nameof(ItemId))]
    public class ItemDetailViewModel : BaseViewModel
    {
        private string itemId;
        private string text;
        private string description;
        public string Id { get; set; }

        public string Text
        {
            get => text;
            set => SetProperty(ref text, value);
        }

        public string Description
        {
            get => description;
            set => SetProperty(ref description, value);
        }

        public string ItemId

```

```

    {
        get
        {
            return itemId;
        }
        set
        {
            itemId = value;
            LoadItemId(value);
        }
    }

    public async void LoadItemId(string itemId)
    {
        try
        {
            var item = await DataStore.GetItemAsync(itemId);
            Id = item.Id;
            Text = item.Text;
            Description = item.Description;
        }
        catch (Exception)
        {
            Debug.WriteLine("Failed to Load Item");
        }
    }
}

```

ItemsViewModel.cs

```

using IURIS.MOVIL.Modelos_y_clases;
using System;
using System.Collections.ObjectModel;
using System.Diagnostics;
using System.Threading.Tasks;
using Xamarin.Forms;

namespace XamarinMarctronAdsApp.ViewModels
{
    public class ItemsViewModel : BaseViewModel
    {
        private Item _selectedItem;

        public ObservableCollection<Item> Items { get; }
        public Command LoadItemsCommand { get; }
        public Command AddItemCommand { get; }
        public Command<Item> ItemTapped { get; }

        public ItemsViewModel()
        {
            Title = "Browse";
            Items = new ObservableCollection<Item>();
            LoadItemsCommand = new Command(async () => await
ExecuteLoadItemsCommand());

```

```

        ItemTapped = new Command<Item>(OnItemSelected);

        AddItemCommand = new Command(OnAddItem);
    }

    async Task ExecuteLoadItemsCommand()
    {
        IsBusy = true;

        try
        {
            Items.Clear();
            var items = await DataStore.GetItemsAsync(true);
            foreach (var item in items)
            {
                Items.Add(item);
            }
        }
        catch (Exception ex)
        {
            Debug.WriteLine(ex);
        }
        finally
        {
            IsBusy = false;
        }
    }

    public void OnAppearing()
    {
        IsBusy = true;
        SelectedItem = null;
    }

    public Item SelectedItem
    {
        get => _selectedItem;
        set
        {
            SetProperty(ref _selectedItem, value);
            OnItemSelected(value);
        }
    }

    private async void OnAddItem(object obj)
    {
        //await Shell.Current.GoToAsync(nameof(NewItemPage));
    }

    async void OnItemSelected(Item item)
    {
        if (item == null)
            return;

        // This will push the ItemDetailPage onto the navigation stack
        // await
        Shell.Current.GoToAsync($"{nameof(ItemDetailPage)}?{nameof(ItemDetailViewModel.
        ItemId)}={item.Id}");
    }

```

```

    }
}

```

LoginViewModel.cs

```

using IURIS.MOVIL.Modelos_y_clases;
using Xamarin.Forms;

namespace XamarinMarctronAdsApp.ViewModels
{
    public class LoginViewModel : BaseViewModel
    {
        public Command LoginCommand { get; }

        public LoginViewModel()
        {
            LoginCommand = new Command(OnLoginClicked);
        }

        private async void OnLoginClicked(object obj)
        {
            // Prefixing with `//` switches to a different navigation stack
            // instead of pushing to the active one
            // await Shell.Current.GoToAsync($"://{nameof(AboutPage)}");
        }
    }
}

```

MockDataStore.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace IURIS.MOVIL.Modelos_y_clases
{
    public class MockDataStore : IDataStore<Item>
    {
        readonly List<Item> items;

        public MockDataStore()
        {
            items = new List<Item>()
            {
                new Item { Id = Guid.NewGuid().ToString(), Text = "First item",
                Description="This is an item description." },
                new Item { Id = Guid.NewGuid().ToString(), Text = "Second
                item", Description="This is an item description." },
                new Item { Id = Guid.NewGuid().ToString(), Text = "Third item",
                Description="This is an item description." },
            }
        }
    }
}

```

```

        new Item { Id = Guid.NewGuid().ToString(), Text = "Fourth
item", Description="This is an item description." },
        new Item { Id = Guid.NewGuid().ToString(), Text = "Fifth item",
Description="This is an item description." },
        new Item { Id = Guid.NewGuid().ToString(), Text = "Sixth item",
Description="This is an item description." }
    };
}

public async Task<bool> AddItemAsync(Item item)
{
    items.Add(item);

    return await Task.FromResult(true);
}

public async Task<bool> UpdateItemAsync(Item item)
{
    var oldItem = items.Where((Item arg) => arg.Id ==
item.Id).FirstOrDefault();
    items.Remove(oldItem);
    items.Add(item);

    return await Task.FromResult(true);
}

public async Task<bool> DeleteItemAsync(string id)
{
    var oldItem = items.Where((Item arg) => arg.Id ==
id).FirstOrDefault();
    items.Remove(oldItem);

    return await Task.FromResult(true);
}

public async Task<Item> GetItemAsync(string id)
{
    return await Task.FromResult(items.FirstOrDefault(s => s.Id ==
id));
}

public async Task<IEnumerable<Item>> GetItemsAsync(bool forceRefresh =
false)
{
    return await Task.FromResult(items);
}
}

```


NewItemViewModel.cs

```
using IURIS.MOVIL.Modelos_y_clases;
using System;
using Xamarin.Forms;

namespace XamarinMarctronAdsApp.ViewModels
{
    public class NewItemViewModel : BaseViewModel
    {
        private string text;
        private string description;

        public NewItemViewModel()
        {
            SaveCommand = new Command(OnSave, ValidateSave);
            CancelCommand = new Command(OnCancel);
            this.PropertyChanged +=
                (_, __) => SaveCommand.ChangeCanExecute();
        }

        private bool ValidateSave()
        {
            return !String.IsNullOrEmpty(text)
                && !String.IsNullOrEmpty(description);
        }

        public string Text
        {
            get => text;
            set => SetProperty(ref text, value);
        }

        public string Description
        {
            get => description;
            set => SetProperty(ref description, value);
        }

        public Command SaveCommand { get; }
        public Command CancelCommand { get; }

        private async void OnCancel()
        {
            // This will pop the current page off the navigation stack
            await Shell.Current.GoToAsync("..");
        }

        private async void OnSave()
        {
            Item newItem = new Item()
            {
                Id = Guid.NewGuid().ToString(),
                Text = Text,
                Description = Description
            };
        }
    }
}
```

```

        await DataStore.AddItemAsync(newItem);

        // This will pop the current page off the navigation stack
        await Shell.Current.GoToAsync("..");
    }
}

```

Carpeta CustomRender

CustomEditor.cs

```

using System;
using System.Collections.Generic;
using System.Text;
using Xamarin.Forms;

namespace IURIS.MOVIL.Modelos_y_clases
{
    public class CustomEditor:Editor
    {
    }
}

```

CustomEntry.cs

```

using System;
using System.Collections.Generic;
using System.Text;
using Xamarin.Forms;

namespace IURIS.MOVIL.Modelos_y_clases
{
    public class CustomEntry : Entry
    {
    }
}

```

CustomLabeJustify.cs

```

using System;
using System.Collections.Generic;
using System.Text;
using Xamarin.Forms;

namespace IURIS.MOVIL.Modelos_y_clases
{
    public class CustomLabeJustify:Label
    {
    }
}

```

```

    {
    }
}

```

CustomLabel.cs

```

using System;
using System.Collections.Generic;
using System.Text;
using Xamarin.Forms;

namespace IURIS.MOVIL.Modelos_y_clases
{
    public class CustomLabel:Label
    {

    }
}

```

Carpeta DB_Local

Database.cs

```

using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
using IURIS.MOVIL.Modelos_y_clases.DB_Local;
using SQLite;
using System;
using System.Collections.Generic;
using System.Text;
using System.Threading.Tasks;

namespace IURIS.MOVIL.Modelos_y_clases
{
    public class Database
    {
        public SQLiteAsyncConnection _database;
        string _Path;

        public Database(string dbPath)
        {
            //Establishing the conection
            _database = new SQLiteAsyncConnection(dbPath);
            //DeleteTable();
            CreateTable();
            _Path = dbPath;
        }
        public bool DeleteTable()
        {
            _database.DropTableAsync<MyUser>().Wait();
            return true;
        }
        public bool CreateTable()

```

```

    {
        _database.CreateTableAsync<MyUser>().Wait();
        return true;
    }

    // Show the registers
    public Task<List<MyUser>> GetPeopleAsync()
    {
        return
        SQLiteNetExtensionsAsync.Extensions.ReadOperations.GetAllWithChildrenAsync<MyUser>(_database);
        //return _database.Table<MyUser>().ToListAsync();
    }

    // Save registers
    public async void SavePersonAsync(MyUser User)
    {
        await
        SQLiteNetExtensionsAsync.Extensions.WriteOperations.InsertWithChildrenAsync(_database, User);
    }

    // Delete registers
    public async void DeletePersonAsync(MyUser User)
    {
        await
        SQLiteNetExtensionsAsync.Extensions.WriteOperations.DeleteAsync(_database, User, true);
        //return _database.DeleteAsync(User);
    }

    // Save registers
    public async Task<bool> UpdateUserAsync(MyUser User)
    {
        try
        {
            await
            SQLiteNetExtensionsAsync.Extensions.WriteOperations.UpdateWithChildrenAsync(_database, User);
            return true;
        }
        catch (Exception)
        {
            return false;
        }
    }
}

```

LocalDataUltimoUser.cs

```
using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using SQLite;
using System;
using System.Collections.Generic;
using System.Text;
using System.Threading.Tasks;

namespace IURIS.MOVIL.Modelos_y_clases.DB_Local
{
    public class LocalDataUltimoUser
    {
        public SQLiteAsyncConnection _database;
        string _Path;

        public LocalDataUltimoUser(string dbPath)
        {
            //Establishing the conection
            _database = new SQLiteAsyncConnection(dbPath);
            CreateTable();
            _Path = dbPath;
        }

        public bool DeleteTable()
        {
            _database.DropTableAsync<MyUser>().Wait();
            return true;
        }

        public bool CreateTable()
        {
            _database.CreateTableAsync<UltimoUser>().Wait();
            return true;
        }

        // Show the registers
        public Task<List<UltimoUser>> GetUltimoUserAsync()
        {
            return
                SQLiteNetExtensionsAsync.Extensions.ReadOperations.GetAllWithChildrenAsync<Ulti
                moUser>(_database);
            //return _database.Table<MyUser>().ToListAsync();
        }

        // Save registers
        public async void SaveUltimoUserAsync(UltimoUser ultimoUser)
        {
            await
                SQLiteNetExtensionsAsync.Extensions.WriteOperations.InsertWithChildrenAsync(_da
                tabase, ultimoUser);
        }

        // Save registers
        public async Task<bool> UpdateUltimoUserAsync(UltimoUser ultimoUser)
        {
            try
```

```

        {
            await
SQLiteNetExtensionsAsync.Extensions.WriteOperations.UpdateWithChildrenAsync(_da
tabase, ultimoUser);
            return true;
        }
        catch (Exception)
        {
            return false;
        }
    }
    public async void DeleteUltimoUserAsync(UltimoUser ultimoUser)
    {
        await
SQLiteNetExtensionsAsync.Extensions.WriteOperations.DeleteAsync(_database,
ultimoUser, true);
        //return _database.DeleteAsync(User);
    }
}
}

```

LocalSaveUser.cs

```

using IURIS.BIZ;
using IURIS.COMMON.Constantes;
using IURIS.COMMON.Entidades.Ley;
using IURIS.COMMON.Entidades.Ley.ComponentesDeLey;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion.ComponentesDeUsuario;
using
IURIS.COMMON.Entidades.UsuariosDeAplicacion.ComponentesDeUsuario.DatosCriticos;
using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace IURIS.MOVIL.Modelos_y_clases.DB_Local
{
    public class LocalSaveUser
    {
        string _Nombre;
        string _APaterno;
        string _AMaterno;

        IManejadorDeLeyPrincipal manejadorDeLeyPrincipal;
        IManejadorDeUsuarioAplicacion manejadorDeUsuarioAplicacion;

        Const _Const = new Const();
        public LocalSaveUser()
        {
        }
    }
}

```

```

public bool SaveUser(string Nombre, string APaterno, string AMaterno)
{
    _Nombre = Nombre;
    _APaterno = APaterno;
    _AMaterno = AMaterno;
    try
    {
        MyUser myUser = CrearUsuario();
        if (myUser == null) return false;

        App.Database.SavePersonAsync(myUser);
        App.MyUser = myUser;
        return true;
    }
    catch (Exception ex)
    {
        return false;
    }
}

public async Task<bool> UpdateUser()
{
    try
    {
        await App.Database.UpdateUserAsync(App.MyUser);
        return true;
    }
    catch (Exception)
    {
        return false;
    }
}

private MyUser CrearUsuario()
{
    manejadorDeLeyPrincipal = new ManejadorDeLeyPrincipal(new
RepositorioGenerico<LeyPrincipal>());
    manejadorDeUsuarioAplicacion = new ManejadorDeUsuarioAplicacion(new
RepositorioGenerico<Usuarios>());
    int NumUser = manejadorDeUsuarioAplicacion.Listar.Count+1;
    Leyes ley = (Leyes)manejadorDeLeyPrincipal.Listar.Where(w =>
w.CodigoLey.ToUpper() == _Const.MiLeyPrincipal.ToUpper() && w.Clasificacion==
"LeyInicial").FirstOrDefault();
    ley.Clasificaciones = new List<ClasificacionPUusuario>();

    MyUser myUser = new MyUser
    {
        Apuntes = new List<Apunte>(),
        MisLeyes = new List<MyLey>(),
        ApellidoMaterno = _AMaterno,
        ApellidoPaterno = _APaterno,
        Nombre = _Nombre,
        DatosSobreUsuario=new MyDatosSobreUsuarioParaLey(),

```

```

        IdApp=NumUser
    };
    myUser.DatosSobreUsuario.NumLeyesPermitidas =
_Const.NumDeLeyesInicial;
    myUser.MisLeyes.Add(LeyToMyley(ley));
    myUser.MiUltimaLeyCargada = ley.CodigoLey;

    Usuarios user = new Usuarios()
    {
        IdApp = NumUser,
        ApellidoPaterno = _APaterno,
        ApellidoMaterno=_AMaterno,
        Nombre = _Nombre,
        DatosSobreUsuario = new DatosSobreUsuarioParaLey()
    };
    user.DatosSobreUsuario.NumLeyesPermitidas =
_Const.NumDeLeyesInicial;

    try
    {
        manejadorDeUsuarioAplicacion.AGREGAR(user);
    }
    catch (Exception)
    {

        myUser = null;
    }

    return myUser;
}

public MyLey LeyToMyley(Leyes Ley)
{
    try
    {
        MyLey myLey = new MyLey()
        {
            Clasificacion = Ley.Clasificacion,
            CodigoLey = Ley.CodigoLey,
            EsModificacion = Ley.EsModificacion,
            FechaDeDescarga = Ley.FechaDeDescarga,
            ListaDeTitulos = Ley.ListaDeTitulos,
            NombreLey = Ley.NombreLey,
            UltimaFechaDeModificacion = Ley.UltimaFechaDeModificacion,
            Clasificaciones = Ley.Clasificaciones
        };

        return myLey;
    }
    catch (Exception)
    {
        return null;
    }
}

```



```

    }
}

```

SaveMyClasificacionDeLey.cs

```

using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using SQLite;
using System;
using System.Collections.Generic;
using System.Text;
using System.Threading.Tasks;

namespace IURIS.MOVIL.Modelos_y_clases.DB_Local
{
    public class SaveMyClasificacionDeLey
    {
        public SQLiteAsyncConnection _database;

        public SaveMyClasificacionDeLey(string dbPath)
        {
            //Establishing the conection
            _database = new SQLiteAsyncConnection(dbPath);
            //DeleteTable();
            CreateTable();
        }

        public bool DeleteTable()
        {
            _database.DropTableAsync<MyClasificacionDeLey>().Wait();
            return true;
        }

        public bool CreateTable()
        {
            _database.CreateTableAsync<MyClasificacionDeLey>().Wait();
            return true;
        }

        // Show the registers
        public Task<List<MyClasificacionDeLey>> GetPeopleAsync()
        {
            return
                SQLiteNetExtensionsAsync.Extensions.ReadOperations.GetAllWithChildrenAsync<MyClasificacionDeLey>(_database);
            //return _database.Table<MyUser>().ToListAsync();
        }

        // Save registers
        public async void SavePersonAsync(MyClasificacionDeLey
myClasificacionDeLey)
        {
            await
                SQLiteNetExtensionsAsync.Extensions.WriteOperations.InsertWithChildrenAsync(_da
tabase, myClasificacionDeLey);
        }
    }
}

```

```

        // Delete registers
        public async void DeletePersonAsync(MyClasificacionDeLey
myClasificacionDeLey)
        {
            await
SQLiteNetExtensionsAsync.Extensions.WriteOperations.DeleteAsync(_database,
myClasificacionDeLey, true);
            //return _database.DeleteAsync(User);
        }

        // Save registers
        public async Task<bool> UpdateUserAsync(MyClasificacionDeLey
myClasificacionDeLey )
        {
            try
            {
                await
SQLiteNetExtensionsAsync.Extensions.WriteOperations.UpdateWithChildrenAsync(_da
tabase, myClasificacionDeLey);
                return true;
            }
            catch (Exception)
            {
                return false;
            }
        }
    }
}

```

UltimoUserLocal.cs

```

using System;
using System.Collections.Generic;
using System.Text;
using System.Threading.Tasks;

namespace IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON
{
    public class UltimoUserLocal
    {
        public bool Save(string IdUser)
        {
            try
            {
                UltimoUser ultimoUser = new UltimoUser() { IdUser = IdUser };

                App.ultimoUser.SaveUltimoUserAsync(ultimoUser);
                return true;
            }
            catch (Exception)
            {
                return false;
            }
        }
    }
}

```

```

        }
    }

    public async Task< bool> Update(UltimoUser ultimoUser)
    {
        try
        {
            await App.ultimoUser.UpdateUltimoUserAsync(ultimoUser);
            return true;
        }
        catch (Exception)
        {
            return false;
        }
    }
}

```

Subcarpeta COMMON

MyClasificacion.cs

```

using System;
using System.Collections.Generic;
using System.Text;

namespace IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON
{
    public class MyClasificacion
    {
        public string Nombre { get; set; }

        public override string ToString()
        {
            return string.Format("{0}", Nombre);
        }
    }
}

```

MyClasificacionDeLey.cs

```

using SQLite;
using System;
using System.Collections.Generic;
using System.Text;

namespace IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON
{
    public class MyClasificacionDeLey
    {
        public MyClasificacionDeLey()

```

```

    {
        Id = Guid.NewGuid().ToString();
    }
    [PrimaryKey]
    public string Id { get; set; }

    public string Nombre { get; set; }

    public override string ToString()
    {
        return string.Format("{0}", Nombre);
    }
}
}

```

MyDatosSobreUsuarioParaLey.cs

```

using System;
using System.Collections.Generic;
using System.Text;

namespace IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON
{
    public class MyDatosSobreUsuarioParaLey
    {
        public int NumLeyesPermitidas { get; set; }
        //public DateTime FechaDeCompra { get; set; }
        //public int NumDeMeses { get; set; }
        //public List<string> TipoDeCompra { get; set; }
    }
}

```

MyLey.cs

```

using IURIS.COMMON.Entidades.Ley.ClasificacionDeLey;
using IURIS.COMMON.Entidades.Ley.ComponentesDeLey;
using SQLiteNetExtensions.Attributes;
using System;
using System.Collections.Generic;
using System.Text;

namespace IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON
{
    public class MyLey
    {
        public string NombreLey { get; set; }
        public string CodigoLey { get; set; }
        public bool EsModificacion { get; set; } //importante
        public DateTime UltimaFechaDeModificacion { get; set; }
        public DateTime? FechaDeDescarga { get; set; }
        public List<ClasificacionPUusuario> Clasificaciones { get; set; }
        public List<Titulo> ListaDeTitulos { get; set; }
    }
}

```

```

        //[TextBlob("MyClasificacionPUusuarioBlogged")]
        //public string MyClasificacionPUusuarioBlogged { get; set; }

        //[TextBlob("MyTituloBlogged")]
        //public string MyTituloBlogged { get; set; }

        public string Clasificacion { get; set; }
        //[TextBlob("MyClasificacionBlogged")]
        //public string MyClasificacionBlogged { get; set; }
    }
}

```

MyUser.cs

```

using IURIS.COMMON.Entidades.Ley;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion.ComponentesDeUsuario;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion.ComponentesDeUsuario.DatosCriticos;
using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using MongoDB.Bson;
using SQLite;
using SQLiteNetExtensions.Attributes;
using System;
using System.Collections.Generic;
using System.Text;

namespace IURIS.MOVIL.Modelos_y_clases.DB_Local
{
    public class MyUser
    {
        public MyUser()
        {
            Id = Guid.NewGuid().ToString();
        }
        [PrimaryKey]
        public string Id { get; set; }
        public string MiUltimaLeyCargada { get; set; }
        public string IdUser { get; set; }
        public int IdApp { get; set; }
        public string Correo { get; set; }
        public int Contraseña { get; set; }

        //public string IdUser { get; set; }
        public string Nombre { get; set; }
        public string ApellidoPaterno { get; set; }
        public string ApellidoMaterno { get; set; }

        [TextBlob("MyDatosSobreUsuarioParaLeyBlogged")]
        public MyDatosSobreUsuarioParaLey DatosSobreUsuario { get; set; }
        public string MyDatosSobreUsuarioParaLeyBlogged { get; set; }

        [TextBlob("MysLeyesBlogged")]
        public List<MyLey> MisLeyes { get; set; }
    }
}

```

```

        public string MysLeyesBlobbered { get; set; }

        [TextBlob("ApunttesBlobbered")]
        public List<Apunte> Apuntes { get; set; }
        public string ApunttesBlobbered { get; set; }
    }
}

```

UltimoUser.cs

```

using SQLite;
using System;
using System.Collections.Generic;
using System.Text;

namespace IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON
{
    public class UltimoUser
    {
        public UltimoUser()
        {
            Id = Guid.NewGuid().ToString();
        }
        [PrimaryKey]
        public string Id { get; set; }
        public string IdUser { get; set; }
    }
}

```

Carpeta Tools

ByteArrayToImageSourceConverter.cs

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using Xamarin.Forms;

namespace IURIS.MOVIL.Modelos_y_clases
{
    public class ByteArrayToImageSourceConverter: IValueConverter
    {
        public object Convert(object value,
            Type targetType,
            object parameter,
            System.Globalization.CultureInfo culture)
        {
            ImageSource retSource = null;
            if (value != null)

```

```

        {
            byte[] imageAsBytes = (byte[])value;
            retSource = ImageSource.FromStream(() => new
MemoryStream(imageAsBytes));
        }
        return retSource;
    }

    public object ConvertBack(object value,
        Type targetType,
        object parameter,
        System.Globalization.CultureInfo culture)
    {
        throw new NotImplementedException();
    }
}
}

```

ClassAnuncio.cs

```

using System;
using IURIS.MOVIL.Utills;
using System.Collections.Generic;
using System.Text;
using Rg.Plugins.Popup.Services;
using IURIS.MOVIL.Views.ViewsVentanasEmergentes;
using MarcTron.Plugin;

namespace IURIS.MOVIL.Modelos_y_clases
{
    public class ClassAnuncio
    {
        public void MostrarAnuncioPantalla()
        {
            if (Settings.CountParaNumAleatorio != "" && Settings.NumAleatorio
!= 0)
            {
                if (int.Parse(Settings.CountParaNumAleatorio) !=
Settings.NumAleatorio)
                {
                    int num = int.Parse(Settings.CountParaNumAleatorio) + 1;
                    Settings.CountParaNumAleatorio = num.ToString();
                }
                else
                {
                    Random rnd = new Random();
                    Settings.NumAleatorio = rnd.Next(10, 30);
                    Settings.CountParaNumAleatorio = "1";

                    var test =
CrossMTAdmob.Current.IsInterstitialLoaded().ToString();
                    CrossMTAdmob.Current.ShowInterstitial();
                }
            }
        }
    }
}

```

```

        CrossMTAdmob.Current.LoadInterstitial("ca-app-pub-
2336879831564913/8704842488");
    }
}
else
{
    Settings.CountParaNumAleatorio = "1";
    Settings.NumAleatorio = 1;
}
}

public void MostrarAnuncioPantallaParaTitulos()
{
    if (Settings.CountParaNumAleatorioTitulos != "" &&
Settings.NumAleatorioTitulos != 0)
    {
        if (int.Parse(Settings.CountParaNumAleatorioTitulos) !=
Settings.NumAleatorioTitulos)
        {
            int num = int.Parse(Settings.CountParaNumAleatorioTitulos)
+ 1;
            Settings.CountParaNumAleatorioTitulos = num.ToString();
        }
        else
        {
            Random rnd = new Random();
            Settings.NumAleatorioTitulos = rnd.Next(1, 4);
            Settings.CountParaNumAleatorioTitulos = "1";

            var test =
CrossMTAdmob.Current.IsInterstitialLoaded().ToString();

            CrossMTAdmob.Current.ShowInterstitial();
            CrossMTAdmob.Current.LoadInterstitial("ca-app-pub-
2336879831564913/8704842488");

            ////var test2 =
CrossMTAdmob.Current.IsRewardedVideoLoaded().ToString();
            ////CrossMTAdmob.Current.ShowRewardedVideo();
            ////CrossMTAdmob.Current.LoadRewardedVideo("ca-app-pub-
3940256099942544/1044960115");
        }
    }
    else
    {
        Settings.CountParaNumAleatorioTitulos = "1";
        Settings.NumAleatorioTitulos = 1;
    }
}
}
}
}

```


HerramientasGenerales.cs

```
using Acr.UserDialogs;
using IURIS.BIZ;
using IURIS.COMMON.Constantes;
using IURIS.COMMON.Entidades.Ley;
using IURIS.COMMON.Entidades.Ley.ComponentesDeLey;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using IURIS.MOVIL.Modelos_y_clases.DB_Local;
using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using IURIS.MOVIL.Utils;
using IURIS.MOVIL.Views.ViewsVentanasEmergentes;
using Rg.Plugins.Popup.Services;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Xamarin.Forms;

namespace IURIS.MOVIL.Modelos_y_clases.Tools
{
    public class HerramientasGenerales
    {
        Usuarios _User;
        Const Const;
        IManejadorDeLeyes manejadorDeLeyes;

        public HerramientasGenerales()
        {
            Const = new Const();
            manejadorDeLeyes = new ManejadorDeLeyes(new
RepositorioGenerico<Leyes>());
        }

        public HerramientasGenerales(Usuarios usuarios)
        {
            _User = usuarios;
            Const = new Const();
        }

        public void RenovarSuscripcion()
        {
            //var num =
GetMonthDifference(App.MyUser.DatosSobreUsuario.FechaDeCompra);

            //if (App.MyUser.DatosSobreUsuario.NumDeMeses == num)
            //{
                // App.masterDetail.IsPresented = false;
                // App.masterDetail.Detail = new NavigationPage(new
WindowOpcionDeCompra());
            //}
        }

        private int GetMonthDifference(DateTime startDate)
```

```

        {
            int monthsApart = 12 * (startDate.Year - DateTime.UtcNow.Year) +
startDate.Month - DateTime.UtcNow.Month;
            return Math.Abs(monthsApart);
        }

        public void TipoDeAlmacenamiento()
        {
            //var x = App.MyUser.DatosSobreUsuario.TipoDeCompra.Find(e => e ==
Const._39Mensuales || e == Const._ComprarEspacio1Ley || e ==
Const._49Mensuales);

            //Settings.TypeDB = App.MyUser.DatosSobreUsuario.TipoDeCompra ==
null || x != null ? Const.TypeDBLocal : Const.TypeDBNube;
        }

        public async Task<string> AgragarLEyAsync(string Codigo)
        {
            bool ExisteLey = false;
            Leyes _LeyComprada = null;
            string _1LeyMas = null;

            //if (App.MyUser.DatosSobreUsuario.TipoDeCompra != null) _1LeyMas =
App.MyUser.DatosSobreUsuario.TipoDeCompra.Find(w => w ==
Const._ComprarEspacio1Ley || w == Const._39Mensuales);

            if (App.MyUser.MisLeyes.Count >=
App.MyUser.DatosSobreUsuario.NumLeyesPermitidas) //_1LeyMas != null
            {
                UserDialogs.Instance.HideLoading();
                //await PopupNavigation.Instance.PushAsync(new
WindowOfComprarEspacio());

                return "Aviso|Has utilizado los espacios gratuitos,\nSi
requieres de mas espacio para agregar leyes a tu lista\n ponte en contacto al
correo\niurisespacio@gmail.com\nCon el asunto ESPACIO|OK";
            }

            if (string.IsNullOrEmpty(Codigo)) return "Advertencia|Introduce un
codigo valido|OK";

            _LeyComprada = manejadorDeLeyes.Consult(Codigo);

            if (_LeyComprada == null)
            {
                UserDialogs.Instance.HideLoading();
                return "Error|Codigo incorrecto\nIntenta de nuevo|OK";
            }

            if (App.MyUser.MisLeyes.Exists(r => r.CodigoLey ==
_LeyComprada.CodigoLey))
            {
                UserDialogs.Instance.HideLoading();
                return "Advertencia|El codigo ingresado\nCorresponde a una ley
que ya \nse encuentra en tu lista|OK";
            }
        }
    }
}

```

```

    }

    UserDialogs.Instance.HideLoading();
    UserDialogs.Instance.ShowLoading("Agregando ley a tu lista",
MaskType.Gradient);
    await Task.Delay(1000);
    _LeyComprada.FechaDeDescarga = DateTime.UtcNow.ToLocalTime();

    LocalSaveUser localSaveUser = new LocalSaveUser();

    App.MyUser.MisLeyes.Add(localSaveUser.LeyToMyley(_LeyComprada));
    App.MyUser.MisLeyes.Where(w => w.CodigoLey ==
_LeyComprada.CodigoLey).SingleOrDefault().Clasificaciones = new
List<ClasificacionPUusuario>();

    if (await App.Database.UpdateUserAsync(App.MyUser))
    {
        //_LeyComprada.numDescargas += 1;
        //manejadorDeLeyes.Modificar(_LeyComprada);
        UserDialogs.Instance.HideLoading();
        return "OK|OK|OK";
    }
    else
    {
        UserDialogs.Instance.HideLoading();
        return "Error|Ocurrio un error\nIntente mas tarde|OK";
    }
}

public async void ShowAlert(WindowAlert _WindowAlert, int duracion)
{
    //string[] datos = await App.Database.UpdateUserAsync(App.MyUser) ?
new[] { "Ok", "Artículo agregado" } : new[] { "Error", "Intente más tarde" };
    try
    {
        await PopupNavigation.Instance.PushAsync(_WindowAlert , false);
        await Task.Delay(duracion);
        await PopupNavigation.Instance.PopAsync(false);
        //return true;
    }
    catch (Exception ex)
    {
        return;
    }
}
}
}

```

Utils

GetClasificaciones.cs

```

using IURIS.BIZ;
using IURIS.COMMON.Constantes;
using IURIS.COMMON.Entidades.Ley.ClasificacionDeLey;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;

```

```

using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using IURIS.MOVIL.Modelos_y_clases.DB_Local;
using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using Matcha.BackgroundService;
using System;
using System.Collections.Generic;
using System.Text;
using System.Threading.Tasks;
using Xamarin.Essentials;

namespace IURIS.MOVIL.Modelos_y_clases.Utils
{
    public class GetClasificaiciones : IPeriodicTask
    {
        IManejadorDeClasificaciones manejadorDeClasificaciones;
        public TimeSpan Interval { get; set; }
        List<Clasificacion> _Clasificaciones;

        public GetClasificaiciones(int min)
        {
            Interval = TimeSpan.FromHours(min);
        }

        public async Task<bool> StartJob()
        {
            try
            {
                // YOUR CODE HERE
                // THIS CODE WILL BE EXECUTE EVERY INTERVAL
                //manejadorDeUsuarioAplicacion = new
ManejadorDeUsuarioAplicacion(new RepositorioGenerico<Usuarios>());

                //var _user =
manejadorDeUsuarioAplicacion.EncontrarUsuario(App.MyUser.Correo,
App.MyUser.Contrasenia);

                Const _Const = new Const();

                manejadorDeClasificaciones = new ManejadorDeClasificaciones(new
RepositorioGenerico<Clasificacion>());
                _Clasificaciones = manejadorDeClasificaciones.Listar;

                var milist = App.MyClasificacionDeLey.GetPeopleAsync();

                foreach (var item in _Clasificaciones)
                {
                    if
(!item.Nombre.Equals(_Const.NombreDeClasificaiconExcluir) &&
!milist.Result.Exists(x => x.Nombre.Equals(item.Nombre)))
SaveMyClasificacion(item);
                    //if ( !milist.Result.Exists(x =>
x.Nombre.Equals(item.Nombre))) SaveMyClasificacion(item);
                }
            }
        }
    }
}

```

```

        return true; //return false when you want to stop or trigger
only once
    }
    catch (Exception)
    {

        return false;
    }
}

private void SaveMyClasificacion(Clasificacion item)
{
App.MyClasificacionDeLey.SavePersonAsync(ConvertirAMyClasificacionDeLEy(item));
}

private MyClasificacionDeLey
ConvertirAMyClasificacionDeLEy(Clasificacion item)
{
    MyClasificacionDeLey myClasificacion = new MyClasificacionDeLey()
    {
        Nombre = item.Nombre
    };

    return myClasificacion;
}
}
}

```

Settings.cs

```

using IURIS.COMMON.Entidades.Ley;
using Plugin.Settings;
using Plugin.Settings.Abstractions;
using System;
using System.Collections.Generic;
using System.Text;

namespace IURIS.MOVIL.Utills
{
    public static class Settings
    {
        private static ISettings AppSettings { get { return
CrossSettings.Current; } }

        #region Settings Constants
        private const string NumUserSettingsKey = "Last_Num_key";
        private const string CorreoUserSettingsKey = "Last_Correo_key";
        private const string ContrasenialUserSettingsKey =
"Last_Cotrasenia_key";
        private const string LeyCodigoUserSettingsKey = "Last_LeyCodigo_key";
        private const string TypeDBSettingsKey = "Last_TypeDB_key";
        private const string ViewsOfCompraUserSettingsKey =
"Last_Contador_key";

```

```

        private const string ViewsOfAnuncioTituloSettingsKey =
"Last_ContadorTitulo_key";
        private const int NumaleatorioSettingsKey = 1;
        private const int NumaleatorioTituloSettingsKey = 1;

        private const bool RecuerdameUserSettingsKey = false;
        private static readonly string SettingsDefault = string.Empty;
        private static readonly string SettingsDefaultCodigo = string.Empty;
        private static readonly string SettingsDefaultTypeDB = string.Empty;
        private static readonly bool SettingsDefaultiBool = default;
        private static readonly int SettingsDefaultviews = 0;
        private static readonly int SettingsDefaultNumaleatorio = 0;

#endregion

        public static string NumUsuario
        {
            get
            {
                return Appsettings.GetValueOrDefault(NumUserSettingsKey,
SettingsDefault);
            }
            set
            {
                Appsettings.AddOrUpdateValue(NumUserSettingsKey, value);
            }
        }

        public static string Email
        {
            get
            {
                return Appsettings.GetValueOrDefault(CorreoUserSettingsKey,
SettingsDefault);
            }
            set
            {
                Appsettings.AddOrUpdateValue(CorreoUserSettingsKey, value);
            }
        }

        public static string Contraseniam
        {
            get
            {
                return
Appsettings.GetValueOrDefault(ContraseniamUserSettingsKey, SettingsDefault);
            }
            set
            {
                Appsettings.AddOrUpdateValue(ContraseniamUserSettingsKey,
value);
            }
        }

```

```

    }

    public static bool Recuerdame
    {
        get
        {
            return
Appsettings.GetValueOrDefault(RecuerdameUserSettingsKey.ToString(),
SettingsDefaultiBool);
        }
        set
        {
            Appsettings.AddOrUpdateValue(RecuerdameUserSettingsKey.ToString(), value);
        }
    }

    public static string CountParaNumAleatorio
    {
        get
        {
            return
Appsettings.GetValueOrDefault(ViewsOfCompraUserSettingsKey, SettingsDefault);
        }
        set
        {
            Appsettings.AddOrUpdateValue(ViewsOfCompraUserSettingsKey,
value);
        }
    }

    public static string CountParaNumAleatorioTitulos
    {
        get
        {
            return
Appsettings.GetValueOrDefault(ViewsOfAnuncioTituloSettingsKey,
SettingsDefault);
        }
        set
        {
            Appsettings.AddOrUpdateValue(ViewsOfAnuncioTituloSettingsKey,
value);
        }
    }

    public static int NumAleatorio
    {
        get
        {

```

```

        return
Appsettings.GetValueOrDefault(NumaleatorioSettingsKey.ToString(),
SettingsDefaultviews);
    }
    set
    {

Appsettings.AddOrUpdateValue(NumaleatorioSettingsKey.ToString(), value);

    }
}

    public static int NumAleatorioTitulos
    {
        get
        {
            return
Appsettings.GetValueOrDefault(NumaleatorioTituloSettingsKey.ToString(),
SettingsDefaultviews);
        }
        set
        {

Appsettings.AddOrUpdateValue(NumaleatorioTituloSettingsKey.ToString(), value);

        }
    }

    public static string LastCode
    {
        get
        {
            return Appsettings.GetValueOrDefault(LeyCodigoUserSettingsKey,
SettingsDefaultCodigo);
        }
        set
        {

            Appsettings.AddOrUpdateValue(LeyCodigoUserSettingsKey, value);

        }
    }

    public static string TypeDB
    {
        get
        {
            return Appsettings.GetValueOrDefault(TypeDBSettingsKey,
SettingsDefaultTypeDB);
        }
        set
        {

            Appsettings.AddOrUpdateValue(TypeDBSettingsKey, value);

        }
    }

```



```

    }
}

```

Apartado Views

Carpeta Ajustes

PageAjustes.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="IURIS.MOVIL.Views.Ajustes.PageAjustes">
    <ContentPage.Content>
        <StackLayout>
            <Label Text="Welcome to Xamarin.Forms!"
                  VerticalOptions="CenterAndExpand"
                  HorizontalOptions="CenterAndExpand" />
        </StackLayout>
    </ContentPage.Content>
</ContentPage>

```

PageAjustes.xaml.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views.Ajustes
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class PageAjustes : ContentPage
    {
        public PageAjustes()
        {
            InitializeComponent();
        }
    }
}

```

Carpeta Baners

baner.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:controls="clr-
namespace:MarcTron.Plugin.Controls;assembly=Plugin.MtAdmob" xmlns:vm="clr-
namespace:IURIS.MOVIL.Modelos_y_clases"
             x:Class="IURIS.MOVIL.Views.Baners.baner">
    <ContentPage.BindingContext>
        <vm:AboutViewModel />
    </ContentPage.BindingContext>

    <ContentPage.Resources>
        <ResourceDictionary>
            <Color x:Key="Accent">#96d1ff</Color>
        </ResourceDictionary>
    </ContentPage.Resources>

    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>

        <StackLayout VerticalOptions="FillAndExpand" HorizontalOptions="Fill">
            <StackLayout HorizontalOptions="EndAndExpand">
                <controls:MTAdView x:Name="myAds" IsVisible="false"
                                AdsId="ca-app-pub-
3940256099942544/6300978111"
                                VerticalOptions="EndAndExpand" >
                    <controls:MTAdView.HeightRequest>
                        <x:OnIdiom>
                            <x:OnIdiom.Phone>50</x:OnIdiom.Phone>
                            <x:OnIdiom.Tablet>90</x:OnIdiom.Tablet>
                            <x:OnIdiom.Desktop>90</x:OnIdiom.Desktop>
                        </x:OnIdiom>
                    </controls:MTAdView.HeightRequest>
                </controls:MTAdView>
            </StackLayout>
        </StackLayout>

        <ScrollView Grid.Row="1">
            <StackLayout Orientation="Vertical" Padding="30,24,30,24"
                        Spacing="10">

                <!--reward points-->
                <Label x:Name="Test" Text="" FontSize="Title"/>
            </StackLayout>
        </ScrollView>
    </Grid>
</ContentPage>
```

```

        <Label Text="Make changes to your XAML file and save to see
your UI update in the running app with XAML Hot Reload.
Give it a try!"
        FontSize="16" Padding="0,0,0,0"/>

        <Button Margin="0,10,0,0" Text="Show Reward Ad"
        Clicked="Button_Clicked"
        TextColor="White" />

        <Button Margin="0,10,0,0" Text="Show Interstitial Ad"
        Clicked="Button_Clicked_2"
        TextColor="White" />

        <!--<Button Margin="0,10,0,0" Text="Show banner"
        Clicked="Button_Clicked_1"
        TextColor="White" />-->

    </StackLayout>

</ScrollView>

</Grid>

</ContentPage>

```

baner.xaml.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

using Xamarin.Forms;
using Xamarin.Forms.Xaml;
using MarcTron.Plugin;
using MarcTron.Plugin.CustomEventArgs;
using System;
using System.Diagnostics;
using System.Threading.Tasks;
using Xamarin.Essentials;
using Xamarin.Forms;

namespace IURIS.MOVIL.Views.Baners
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class baner : ContentPage
    {
        int reward = 0;

        private bool _shouldSetEvents = true;

        public int count { get; set; }

        public baner()

```

```

{
    InitializeComponent();
    MainThread.BeginInvokeOnMainThread(async () => {
        await Task.Delay(5000);
        myAds.IsVisible = true;

    });
    CrossMTAdmob.Current.LoadInterstitial("ca-app-pub-
3940256099942544/1033173712");

    CrossMTAdmob.Current.LoadRewardedVideo("ca-app-pub-
3940256099942544/5224354917");
}

private void Button_Clicked(object sender, EventArgs e)
{
    //myAds.IsVisible = false;

    var test = CrossMTAdmob.Current.IsRewardedVideoLoaded().ToString();
    CrossMTAdmob.Current.ShowRewardedVideo();
    //CrossMTAdmob.Current.LoadRewardedVideo("ca-app-pub-
3940256099942544/5224354917");
    SetEvents();

    //count++;
    //Test.Text = count.ToString();
}

private void Button_Clicked_1(object sender, EventArgs e)
{
}

void SetEvents()
{
    if (_shouldSetEvents)
    {
        _shouldSetEvents = false;
        CrossMTAdmob.Current.OnRewardedVideoStarted +=
Current_OnRewardedVideoStarted;
        CrossMTAdmob.Current.OnRewarded += Current_OnRewarded;
        CrossMTAdmob.Current.OnRewardedVideoAdClosed +=
Current_OnRewardedVideoAdClosed;
        CrossMTAdmob.Current.OnRewardedVideoAdFailedToLoad +=
Current_OnRewardedVideoAdFailedToLoad;
        CrossMTAdmob.Current.OnRewardedVideoAdLeftApplication +=
Current_OnRewardedVideoAdLeftApplication;
        CrossMTAdmob.Current.OnRewardedVideoAdLoaded +=
Current_OnRewardedVideoAdLoaded;
        CrossMTAdmob.Current.OnRewardedVideoAdOpened +=
Current_OnRewardedVideoAdOpened;
        CrossMTAdmob.Current.OnRewardedVideoAdCompleted +=
Current_OnRewardedVideoAdCompleted;

        //CrossMTAdmob.Current.OnInterstitialLoaded +=
Current_OnInterstitialLoaded;
        //CrossMTAdmob.Current.OnInterstitialOpened +=
Current_OnInterstitialOpened;
    }
}

```

```

        //CrossMTAdmob.Current.OnInterstitialClosed +=
Current_OnInterstitialClosed;
    }
}

private void DisableEvents()
{
    _shouldSetEvents = true;
    CrossMTAdmob.Current.OnRewardedVideoStarted -=
Current_OnRewardedVideoStarted;
    CrossMTAdmob.Current.OnRewarded -= Current_OnRewarded;
    CrossMTAdmob.Current.OnRewardedVideoAdClosed -=
Current_OnRewardedVideoAdClosed;
    CrossMTAdmob.Current.OnRewardedVideoAdFailedToLoad -=
Current_OnRewardedVideoAdFailedToLoad;
    CrossMTAdmob.Current.OnRewardedVideoAdLeftApplication -=
Current_OnRewardedVideoAdLeftApplication;
    CrossMTAdmob.Current.OnRewardedVideoAdLoaded -=
Current_OnRewardedVideoAdLoaded;
    CrossMTAdmob.Current.OnRewardedVideoAdOpened -=
Current_OnRewardedVideoAdOpened;

    //CrossMTAdmob.Current.OnInterstitialLoaded -=
Current_OnInterstitialLoaded;
    //CrossMTAdmob.Current.OnInterstitialOpened -=
Current_OnInterstitialOpened;
    //CrossMTAdmob.Current.OnInterstitialClosed -=
Current_OnInterstitialClosed;
}

private void Current_OnInterstitialClosed(object sender, EventArgs e)
{
    Debug.WriteLine("OnInterstitialClosed");
}

private void Current_OnInterstitialOpened(object sender, EventArgs e)
{
    Debug.WriteLine("OnInterstitialOpened");
}

private void Current_OnInterstitialLoaded(object sender, EventArgs e)
{
    Debug.WriteLine("OnInterstitialLoaded");
}

e) private void Current_OnRewardedVideoAdOpened(object sender, EventArgs
{
    Debug.WriteLine("OnRewardedVideoAdOpened");
}

e) private void Current_OnRewardedVideoAdLoaded(object sender, EventArgs
{
    Debug.WriteLine("OnRewardedVideoAdLoaded");
}

```

```

        private void Current_OnRewardedVideoAdLeftApplication(object sender,
EventArgs e)
        {
            Debug.WriteLine("OnRewardedVideoAdLeftApplication");
        }

        private void Current_OnRewardedVideoAdFailedToLoad(object sender,
MTEEventArgs e)
        {
            Debug.WriteLine("OnRewardedVideoAdFailedToLoad");
        }

        private void Current_OnRewardedVideoAdClosed(object sender, EventArgs
e)
        {
            CrossMTAdmob.Current.LoadRewardedVideo("ca-app-pub-
2000687856046168/8183902079");
            var test = CrossMTAdmob.Current.IsRewardedVideoLoaded().ToString();
        }

        private void Current_OnRewarded(object sender, MTEEventArgs e)
        {
            Debug.WriteLine($"OnRewarded: {e.RewardType} - {e.RewardAmount}");
            reward = reward + e.RewardAmount;
            Test.Text = "Reward Points : " + reward;
            //Downloads.Text = reward.ToString();
        }

        private void Current_OnRewardedVideoStarted(object sender, EventArgs e)
        {
            Debug.WriteLine("OnRewardedVideoStarted");
        }

        private void Current_OnRewardedVideoAdCompleted(object sender,
EventArgs e)
        {
            Debug.WriteLine("OnRewardedVideoAdCompleted");
        }

        private void MyAds_AdVOpened(object sender, EventArgs e)
        {
            Console.WriteLine("MyAds_AdVOpened");
        }

        private void MyAds_AdVImpression(object sender, EventArgs e)
        {
            Console.WriteLine("MyAds_AdVImpression");
        }

        private void MyAds_AdVClosed(object sender, EventArgs e)
        {
            Console.WriteLine("MyAds_AdVClosed");
        }

        private void MyAdsAdsClicked(object sender, EventArgs e)
        {
            Console.WriteLine("MyAdsAdsClicked");
        }

```

```

        private void LoadReward_OnClicked(object sender, EventArgs e)
        {
            CrossMTAdmob.Current.LoadRewardedVideo("ca-app-pub-
2000687856046168/8183902079");
        }

        private void ShowReward_OnClicked(object sender, EventArgs e)
        {
            var test = CrossMTAdmob.Current.IsRewardedVideoLoaded().ToString();
            CrossMTAdmob.Current.LoadRewardedVideo("ca-app-pub-
2000687856046168/8183902079");
            CrossMTAdmob.Current.ShowRewardedVideo();
        }

        private void IsRewardLoaded_OnClicked(object sender, EventArgs e)
        {
            var test = CrossMTAdmob.Current.IsRewardedVideoLoaded().ToString();
        }

        private void LoadInterstitial_OnClicked(object sender, EventArgs e)
        {
            CrossMTAdmob.Current.LoadInterstitial("ca-app-pub-
3940256099942544/1033173712");
        }

        private void ShowInterstitial_OnClicked(object sender, EventArgs e)
        {
            CrossMTAdmob.Current.ShowInterstitial();
        }

        private void IsLoadedInterstitial_OnClicked(object sender, EventArgs e)
        {
            var test = CrossMTAdmob.Current.IsInterstitialLoaded().ToString();
        }

        private void NextPage(object sender, EventArgs e)
        {
            DisableEvents();
            //Navigation.PushAsync(new SecondPage());
        }

        private void Button_Clicked_2(object sender, EventArgs e)
        {
            var test = CrossMTAdmob.Current.IsInterstitialLoaded().ToString();
            CrossMTAdmob.Current.ShowInterstitial();
            CrossMTAdmob.Current.LoadInterstitial("ca-app-pub-
3940256099942544/1033173712");
        }
    }
}

```

Carpeta ViewPayPal

PayPalPage.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="IURIS.MOVIL.Views.ViewPayPal.PayPalPage">
    <ContentPage.Content>
        <StackLayout>
            <Label
                HorizontalOptions="Center"
                Text="PayPal"
                VerticalOptions="CenterAndExpand" />
            <Button
                x:Name="btnPayment"
                Clicked="BtnPayment_OnClicked"
                Text="Pay here" />
        </StackLayout>
    </ContentPage.Content>
</ContentPage>
```

PayPalPage.xaml.cs

```
using PayPal.Forms;
using PayPal.Forms.Abstractions;
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views.ViewPayPal
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
```



```

public partial class PayPalPage : ContentPage
{
    int _Monto;

    public PayPalPage(int Monto)
    {
        InitializeComponent();

        _Monto = Monto;
    }

    private async void BtnPayment_OnClicked(object sender, EventArgs e)
    {
        try
        {
            var result = await CrossPayPalManager.Current.Buy(new PayPalItem("Compra en IURIS",
new Decimal(_Monto), "MXN"),
                new Decimal(0));

            if (result.Status == PayPalStatus.Cancelled)
            {
                Debug.WriteLine("Cancelled");
            }

            else if (result.Status == PayPalStatus.Error)
            {
                Debug.WriteLine(result.ErrorMessage);
            }

            else if (result.Status == PayPalStatus.Successful)
            {
                Debug.WriteLine(result.ServerResponse.Response.Id);
            }
        }
        catch (Exception exception)
    }
}

```

```

    {
        Console.WriteLine(exception);
    }
}
}
}
}

```

Carpeta ViewsApuntes

MisApuntes.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:d="http://xamarin.com/schemas/2014/forms/design"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006" xmlns:Local="clr-namespace:IURIS.MOVIL.Modelos_y_clases"
    mc:Ignorable="d"
    x:Class="IURIS.MOVIL.Views.MisApuntes" Title="Mis Apuntes">

    <NavigationPage.TitleView>
        <Grid ColumnDefinitions="Auto,*" >
            <Label Grid.Column="0" Style="{StaticResource
TextoAMostrarColorBlanco}" Text="Apuntes" FontSize="Title"
HorizontalTextAlignment="Center" HorizontalOptions="Start"
VerticalOptions="Center"/>
            <StackLayout Orientation="Horizontal" Grid.Column="1"
HorizontalOptions="End" VerticalOptions="Center">
                <Image x:Name="IMGGuardar" Source="IconGuardar.png"
WidthRequest="25" HeightRequest="25" HorizontalOptions="End"
VerticalOptions="Center">
                    <Image.GestureRecognizers>
                        <TapGestureRecognizer
Tapped="TapGestureRecognizer_Tapped_1"/>
                    </Image.GestureRecognizers>
                </Image>
                <Image x:Name="IMGTresPuntos" Source="Tres_puntos.png"
WidthRequest="25" HeightRequest="25" HorizontalOptions="End"
VerticalOptions="Center" Margin="20,0">
                    <Image.GestureRecognizers>
                        <TapGestureRecognizer
Tapped="TapGestureRecognizer_Tapped"/>
                    </Image.GestureRecognizers>
                </Image>
            </StackLayout>
        </Grid>
    </NavigationPage.TitleView>

    <FlexLayout JustifyContent="Start">

```

```

        <Label Style="{StaticResource TextoAMostrarColorRojo}"
HorizontalTextAlignment="Center" VerticalOptions="Center" FontSize="Title"
FontAttributes="Bold" Margin="0,20" Text="MIS APUNTES"/>
        <Local:CustomEditor x:Name="TxtMiApunte" Placeholder="Escribe tus
ideas..." PlaceholderColor="{StaticResource ColorTexto}"
TextColor="{StaticResource ColorTexto}" AutoSize="TextChanges" MaxLength="2000"
Margin="20,0"/>
    </FlexLayout>
</ContentPage>

```

MisApuntes.xaml.cs

```

using IURIS.BIZ;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion.ComponentesDeUsuario;
using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using IURIS.MOVIL.Views.ViewsApuntes;
using IURIS.MOVIL.Views.ViewsVentanasEmergentes;
using Rg.Plugins.Popup.Services;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class MisApuntes : ContentPage
    {
        public Apunte _Apunte;
        public Apunte _ApunteCopia;

        protected override bool OnBackButtonPressed()
        {
            Navigation.PopAsync(false);
            Navigation.PushModalAsync(new FirtsView(), false);
            return true;
        }

        public MisApuntes(Apunte apunte)
        {
            InitializeComponent();
            _Apunte = apunte;
            _ApunteCopia = apunte;

            if (_Apunte != null) TxtMiApunte.Text = apunte.MiApunte;
        }

        private async void TapGestureRecognizer_Tapped(object sender, EventArgs
e)
        {

```

```

        await Navigation.PushAsync(new ViewsMisApuntes(), false);
    }

    private async void TapGestureRecognizer_Tapped_1(object sender,
EventArgs e)
    {
        if (string.IsNullOrEmpty(TxtMiApunte.Text)) return;

        if (_Apunte != null)
        {
            if (_Apunte.MiApunte.Equals(TxtMiApunte.Text)) return;

            _Apunte.MiApunte = TxtMiApunte.Text;
            TxtMiApunte.Text = null;

            if (!await App.Database.UpdateUserAsync(App.MyUser))
            {
                try
                {
                    await PopupNavigation.Instance.PushAsync(new
WindowAlert("Error", "Intenta mas tarde", "Aceptar"), false);
                    await Task.Delay(3000);
                    await PopupNavigation.Instance.PopAsync(false);
                    return;
                }
                catch (Exception)
                {
                    return;
                }
            }
        }
        else
        {
            WindowOfEmergencyNombreDeApunte pantalla = new
WindowOfEmergencyNombreDeApunte(TxtMiApunte.Text);
            await PopupNavigation.Instance.PushAsync(pantalla);
        }
    }
}

```

ViewsMisApuntes.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
xmlns:d="http://xamarin.com/schemas/2014/forms/design"
xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
mc:Ignorable="d"
x:Class="IURIS.MOVIL.Views.ViewsApuntes.ViewsMisApuntes"
Title="Apuntes guardados">

```

```

        <CollectionView x:Name="clltionApuntes" SelectionMode="Single"
ItemsSource="{Binding Apuntes}"
SelectionChanged="clltionApuntes_SelectionChanged">

            <CollectionView.EmptyView>
                <Label FontAttributes="Italic" FontSize="Large"
HorizontalTextAlignment="Center" Style="{StaticResource
TextoAMostrarColorRojo}" Text="Aun no tienes ninguna clasificación"/>
            </CollectionView.EmptyView>

            <CollectionView.ItemsLayout>
                <LinearItemsLayout Orientation="Vertical" ItemSpacing="5"
SnapPointsAlignment="Center"/>
            </CollectionView.ItemsLayout>
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <SwipeView>
                        <SwipeView.RightItems>
                            <SwipeItems Mode="Reveal" >
                                <SwipeItemView Invoked="SwipeItemView_Invoked"
BackgroundColor="{StaticResource PrimaryColor}" IsTabStop="True">
                                    <FlexLayout WidthRequest="100" Padding="5"
AlignItems="Center">
                                        <Image Source="BBasura.png"
Margin="0,2,0,0" WidthRequest="25" HeightRequest="25"></Image>
                                        <Label Text="Borrar" Style="{StaticResource
TextoAMostrarColorBlanco}"></Label>
                                    </FlexLayout>
                                </SwipeItemView>
                            </SwipeItems>
                        </SwipeView.RightItems>
                    <StackLayout>

                        <Label Text="{Binding Nombre}" Style="{StaticResource
TextoAMostrarColorRojo}" FontSize="Title" VerticalOptions="Center"
Padding="15,10"></Label>
                    </StackLayout>
                </SwipeView>
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>
</ContentPage>

```

ViewsMisApuntes.xaml.cs

```

using IURIS.BIZ;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion.ComponentesDeUsuario;
using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using IURIS.MOVIL.Views.ViewsVentanasEmergentes;
using Rg.Plugins.Popup.Services;
using System;
using System.Collections.Generic;
using System.Linq;

```

```

using System.Text;
using System.Threading.Tasks;

using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views.ViewsApuntes
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class ViewsMisApuntes : ContentPage
    {
        public ViewsMisApuntes()
        {
            InitializeComponent();
            DatosAInicializar();
        }

        private void DatosAInicializar()
        {
            LlenadosDeCampos();
        }

        private void LlenadosDeCampos()
        {
            clltionApuntes.ItemsSource = null;
            clltionApuntes.ItemsSource = App.MyUser.Apuntes;
        }

        private void clltionApuntes_SelectionChanged(object sender,
        SelectionChangedEventArgs e)
        {
            Apunte apunte = (Apunte)clltionApuntes.SelectedItem;
            if (apunte == null) return;

            App.masterDetail.IsPresented = false;
            App.masterDetail.Detail = new NavigationPage(new
            MisApuntes(apunte));
        }

        private async void SwipeItemView_Invoked(object sender, EventArgs e)
        {
            var MiApunte = ((SwipeItemView)sender).BindingContext as Apunte;

            if (MiApunte == null) return;

            App.MyUser.Apuntes.Remove(MiApunte);

            if (!await App.Database.UpdateUserAsync(App.MyUser)) return;

            LlenadosDeCampos();
            try
            {
                await PopupNavigation.Instance.PushAsync(new WindowAlert("Se
elimino el apunte"), false);
                await Task.Delay(2000);
                await PopupNavigation.Instance.PopAsync(false);
                return;
            }
        }
    }
}

```

```

        catch (Exception)
        {
            return;
        }
    }
}

```

Carpeta ViewsCargarLey

ViewCargarLey.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:d="http://xamarin.com/schemas/2014/forms/design"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006" xmlns:Local="clr-namespace:IURIS.MOVIL.Modelos_y_clases"
    mc:Ignorable="d"
    x:Class="IURIS.MOVIL.Views.ViewsCargarLey.WindowDeCopmpa">

    <NavigationPage.TitleView>

        <StackLayout Orientation="Horizontal" >
            <Image Source="Logo.png" Margin="0,5,5,5" VerticalOptions="Center"
WidthRequest="40" HeightRequest="40"></Image>
            <Label Text="IURIS" Style="{StaticResource LabelCrearCuenta}"
FontAttributes="Bold" FontSize="Title" Margin="15,0" VerticalOptions="Center"
></Label>
        </StackLayout>

    </NavigationPage.TitleView>

    <FlexLayout >
        <StackLayout Orientation="Horizontal" FlexLayout.Shrink="0"
Margin="10,5">

            <Local:CustomEntry x:Name="EntryCodigo" TextColor="{StaticResource
PrimaryColor}" Placeholder=" Introducir código"
PlaceholderColor="{StaticResource PrimaryColor}" FontSize="Title"
HorizontalOptions="FillAndExpand" Completed="TapGestureRecognizer_Tapped"/>
            <Image Source="Refresh.png" WidthRequest="40"
HorizontalOptions="End" VerticalOptions="Center" HeightRequest="40"
Margin="20,0" >
                <Image.GestureRecognizers>
                    <TapGestureRecognizer
Tapped="TapGestureRecognizer_Tapped"/>
                </Image.GestureRecognizers>
            </Image>

        </StackLayout>
        <Frame FlexLayout.Shrink="0" BackgroundColor="{StaticResource
PrimaryColor}" HorizontalOptions="FillAndExpand" HeightRequest="1"
Padding="0,1.5"/>

```

```

        <CollectionView Grid.Row="1" x:Name="clltionLeyes"
ItemsSource="{Binding MisLeyes}" Margin="10" SelectionMode="Single"
SelectionChanged="clltionLeyes_SelectionChanged">
            <CollectionView.EmptyView>
                <Label FontAttributes="Bold" HorizontalTextAlignment="Center"
FontSize="Large" HorizontalOptions="Center" Style="{StaticResource
TextoAMostrarColorRojo}">No se encontraron datos</Label>
            </CollectionView.EmptyView>
            <CollectionView.ItemsLayout>
                <LinearItemsLayout Orientation="Vertical" ItemSpacing="5"
SnapPointsAlignment="Center"/>
            </CollectionView.ItemsLayout>
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <SwipeView>
                        <SwipeView.RightItems>
                            <SwipeItems Mode="Reveal" >
                                <SwipeItemView Invoked="SwipeItem_Invoked"
BackgroundColor="{StaticResource PrimaryColor}" IsTabStop="True">
                                    <FlexLayout WidthRequest="100" Padding="5"
AlignItems="Center">
                                        <Image Source="BBasura.png"
Margin="0,2,0,0" WidthRequest="25" HeightRequest="25"></Image>
                                        <Label Text="Borrar"
Style="{StaticResource TextoAMostrarColorBlanco}"></Label>
                                    </FlexLayout>
                                </SwipeItemView>
                            </SwipeItems>
                        </SwipeView.RightItems>
                    </SwipeView>
                    <StackLayout>
                        <FlexLayout JustifyContent="Start" Margin="15,0" >
                            <Label Text="{Binding NombreLey}" MaxLines="3"
FontSize="15" FontAttributes="Bold" Style="{StaticResource
TextoAMostrarColorRojo}" HorizontalOptions="Start"/>
                            <Label Text="{BindingCodigoLey}" FontSize="14"
Style="{StaticResource TextoAMostrarColorRojo}" HorizontalOptions="Start"/>
                        </FlexLayout>
                        <Frame BackgroundColor="{StaticResource
PrimaryColor}" HeightRequest="1" Padding="0.5" Margin="5,0"/>
                    </StackLayout>
                </SwipeView>
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>
</FlexLayout>
</ContentPage>

```

ViewCargarLey.xaml.cs

```

using Acr.UserDialogs;
using IURIS.BIZ;
using IURIS.COMMON.Constantes;
using IURIS.COMMON.Entidades.Ley;
using IURIS.COMMON.Entidades.Ley.ComponentesDeLey;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;

```



```

using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using IURIS.MOVIL.Detail;
using IURIS.MOVIL.Modelos_y_clases.DB_Local;
using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using IURIS.MOVIL.Modelos_y_clases.Tools;
using IURIS.MOVIL.Utils;
using IURIS.MOVIL.Views.ViewsVentanasEmergentes;
using MarcTron.Plugin;
using Rg.Plugins.Popup.Services;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Xamarin.Essentials;
using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views.ViewsCargarLey
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class WindowDeCopmpa : ContentPage
    {
        IManejadorDeLeyes manejadorDeLeyes;
        //IManejadorDeUsuarioAplicacion manejadorDeUsuario;
        List<Leyes> Leyes;
        Const _Const = new Const();
        public WindowDeCopmpa()
        {
            InitializeComponent();
            MainThread.BeginInvokeOnMainThread(async () =>
            {
                await Task.Delay(30);
                CargarDatos();
                if (App.MyUser.MisLeyes.Count == 1)
                {
                    //UserDialogs.Instance.Toast(" Para eliminar una ley,\n
desliza hacia la izquierda la ley y preciona eliminar",
                    TimeSpan.FromMilliseconds(3000));
                    try
                    {
                        await PopupNavigation.Instance.PushAsync(new
WindowAlert("Para eliminar una ley,\n desliza hacia la izquierda la ley y
preciona eliminar"), false);
                        await Task.Delay(2000);
                        await PopupNavigation.Instance.PopAsync(false);
                        return;
                    }
                    catch (Exception)
                    {
                        return;
                    }
                }
            }
            }
    }
}

```

```

        manejadorDeLeyes = new ManejadorDeLeyes(new
RepositorioGenerico<Leyes>());

    });
    BindingContext = this;

    CrossMTAdmob.Current.LoadInterstitial("ca-app-pub-
3940256099942544/1033173712");

    CrossMTAdmob.Current.LoadRewardedVideo("ca-app-pub-
3940256099942544/5224354917");

    }

    private void CargarDatos()
    {
        cltionLeyes.ItemsSource = null;
        cltionLeyes.ItemsSource =
App.MyUser.MisLeyes.OrderByDescending(e=>e.FechaDeDescarga);
        Limpiardatos();
    }

    private async void TapGestureRecognizer_Tapped(object sender, EventArgs
e)
    {
        try
        {
            UserDialogs.Instance.ShowLoading("Validando datos\nEspere...",
MaskType.Gradient);
            await Task.Delay(1000);

            //manejadorDeUsuario = new ManejadorDeUsuarioAplicacion(new
RepositorioGenerico<Usuarios>(true));
            //Usuarios _User =
manejadorDeUsuario.EncontrarUsuarioID(App.MyUser.IdApp);

            //if (_User == null)
            //{
            //    UserDialogs.Instance.HideLoading();
            //    return;
            //}

            //HerramientasGenerales herramientasGenerales = new
HerramientasGenerales();
            //herramientasGenerales.RenovarSuscripcion();

            UserDialogs.Instance.HideLoading();
            UserDialogs.Instance.ShowLoading("Buscando ley",
MaskType.Gradient);
            await Task.Delay(1000);

            HerramientasGenerales herramientasGenerales1 = new
HerramientasGenerales();

```

```

        string x = await
herramientasGenerales1.AgragarLEyAsync(EntryCodigo.Text);
        string[] subs = x.Split('|');

        UserDialogs.Instance.HideLoading();

        if (subs[0] != "OK")
        {
            await PopupNavigation.Instance.PushAsync(new
WindowAlert(subs[0], true), false);
            await Task.Delay(5000);
            await PopupNavigation.Instance.PopAsync(false);
            return;
        }

        await PopupNavigation.Instance.PushAsync(new WindowAlert("Ley
agregada"), false);
        await Task.Delay(2000);
        await PopupNavigation.Instance.PopAsync(false);

        CargarDatos();
        Limpiardatos();
    }
    catch (Exception)
    {
        UserDialogs.Instance.HideLoading();
        //await DisplayAlert("Error", "Esta acción no se puede realizar
por el momento\nIntente mas tarde", "OK");
        await PopupNavigation.Instance.PushAsync(new
WindowAlert("Error", "Esta acción no se puede realizar por el momento\nIntente
mas tarde", "OK"), false);
        await Task.Delay(2000);
        await PopupNavigation.Instance.PopAsync(false);
        return;
    }
}

private void Limpiardatos()
{
    EntryCodigo.Text = null;
}

private async void clltionLeyes_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    UserDialogs.Instance.ShowLoading("Cargando ley");
    await Task.Delay(200);
    if (clltionLeyes.SelectedItem == null) return;

    App.MyUser.MiUltimaLeyCargada =
((MyLey)clltionLeyes.SelectedItem).CodigoLey;
    Settings.LastCode = App.MyUser.MiUltimaLeyCargada;

    UserDialogs.Instance.HideLoading();
    if (!await App.Database.UpdateUserAsync(App.MyUser))
    {

```

```

        await PopupNavigation.Instance.PushAsync(new
WindowAlert("error", "No se ha podido cargar la ley\n por favor intente mas
tarde", "OK"), false);
        await Task.Delay(2000);
        await PopupNavigation.Instance.PopAsync(false);

        return;
    }

    App.masterDetail.IsPresented = false;
    App.masterDetail.Detail = new NavigationPage(new ViewDetail());
}

private async void SwipeItem_Invoked(object sender, EventArgs e)
{
    UserDialogs.Instance.ShowLoading("Borrando ley",
MaskType.Gradient);
    await Task.Delay(1000);
    var MiLey = ((SwipeItemView)sender).BindingContext as MyLey;

    if (MiLey == null || MiLey.CodigoLey=="ley#1") return;

    if (App.MyUser.MiUltimaLeyCargada.Equals(MiLey.CodigoLey))
App.MyUser.MiUltimaLeyCargada = App.MyUser.MisLeyes.Where(w => w.CodigoLey !=
MiLey.CodigoLey).First().CodigoLey;

    App.MyUser.MisLeyes.Remove(MiLey);

    if (!await App.Database.UpdateUserAsync(App.MyUser)) return;

    UserDialogs.Instance.HideLoading();
    CargarDatos();

    await PopupNavigation.Instance.PushAsync(new WindowAlert("Se borro
la ley de tu lista"), false);
    await Task.Delay(2000);
    await PopupNavigation.Instance.PopAsync(false);
}
}
}

```

Subcarpeta TabbPage

TabbedPageCargar_CambiarLey.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<TabbedPage xmlns="http://xamarin.com/schemas/2014/forms"
xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
xmlns:local="clr-
namespace:IURIS.MOVIL.Views.ViewsCargarLey.ViewClasificaciones"
xmlns:local1="clr-namespace:IURIS.MOVIL.Views.ViewsCargarLey"

x:Class="IURIS.MOVIL.Views.ViewsCargarLey.TabbPage.TabbedPageCargar_CambiarLey"
>
    <NavigationPage.TitleView>

```

```

        <StackLayout Orientation="Horizontal" >
            <Image Source="Logo.png" Margin="0,5,5,5" VerticalOptions="Center"
WidthRequest="40" HeightRequest="40"></Image>
            <Label Text="IURIS" Style="{StaticResource LabelCrearCuenta}"
FontAttributes="Bold" FontSize="Title" Margin="15,0" VerticalOptions="Center"
></Label>
        </StackLayout>

    </NavigationView.TitleView>

    <local1:WindowDeCoppma x:Name="cambiar" Title="Cambiar Ley"
IconImageSource="shuffle.png"/>

    <local:ViewOfClasificaciones Title="Buscar Ley"
IconImageSource="Buscador.png"/>
</TabbedPage>

```

TabbedPageCargar_CambiarLey.xaml.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views.ViewsCargarLey.TabbPage
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class TabbedPageCargar_CambiarLey : TabbedPage
    {
        protected override bool OnBackButtonPressed()
        {
            Navigation.PopAsync(false);
            Navigation.PushModalAsync(new FirtsView(), false);
            return true;
        }
        public TabbedPageCargar_CambiarLey()
        {
            InitializeComponent();
        }
    }
}

```

Subcarpeta ViewClasificaciones

ViewOfClasificaciones.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
```

```

<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              xmlns:Local="clr-namespace:IURIS.MOVIL.Modelos_y_clases"

              x:Class="IURIS.MOVIL.Views.ViewsCargarLey.ViewClasificaciones.ViewOfClasificaci
              ones">
    <NavigationPage.TitleView>

        <StackLayout Orientation="Horizontal" >
            <Image Source="Logo.png" Margin="0,5,5,5" VerticalOptions="Center"
            WidthRequest="40" HeightRequest="40"></Image>
            <Label Text="IURIS" Style="{StaticResource LabelCrearCuenta}"
            FontAttributes="Bold" FontSize="Title" Margin="15,0" VerticalOptions="Center"
            ></Label>
        </StackLayout>

    </NavigationPage.TitleView>

    <FlexLayout >
        <StackLayout Orientation="Horizontal" FlexLayout.Shrink="0"
        Margin="10,5">
            <Local:CustomEntry x:Name="EntryCodigo" TextColor="{StaticResource
            PrimaryColor}" Placeholder="Buscar clasificación..."
            PlaceholderColor="{StaticResource PrimaryColor}" FontSize="Title"
            HorizontalOptions="FillAndExpand" Completed="TapGestureRecognizer_Tapped"
            TextChanged="EntryCodigo_TextChanged"/>
        </StackLayout>

        <Frame FlexLayout.Shrink="0" BackgroundColor="{StaticResource
        PrimaryColor}" HorizontalOptions="FillAndExpand" HeightRequest="1"
        Padding="0,1.5"/>

        <CollectionView Grid.Row="1" x:Name="clltionClasificaciones"
        Margin="10" SelectionMode="Single"
        SelectionChanged="clltionClasificaciones_SelectionChanged">
            <CollectionView.EmptyView>
                <Label FontAttributes="Bold" HorizontalTextAlignment="Center"
                FontSize="Large" HorizontalOptions="Center" Style="{StaticResource
                TextoAMostrarColorRojo}">No se encontraron datos</Label>
            </CollectionView.EmptyView>
            <CollectionView.ItemsLayout>
                <LinearLayout Orientation="Vertical" ItemSpacing="5"
                SnapPointsAlignment="Center"/>
            </CollectionView.ItemsLayout>
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <FlexLayout JustifyContent="Start" Margin="15,0" >
                        <Label Text="{Binding Nombre}" MaxLines="1"
                        FontSize="25" FontAttributes="Bold" Style="{StaticResource
                        TextoAMostrarColorRojo}" HorizontalOptions="Start" Margin="20,0"/>
                        <Frame BackgroundColor="{StaticResource PrimaryColor}"
                        HeightRequest="1" Padding="0.5" Margin="5,0"/>
                    </FlexLayout>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </FlexLayout>
</ContentPage>

```

ViewOfClasificaciones.xaml.cs

```
using Acr.UserDialogs;
using IURIS.BIZ;
using IURIS.COMMON.Entidades.Ley.ClasificacionDeLey;
using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Xamarin.Essentials;
using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views.ViewsCargarLey.ViewClasificaciones
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class ViewOfClasificaciones : ContentPage
    {
        IManejadorDeClasificaciones manejadorDeClasificaciones;
        public ViewOfClasificaciones()
        {
            UserDialogs.Instance.ShowLoading("Obteniendo datos...",
MaskType.None);
            InitializeComponent();
            Task.Delay(2000);
            clltionClasificaciones.ItemsSource =
App.MyClasificacionDeLey.GetPeopleAsync().Result;
            UserDialogs.Instance.HideLoading();
        }

        private void TapGestureRecognizer_Tapped(object sender, EventArgs e)
        {
            SearchText(EntryCodigo.Text);
        }

        private void SearchText(string text)
        {
            clltionClasificaciones.ItemsSource =
App.MyClasificacionDeLey.GetPeopleAsync().Result.Where(e=>e.Nombre.ToUpper().Co
ntains(text.ToUpper()));
        }

        private void EntryCodigo_TextChanged(object sender,
TextChangedEventArgs e)
        {
            SearchText(e.NewTextValue);
        }

        private async void clltionClasificaciones_SelectionChanged(object
sender, SelectionChangedEventArgs e)
```

```

        {
            var x= clltionClasificaciones.SelectedItem as MyClasificacionDeLey;
            await Navigation.PushAsync(new ViewLeyes.ViewLeyes(x.Nombre),
false);
            //clltionClasificaciones.SelectedItem = null;
        }
    }
}

```

Sub-subcarpeta ViewPageLeyes

ViewLeyes.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:Local="clr-namespace:IURIS.MOVIL.Modelos_y_clases"

x:Class="IURIS.MOVIL.Views.ViewsCargarLey.ViewClasificaciones.ViewLeyes.ViewLeyes">
    <NavigationPage.TitleView>

        <StackLayout Orientation="Horizontal" >
            <Image Source="Logo.png" Margin="0,5,5,5" VerticalOptions="Center"
WidthRequest="40" HeightRequest="40"></Image>
            <Label Text="IURIS" Style="{StaticResource LabelCrearCuenta}"
FontAttributes="Bold" FontSize="Title" Margin="15,0" VerticalOptions="Center"
></Label>
        </StackLayout>

    </NavigationPage.TitleView>
    <FlexLayout>
        <StackLayout Orientation="Horizontal" FlexLayout.Shrink="0"
Margin="10,5">
            <Local:CustomEntry x:Name="EntryCodigo" TextColor="{StaticResource
PrimaryColor}" Placeholder="Buscar nombre de ley..."
PlaceholderColor="{StaticResource PrimaryColor}" FontSize="Title"
HorizontalOptions="FillAndExpand" Completed="EntryCodigo_Completed"
TextChanged="EntryCodigo_TextChanged"/>
        </StackLayout>

        <CollectionView Grid.Row="1" x:Name="clltionLeyes" Margin="10"
SelectionMode="Single" ItemsUpdatingScrollMode="KeepItemsInView"
SelectionChanged="clltionLeyes_SelectionChanged">
            <CollectionView.EmptyView>
                <Label FontAttributes="Bold" HorizontalTextAlignment="Center"
FontSize="Large" HorizontalOptions="Center" Style="{StaticResource
TextoAMostrarColorRojo}">No se encontraron datos</Label>
            </CollectionView.EmptyView>
            <CollectionView.ItemsLayout>
                <LinearItemsLayout Orientation="Vertical" ItemSpacing="5"
SnapPointsAlignment="Center"/>
            </CollectionView.ItemsLayout>
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <StackLayout>

```



```

                <FlexLayout JustifyContent="Start" Margin="15,0" >
                    <Label Text="{Binding NombreLey}" MaxLines="3"
FontSize="15" FontAttributes="Bold" Style="{StaticResource
TextoAMostrarColorRojo}" HorizontalOptions="Start"/>
                    <Label Text="{Binding CodigoLey}" FontSize="14"
Style="{StaticResource TextoAMostrarColorRojo}" HorizontalOptions="Start"/>
                </FlexLayout>
                <Frame BackgroundColor="{StaticResource PrimaryColor}"
HeightRequest="1" Padding="0.5" Margin="5,0"/>
            </StackLayout>
        </DataTemplate>
    </CollectionView.ItemTemplate>
</CollectionView>
</FlexLayout>
</ContentPage>

```

ViewLeyes.xaml.cs

```

using Acr.UserDialogs;
using IURIS.BIZ;
using IURIS.COMMON.Entidades.Ley;
using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using IURIS.MOVIL.Modelos_y_clases.Tools;
using IURIS.MOVIL.Views.ViewsCargarLey.TabbPage;
using IURIS.MOVIL.Views.ViewsVentanasEmergentes;
using Rg.Plugins.Popup.Services;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Xamarin.Essentials;
using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views.ViewsCargarLey.ViewClasificaciones.ViewLeyes
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class ViewLeyes : ContentPage
    {
        IManejadorDeLeyes manejadorDeLeyes;
        List<Leyes> _Leyes=null;
        public ViewLeyes(string Clasificacion)
        {
            UserDialogs.Instance.ShowLoading("Descargando leyes\nde nuestro
servidor", MaskType.None);
            Task.Delay(500);
            InitializeComponent();

            MainThread.BeginInvokeOnMainThread(async () =>
            {
                await Task.Delay(1000);
                manejadorDeLeyes = new ManejadorDeLeyes(new
RepositorioGenerico<Leyes>());
            });
        }
    }
}

```

```

        _Leyes = await manejadorDeLeyes.Consults(Clasificacion);
        clltionLeyes.ItemsSource = _Leyes;
        // clltionLeyes.ItemsSource = _Leyes;
        UserDialogs.Instance.HideLoading();
    });
}

private void EntryCodigo_TextChanged(object sender,
TextChangedEventArgs e)
{
    SearchText(e.NewTextValue);
}

private void SearchText(string text)
{
    clltionLeyes.ItemsSource =
_Leyes.Where(e=>e.NombreLey.ToUpper().Contains(text.ToUpper()));
}

private void EntryCodigo_Completed(object sender, EventArgs e)
{
    SearchText(EntryCodigo.Text);
}

private async void clltionLeyes_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    try
    {
        UserDialogs.Instance.ShowLoading("Buscando ley",
MaskType.Gradient);
        await Task.Delay(1000);

        HerramientasGenerales herramientasGenerales = new
HerramientasGenerales();
        string x = await
herramientasGenerales.AgragarLEyAsync(((Leyes)clltionLeyes.SelectedItem).Codigo
Ley);

        string[] subs = x.Split('|');

        if (subs[0] != "OK")
        {
            await PopupNavigation.Instance.PushAsync(new
WindowAlert(subs[0], true), false);
            await Task.Delay(6000);
            await PopupNavigation.Instance.PopAsync(false);
            return;
        }

        await PopupNavigation.Instance.PushAsync(new WindowAlert("Ley
agregada"), false);
        await Task.Delay(2000);
        await PopupNavigation.Instance.PopAsync(false);

        App.masterDetail.IsPresented = false;
        //App.masterDetail.Detail = new NavigationPage(new
WindowDeCopmpa());
    }
}

```

```

        App.masterDetail.Detail = new NavigationPage(new
        TabbedPageCargar_CambiarLey());
    }
    catch (Exception)
    {
        UserDialogs.Instance.HideLoading();
    }
}
}
}

```

Carpeta ViewsClasificaciones

ViewMiClasificacionPersonalizada.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:d="http://xamarin.com/schemas/2014/forms/design"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
    xmlns:local="clr-namespace:IURIS.MOVIL.Modelos_y_clases"
    xmlns:pv="clr-
namespace:Xamarin.Forms.PancakeView;assembly=Xamarin.Forms.PancakeView"
    xmlns:xct="http://xamarin.com/schemas/2020/toolkit"
    mc:Ignorable="d"
    x:Class="IURIS.MOVIL.Views.ViewMiClasificacionPersonalizada">

    <NavigationPage.TitleView>
        <FlexLayout Direction="Row">

            <Grid ColumnDefinitions="*,Auto" ColumnSpacing="5" >
                <FlexLayout Grid.Column="0" >
                    <local:CustomLabel x:Name="lblTitle" Style="{StaticResource
TextoAMostrarColorBlanco}" FontSize="Title"/>
                </FlexLayout>

                <Image Grid.Column="1" x:Name="IMGBasura" Source="BBasura.png"
WidthRequest="20" HorizontalOptions="End" HeightRequest="20" Margin="10,0,20,0"
IsVisible="false">
                    <Image.GestureRecognizers>
                        <TapGestureRecognizer
Tapped="TapGestureRecognizer_Tapped"/>
                    </Image.GestureRecognizers>
                </Image>
            </Grid>
        </FlexLayout>
    </NavigationPage.TitleView>

    <StackLayout>
        <CollectionView x:Name="clltionMiClasificacionPersonalziada"
SelectionMode="Multiple" ItemsSource="{Binding MisArticulos}" Margin="10,5"
SelectionChanged="clltionMiClasificacionPersonalziada_SelectionChanged">

```

```

        <CollectionView.EmptyView>
            <Label Style="{StaticResource TextoAMostrarColorRojo}"
FontAttributes="Bold" FontSize="Large" HorizontalTextAlignment="Center"
Text="No se encontraron datos"/>
        </CollectionView.EmptyView>
        <CollectionView.ItemsLayout>
            <LinearItemsLayout Orientation="Vertical" ItemSpacing="10"/>
        </CollectionView.ItemsLayout>
        <CollectionView.ItemTemplate>
            <DataTemplate>

                <pv:PancakeView HorizontalOptions="FillAndExpand" >
                    <Grid>

                        <FlexLayout Direction="Row"
Margin="40,0,0,0" JustifyContent="Start">
                            <Label FlexLayout.Shrink="0"
Text="{Binding NumArticulo}" FontSize="17" HorizontalTextAlignment="Start"
VerticalOptions="Center" Style="{StaticResource TextoAMostrarColorRojo}"
FontAttributes="Bold"/>

                            <Label Text="{Binding NombreArticulo}"
FontSize="16" Style="{StaticResource TextoAMostrarColorRojo}"
HorizontalTextAlignment="Start" VerticalOptions="Center" FontAttributes="Bold"
Margin="5,0"/>
                        </FlexLayout>
                        <xct:Expander HorizontalOptions="Start"
VerticalOptions="Center" >

                            <xct:Expander.Header>

                                <StackLayout Orientation="Horizontal">
                                    <Image Source="Mas.png" WidthRequest="30"
HeightRequest="30" FlexLayout.Shrink="0" HorizontalOptions="Start">
                                        <Image.Triggers>
                                            <DataTrigger TargetType="Image"
Binding="{Binding Source={RelativeSource AncestorType={x:Type xct:Expander}}},
Path=IsExpanded" Value="True">
                                                <Setter Property="Source"
Value="menos.png"/>
                                            </DataTrigger>
                                        </Image.Triggers>
                                    </Image>
                                </StackLayout>

                                </xct:Expander.Header>
                                <xct:Expander.Content>

                                    <Label Text="{Binding Contenido}"
FontSize="Subtitle" Style="{StaticResource TextoAMostrarColorRojo}"
Margin="0,20"/>
                                </xct:Expander.Content>

                            </xct:Expander>
                        </Grid>
                    </pv:PancakeView>

```

```

        </DataTemplate>
    </CollectionView.ItemTemplate>
</CollectionView>
</StackLayout>
</ContentPage>

```

ViewMiClasificacionPersonalizada.xaml.cs

```

using IURIS.BIZ;
using IURIS.COMMON.Entidades.Ley;
using IURIS.COMMON.Entidades.Ley.ComponentesDeLey;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion.ComponentesDeUsuario;
using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using IURIS.MOVIL.Views.ViewsVentanasEmergentes;
using Rg.Plugins.Popup.Services;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class ViewMiClasificacionPersonalizada : ContentPage
    {
        ClasificacionPUusuario _Clasificacion;
        MyLey _MyLey;
        public List<Articulo> _Articulos;

        public ViewMiClasificacionPersonalizada(ClasificacionPUusuario
clasificacion, MyLey leyes)
        {
            InitializeComponent();

            _Clasificacion = clasificacion;
            _MyLey = leyes;
            DatosAInicializar();
        }

        private void DatosAInicializar()
        {
            lblTitle.Text = _Clasificacion.Nombre;
            _Articulos = new List<Articulo>();

            ActualizarTabla();
        }
    }
}

```

```

        private void ActualizarTabla()
        {
            clltionMiClasificacionPersonalziada.ItemsSource = null;
            clltionMiClasificacionPersonalziada.ItemsSource =
            _Clasificacion.MisArticulos;
        }

        private void
        clltionMiClasificacionPersonalziada_SelectionChanged(object sender,
        SelectionChangedEventArgs e)
        {
            IMGBasura.IsVisible = e.CurrentSelection.Count() >= 1 ? true :
            false;
        }

        private async void TapGestureRecognizer_Tapped(object sender, EventArgs
        e)
        {
            var Articulo = clltionMiClasificacionPersonalziada.SelectedItems;
            foreach (var item in Articulo)
            _Clasificacion.MisArticulos.Remove(item as Articulo);

            try
            {
                if (!await App.Database.UpdateUserAsync(App.MyUser))
                {
                    await showAlert("Por favor intenta mas tarde");
                    //await App.HerramientasGenerales.showAlerta(new
                    WindowAlert("Por favor intenta mas tarde"));
                    return;
                }

                await showAlert("Se eliminaron los elementos");

                IMGBasura.IsVisible = false;
                ActualizarTabla();
            }
            catch (Exception ex)
            {
                await showAlert("Por el momento no se puede agregar su
                clasificacion\n por favor intente mas tarde\nError:" + ex.Message);
                return;
            }
        }

        private async Task showAlert(string mensaje)
        {
            try
            {
                await PopupNavigation.Instance.PushAsync(new
                WindowAlert(mensaje), false);
                await Task.Delay(2000);
                await PopupNavigation.Instance.PopAsync(false);
            }
        }
    }

```

```

        return;
    }
    catch (Exception)
    {
        return;
    }
}
}
}

```

ViewsMisClasificaciones.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:d="http://xamarin.com/schemas/2014/forms/design"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
    mc:Ignorable="d"
    x:Class="IURIS.MOVIL.Views.ViewsMisClasificaciones" Title="Mis
clasificaciones">
    <FlexLayout >

        <Button Text="Crear clasificación" Style="{StaticResource
buttonSecundario}" FlexLayout.Shrink="0" FlexLayout.Basis="8%" FontSize="Title"
Clicked="TapGestureRecognizer_Tapped" Margin="-5"/>

        <CollectionView BackgroundColor="White" x:Name="clltionClasificaciones"
SelectionMode="Single" ItemsSource="{Binding Clasificaciones}"
SelectionChanged="clltionClasificaciones_SelectionChanged" >
            <CollectionView.EmptyView>
                <Label FlexLayout.Basis="7%" FontAttributes="Italic"
FontSize="Large" HorizontalTextAlignment="Center" Style="{StaticResource
TextoAMostrarColorRojo}">Aún no tienes ninguna clasificación</Label>
            </CollectionView.EmptyView>
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <SwipeView>
                        <SwipeView.RightItems>
                            <SwipeItems Mode="Reveal" >
                                <SwipeItemView Invoked="SwipeItemView_Invoked"
BackgroundColor="{StaticResource PrimaryColor}" IsTabStop="True">
                                    <FlexLayout WidthRequest="100" Padding="5"
AlignItems="Center">
                                        <Image Source="BBasura.png"
Margin="0,2,0,0" WidthRequest="25" HeightRequest="25"></Image>
                                        <Label Text="Borrar"
Style="{StaticResource TextoAMostrarColorBlanco}"></Label>
                                    </FlexLayout>
                                </SwipeItemView>
                            </SwipeItems>
                        </SwipeView.RightItems>
                    </SwipeView>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </FlexLayout>

```

```

                                <Label FlexLayout.Basis="7%" Text="{Binding
Nombre}" Style="{StaticResource TextoAMostrarColorRojo}" FontSize="Title"
VerticalOptions="Center" Padding="15,10"></Label>
                                </StackLayout>
                            </SwipeView>
                        </DataTemplate>
                    </CollectionView.ItemTemplate>
                </CollectionView>
            </FlexLayout>
        </ContentPage>

```

ViewsMisClasificaciones.xaml.cs

```

using IURIS.BIZ;
using IURIS.COMMON.Entidades.Ley;
using IURIS.COMMON.Entidades.Ley.ComponentesDeLey;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion.ComponentesDeUsuario;
using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using IURIS.MOVIL.Detail;
using IURIS.MOVIL.Modelos_y_clases;
using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using IURIS.MOVIL.Views.ViewsVentanasEmergentes;
using Rg.Plugins.Popup.Services;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Xamarin.Essentials;
using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class ViewsMisClasificaciones : ContentPage
    {
        Artículo _Articulo;
        public bool nuevoArticulo = false;
        readonly MyLey _MyLey;

        protected override bool OnBackButtonPressed()
        {
            Navigation.PopAsync(false);
            Navigation.PushModalAsync(new FirtsView(), false);
            return true;
        }

        public ViewsMisClasificaciones(Articulo articulo, MyLey leyes)
        {
            InitializeComponent();

```



```

        _MyLey = leyes;
        _Articulo = articulo;
        ActualizarDatos();
    }

    void ActualizarDatos()
    {
        MainThread.BeginInvokeOnMainThread(async () =>
        {
            await Task.Delay(100);

            clltionClasificaciones.ItemsSource = null;
            var clas = App.MyUser.MisLeyes.Find(w => w.CodigoLey ==
_MyLey.CodigoLey).Clasificaciones;
            clltionClasificaciones.ItemsSource = clas;
            if (_Articulo != null && clas.Count == 0)
            {
                await PopupNavigation.Instance.PushAsync(new
WindowOfEmergencyNuevaClasificacion(_MyLey, _Articulo));
            }
        });
    }

    private async void TapGestureRecognizer_Tapped(object sender, EventArgs
e)
    {
        try
        {
            await PopupNavigation.Instance.PushAsync(new
WindowOfEmergencyNuevaClasificacion(_MyLey, _Articulo));
        }
        catch(Exception ex)
        {
        }
    }

    private async void clltionClasificaciones_SelectionChanged(object
sender, SelectionChangedEventArgs e)
    {
        ClasificacionPUusuario clasificacion =
(ClasificacionPUusuario)clltionClasificaciones.SelectedItem;
        if (_Articulo != null)
        {

            if (!(clasificacion.MisArticulos.Where(w => w.NumArticulo ==
_Articulo.NumArticulo).Count() >= 1))
            {

                clasificacion.MisArticulos.Add(_Articulo);
                try
                {

```

```

        if (!await App.Database.UpdateUserAsync(App.MyUser))
        {
            await showAlert("Por favor intenta mas tarde");
            return;
        }

        await showAlert("Agregada correctamente");
        _Articulo = null;
        return;
    }
    catch (Exception ex)
    {
        await showAlert("Por el momento no se puede agregar su
clasificacion\n por favor intente mas tarde\nError:" + ex.Message);
        return;
    }
}

await Navigation.PushAsync(new
ViewMiClasificacionPersonalizada(clasificacion, _MyLey), false);
}

private async void SwipeItemView_Invoked(object sender, EventArgs e)
{
    var MiClasificacion = ((SwipeItemView)sender).BindingContext as
ClasificacionPUusuario;

    if (MiClasificacion == null) return;
    App.MyUser.MisLeyes.Where(w => w.CodigoLey ==
_MyLey.CodigoLey).SingleOrDefault().Clasificaciones.Remove(MiClasificacion);

    if (!await App.Database.UpdateUserAsync(App.MyUser)) return;
    ActualizarDatos();
}

private async Task showAlert(string mensaje)
{
    try
    {
        await PopupNavigation.Instance.PushAsync(new
WindowAlert(mensaje), false);
        await Task.Delay(2000);
        await PopupNavigation.Instance.PopAsync(false);
        return;
    }
    catch (Exception)
    {
        return;
    }
}
}
}
}

```

Carpeta ViewsVentanasEmergentes

PageVistaSinInternet.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"

x:Class="IURIS.MOVIL.Views.ViewsVentanasEmergentes.PageVistaSinInternet"
        BackgroundColor="{StaticResource PrimaryColor}"
        NavigationPage.HasBackButton="False"
        NavigationPage.HasNavigationBar="False">

    <ContentPage.Content>
        <FlexLayout Padding="20" JustifyContent="Center">

            <StackLayout Padding="50,0,50,30">

                <Image FlexLayout.Shrink="0" Source="SinConection.png"
Aspect="AspectFit" HorizontalOptions="Center" VerticalOptions="Center"/>

                <Image FlexLayout.Shrink="0" Source="op.png" Aspect="AspectFit"
HorizontalOptions="Center" VerticalOptions="Center"/>
            </StackLayout>

            <Label x:Name="txtData" Margin="0,20" TextColor="White"
FontSize="40" FontAttributes="Bold" HorizontalTextAlignment="Center" ></Label>
            <Button x:Name="btnIntentarDeNuevo" Margin="60,40,60,0"
FontSize="20" Text="Volver a Intentar" Clicked="btnIntentarDeNuevo_Clicked"
HorizontalOptions="CenterAndExpand" CornerRadius="20" BackgroundColor="White"
TextColor="{StaticResource PrimaryColor}" BorderColor="Black"></Button>

        </FlexLayout>
    </ContentPage.Content>
</ContentPage>
```

PageVistaSinInternet.xaml.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Net.NetworkInformation;
using System.Text;
using System.Threading.Tasks;

using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views.ViewsVentanasEmergentes
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class PageVistaSinInternet : ContentPage
    {
        public PageVistaSinInternet(String Txt)
        {
            InitializeComponent();
        }
    }
}
```

```

    {
        InitializeComponent();
        txtData.Text = Txt;
    }

e) private async void btnIntentarDeNuevo_Clicked(object sender, EventArgs
    {
        if (HayConexion())
        {
            await Navigation.PushAsync(new PagePrimeravista(), true);
        }
    }

    public static bool HayConexion()
    {
        try
        {
            string huesped = $"8.8.8.8";

            return new Ping().Send(huesped).Status == IPStatus.Success;
        }
        catch
        {
            return false;
        }
    }
}

```

WindowAlert.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<Pages:PopupPage xmlns="http://xamarin.com/schemas/2014/forms"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="IURIS.MOVIL.Views.ViewsVentanasEmergentes.WindowAlert"
    BackgroundColor="Transparent"
    xmlns:Pages="clr-
namespace:Rg.Plugins.Popup.Pages;assembly=Rg.Plugins.Popup"

    xmlns:d="http://xamarin.com/schemas/2014/forms/design"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006" xmlns:Local="clr-namespace:IURIS.MOVIL.Modelos_y_clases"
    mc:Ignorable="d">

    <Frame x:Name="Frame"
        BackgroundColor="{StaticResource PrimaryColor}"
        Margin="50"
        Padding="5"
        HorizontalOptions="FillAndExpand"
        VerticalOptions="Center"

```

```

        CornerRadius="7"
        Opacity="0.75">
        <Grid ColumnDefinitions="auto,*,auto" RowDefinitions="auto,*,auto"
        ColumnSpacing="0" RowSpacing="0" VerticalOptions="Center"
        HorizontalOptions="Center" Padding="15">

            <Label x:Name="LblTitulo" IsVisible="false" Grid.Row="0"
            Grid.Column="0" Grid.ColumnSpan="2" HorizontalTextAlignment="Start"
            VerticalOptions="Center" VerticalTextAlignment="Center" Style="{StaticResource
            TextoAMostrarColorBlanco}" ></Label>
            <Label x:Name="LblMensaje" IsVisible="false" Grid.Row="1"
            Grid.Column="0" Grid.ColumnSpan="3" HorizontalTextAlignment="Center"
            VerticalOptions="Center" VerticalTextAlignment="Center" Style="{StaticResource
            TextoAMostrarColorBlanco}" ></Label>
            <Label x:Name="LblMensajeAgregarLey" IsVisible="false" Grid.Row="1"
            Grid.Column="0" Grid.ColumnSpan="3" HorizontalTextAlignment="Center"
            VerticalOptions="Center" VerticalTextAlignment="Center" Style="{StaticResource
            TextoAMostrarColorBlanco}" >
                <Label.FormattedText>
                    <FormattedString>
                        <Span Text="Has utilizado los espacios gratuitos, si
                        requieres de mas espacio para agregar leyes a tu lista ponte en contacto al
                        correo: &#10;"></Span>
                        <Span Text="Para solicitar mas espacio ponte en
                        contacto con: "></Span>
                        <Span Text="iurisespacio@gmail.com"
                        TextDecorations="Underline" FontSize="Body" FontAttributes="Bold">
                            <!--<Span.GestureRecognizers>
                                <TapGestureRecognizer
                                </Span.GestureRecognizers>-->
                            </Span>
                        </FormattedString>
                    </Label.FormattedText>
                </Label>

            <Button Style="{StaticResource buttonPrincipal}"
            x:Name="LblCancelar" IsVisible="false" Grid.Row="2" Grid.Column="0"
            VerticalOptions="Center" ></Button>
            <Button Style="{StaticResource buttonPrincipal}"
            x:Name="LblAceptar" IsVisible="false" Grid.Row="2" Grid.Column="2"
            VerticalOptions="Center" Clicked="LblAceptar_Clicked" ></Button>
        </Grid>

    </Frame>
</Pages:PopupPage>

```

WindowAlert.xaml.cs

```

using Rg.Plugins.Popup.Pages;
using Rg.Plugins.Popup.Services;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

```

using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views.ViewsVentanasEmergentes
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class WindowAlert : PopupPage
    {
        public WindowAlert(string Mensaje)
        {
            InitializeComponent();
            LblMensaje.IsVisible = true;

            LblMensaje.Text = Mensaje;
        }

        public WindowAlert(string Titulo, string Mensaje)
        {
            InitializeComponent();
            LblMensaje.IsVisible = true;
            LblTitulo.IsVisible = true;

            LblMensaje.Text = Mensaje;
            LblTitulo.Text= Titulo;
        }

        public WindowAlert(string Titulo, bool AñadirMasEspacio)
        {
            InitializeComponent();

            LblMensajeAgregarLey.IsVisible = AñadirMasEspacio;
            LblTitulo.IsVisible = AñadirMasEspacio;

            LblTitulo.Text = Titulo;
            Frame.Opacity=1;
        }

        public WindowAlert(string Titulo, string Mensaje, string
MensajeAceptar)
        {
            InitializeComponent();
            LblMensaje.IsVisible = true;
            LblTitulo.IsVisible=true;
            LblAceptar.IsVisible=true;

            LblMensaje.Text = Mensaje;
            LblAceptar.Text = MensajeAceptar;
            LblTitulo.Text= Titulo;
        }

        public WindowAlert(string Titulo, string Mensaje, string
MensajeAceptar, string MensajeCancelar)

```

```

        {
            InitializeComponent();
            LblMensaje.Text = Mensaje;
            for (int i = 0; i < 5; i++)
            {
                Task.Delay(700);
            }
        }

private async void LblAceptar_Clicked(object sender, EventArgs e)
{
    await PopupNavigation.Instance.PopAsync(false);
}
}
}

```

WindowListadoDeClasificaciones.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<Pages:PopupPage xmlns:Pages="clr-
namespace:Rg.Plugins.Popup.Pages;assembly=Rg.Plugins.Popup"
xmlns="http://xamarin.com/schemas/2014/forms"
xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"

x:Class="IURIS.MOVIL.Views.ViewsVentanasEmergentes.WindowListadoDeClasificacion
es"

    BackgroundColor="Transparent"
    Padding="15,30" >

    <Frame BackgroundColor="{StaticResource PrimaryColor}"
        Margin="0,50"
        Padding="15"
        HorizontalOptions="FillAndExpand"
        VerticalOptions="EndAndExpand"
        CornerRadius="10"
        >
        <StackLayout>

            <CollectionView BackgroundColor="{StaticResource PrimaryColor}" "
                Opacity="0.8" x:Name="clltionClasificaciones"
                SelectionMode="Single" ItemsSource="{Binding Clasificaciones}"
                SelectionChanged="clltionClasificaciones_SelectionChanged" >
                <CollectionView.Header>
                    <StackLayout>
                        <Label FlexLayout.Basis="7%" Text="Elige una
clasificación" Style="{StaticResource TextoAMostrarColorBlanco}"
                        FontSize="Title" VerticalOptions="Center" Padding="15,10"></Label>
                        <Frame BackgroundColor="{StaticResource PrimaryColor}"
                        HeightRequest="1" Padding="0.5" Margin="5,0"/>
                    </StackLayout>
                </CollectionView.Header>
                <CollectionView.EmptyView>

```

```

                <Label FlexLayout.Basis="7%" FontAttributes="Italic"
FontSize="Large" HorizontalTextAlignment="Center" Style="{StaticResource
TextoAMostrarColorBlanco}">Aún no tienes ninguna clasificación</Label>
            </CollectionView.EmptyView>
        </CollectionView.ItemTemplate>
        <DataTemplate>
            <StackLayout>
                <Label FlexLayout.Basis="7%" Text="{Binding
Nombre}" Style="{StaticResource TextoAMostrarColorBlanco}" FontSize="Title"
VerticalOptions="Center" Padding="15,10"></Label>
            </StackLayout>
        </DataTemplate>
    </CollectionView.ItemTemplate>
</CollectionView>
<Grid ColumnDefinitions="*,*" ColumnSpacing="5" Margin="5"
RowDefinitions="auto">

        <Button Grid.Column="0" Grid.Row="0" Style="{StaticResource
buttonPrincipal}" x:Name="LblCancelar" FontSize="Subtitle" CornerRadius="8"
BorderColor="White" Text="Cancelar" Clicked="LblCancelar_Clicked"
Padding="5"></Button>
        <Button Grid.Column="1" Grid.Row="0" Style="{StaticResource
buttonPrincipal}" x:Name="LblAceptar" FontSize="Subtitle" CornerRadius="8"
BorderColor="White" Text="Crear nueva clasificación"
Clicked="LblAceptar_Clicked" Padding="5"></Button>

    </Grid>
</StackLayout>
</Frame>
</Pages:PopupPage>

```

WindowListadoDeClasificaciones.xaml.cs

```

using IURIS.COMMON.Entidades.Ley.ComponentesDeLey;
using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using Rg.Plugins.Popup.Pages;
using Rg.Plugins.Popup.Services;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views.ViewsVentanasEmergentes
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class WindowListadoDeClasificaciones : PopupPage
    {
        MyLey _MyLey;
        Artículo _Articulo;
        public WindowListadoDeClasificaciones(MyLey Myley, Artículo articulo)
        {

```



```

        InitializeComponent();
        _MyLey = MyLey;
        _Articulo = articulo;
        clltionClasificaciones.ItemsSource =
App.MyUser.MisLeyes.Find(d=>d.CodigoLey==_MyLey.CodigoLey).Clasificaciones;
    }

    private async void clltionClasificaciones_SelectionChanged(object
sender, SelectionChangedEventArgs e)
    {
        var classi=clltionClasificaciones.SelectedItem as
ClasificacionPUusuario;
        if (classi != null)
        {
            try
            {
                _MyLey.Clasificaciones.Find(clasi =>
clasi.Nombre.Equals(classi.Nombre)).MisArticulos.Add(_Articulo);
                await PopupNavigation.Instance.PopAllAsync(false);

                string mensaje = await
App.Database.UpdateUserAsync(App.MyUser) ? "Artículo agregado" : "Intente más
tarde" ;

                await PopupNavigation.Instance.PushAsync(new
WindowAlert(mensaje), false);
                await Task.Delay(2000);
                await PopupNavigation.Instance.PopAsync(false);
                return;
            }
            catch (Exception)
            {
                await PopupNavigation.Instance.PopAllAsync(false);
                return;
            }
        }
    }

    private async void LblCancelar_Clicked(object sender, EventArgs e)
    {
        await PopupNavigation.Instance.PopAllAsync(false);
    }

    private async void LblAceptar_Clicked(object sender, EventArgs e)
    {
        await PopupNavigation.Instance.PopAllAsync(false);
        await PopupNavigation.Instance.PushAsync(new
WindowOfEmergencyNuevaClasificacion(_MyLey, _Articulo));
    }
}
}

```

WindowOfComprarEspacio.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<Pages:PopupPage xmlns:Pages="clr-
namespace:Rg.Plugins.Popup.Pages;assembly=Rg.Plugins.Popup"
xmlns="http://xamarin.com/schemas/2014/forms"
xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
xmlns:Local="clr-namespace:IURIS.MOVIL.Modelos_y_clases"

x:Class="IURIS.MOVIL.Views.ViewsVentanasEmergentes.WindowOfComprarEspacio">

    <Frame BackgroundColor="{StaticResource SecondaryColor}"
        BorderColor="White"
        HasShadow="True"
        IsClippedToBounds="True"
        Padding="2"
        Margin="20"
        HorizontalOptions="FillAndExpand"
        VerticalOptions="CenterAndExpand"
        CornerRadius="3" >

        <Grid RowDefinitions="auto,auto,auto,*,auto"
            HorizontalOptions="FillAndExpand" VerticalOptions="FillAndExpand" >
            <Image Grid.Row="0" Source="Logo.png" WidthRequest="100"
                HeightRequest="100"/>
            <Label Grid.Row="1" Style="{StaticResource
                TextoAMostrarColorBlanco}" HorizontalTextAlignment="Center" FontSize="Title"
                FontAttributes="Bold">IURIS</Label>
            <Label x:Name="lblNoCuentas" Grid.Row="2" Style="{StaticResource
                TextoAMostrarColorBlanco}" HorizontalTextAlignment="Center" FontSize="Large"
                FontAttributes="Italic"></Label>
            <Label x:Name="lblContent" Grid.Row="3" Style="{StaticResource
                TextoAMostrarColorBlanco}" HorizontalTextAlignment="Center" FontSize="30"
                FontAttributes="Bold"></Label>

            <Grid ColumnSpacing="-1.5" ColumnDefinitions="*,*" Grid.Row="4"
                Margin="-5">
                <Button Grid.Column="0" x:Name="BtnAceptar" Text="Aceptar"
                    Style="{StaticResource buttonPrincipal}" FontSize="Medium"
                    Clicked="BtnAceptar_Clicked" ></Button>
                <Button Grid.Column="1" x:Name="BtnCancelar" Text="Cancelar"
                    Style="{StaticResource buttonPrincipal}" FontSize="Medium"
                    Clicked="BtnCancelar_Clicked"></Button>
            </Grid>

        </Grid>
    </Frame>

</Pages:PopupPage>
```

WindowOfComprarEspacio.xaml.cs

```
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
```

```

using Rg.Plugins.Popup.Pages;
using Rg.Plugins.Popup.Services;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views.ViewsVentanasEmergentes
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class WindowOfComprarEspacio : PopupPage
    {

        public WindowOfComprarEspacio()
        {
            InitializeComponent();
            lblNoCuentas.Text = "No cuentas con espacio\npara descargar otra
ley";
            lblContent.Text = "Comprar un espacio \nmás para leyes por \n$9.00
M.N";
        }

        private void BtnCancelar_Clicked(object sender, EventArgs e)
        {
            PopupNavigation.Instance.PopAsync(false);
        }

        private void BtnAceptar_Clicked(object sender, EventArgs e)
        {
            App.masterDetail.IsPresented = false;
            App.masterDetail.Detail = new NavigationPage(new
WindowOpcionDeCompra());
        }
    }
}

```

WindowOfEmergencyCrearNota.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<Pages:PopupPage xmlns:Pages="clr-
namespace:Rg.Plugins.Popup.Pages;assembly=Rg.Plugins.Popup"
xmlns="http://xamarin.com/schemas/2014/forms"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:d="http://xamarin.com/schemas/2014/forms/design"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006" xmlns:Local="clr-namespace:IURIS.MOVIL.Modelos_y_clases"
    mc:Ignorable="d"

x:Class="IURIS.MOVIL.Views.ViewsVentanasEmergentes.WindowOfEmergencyCrearNota">
    <Frame BackgroundColor="{StaticResource PrimaryColor}"
        Padding="5"

```

```

        Margin="20"
        HorizontalOptions="FillAndExpand"
        VerticalOptions="Center"
        CornerRadius="5" >

        <StackLayout>
            <Local:CustomEditor x:Name="EntryNombreApunte"
                AutoSize="TextChanges" Placeholder="Escribe una nota o comentario"
                HorizontalOptions="FillAndExpand" Style="{StaticResource EntryCrearCuenta}"
                MaxLength="300" FontSize="Medium"/>
            <Grid ColumnDefinitions="*,*" ColumnSpacing="-1.5" Margin="-5,0,-
                5,-5">
                <Button Grid.Column="0" x:Name="BtnGuardar" Text="Guardar"
                    Style="{StaticResource buttonPrincipal}" Clicked="BtnGuardar_Clicked"
                    FontSize="Large"/>

                <Button Grid.Column="1" x:Name="BtnCancelar" Text="Cancelar"
                    Style="{StaticResource buttonPrincipal}" Clicked="BtnCancelar_Clicked"
                    FontSize="Large"/>

                <Button Grid.Column="1" x:Name="BtnEliminar" Text="Eliminar"
                    Style="{StaticResource buttonPrincipal}" Clicked="BtnEliminar_Clicked"
                    FontSize="Large" IsVisible="false"/>
            </Grid>
        </StackLayout>
    </Frame>
</Pages:PopupPage>

```

WindowOfEmergencyCrearNota.xaml.cs

```

using IURIS.BIZ;
using IURIS.COMMON.Entidades.Ley;
using IURIS.COMMON.Entidades.Ley.ComponentesDeLey;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using IURIS.MOVIL.Detail;
using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using Rg.Plugins.Popup.Pages;
using Rg.Plugins.Popup.Services;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views.ViewsVentanasEmergentes
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class WindowOfEmergencyCrearNota : PopupPage
    {

```

```

        readonly Titulo _Titulo;
        readonly Artículo _Articulo;
        MyLey _MyLey;
        readonly Capitulo _Capitulo;

        public WindowOfEmergencyCrearNota(Titulo titulo, Artículo
articulo, MyLey ley, Capitulo capitulo)
        {
            InitializeComponent();
            _Articulo = articulo;
            _Titulo = titulo;
            _MyLey = ley;
            _Capitulo = capitulo;

            DatosAInicializar();
        }

        private void DatosAInicializar()
        {
            if (!_Articulo.NotaAdjunta) return;

            EntryNombreApunte.Text = _Articulo.TextoDeNota;
            MostarBotonEliminar(true);
        }

        private void MostarBotonEliminar(bool v)
        {
            BtnCancelar.IsVisible = !v;
            BtnEliminar.IsVisible = v;
        }

        private void BtnCancelar_Clicked(object sender, EventArgs e)
        {
            PopupNavigation.Instance.PopAsync(false);
        }

        private async void BtnGuardar_Clicked(object sender, EventArgs e)
        {
            if (string.IsNullOrEmpty(EntryNombreApunte.Text)) return;

            //List<Titulo> titulos = (List<Titulo>)_Ley.ListaDeTitulos;
            //_titulo = (Titulo)titulos.Find(i => i.NumTitulo ==
_Titulo.NumTitulo);

            //List<Capitulo> capitulos = _titulo.ListaCapitulos as
List<Capitulo>;
            //_capitulo = capitulos.Find(i => i.NumCapitulo ==
_Capitulo.NumCapitulo) as Capitulo;

            //List<Articulo> articulos = _capitulo.ListaArticulos as
List<Articulo>;

            //_articulo = articulos.Find(i => i.NumArticulo ==
_Articulo.NumArticulo) as Artículo;

            if (!EntryNombreApunte.Text.Equals(_Articulo.TextoDeNota))
            {

```

```

        _Articulo.NotaAdjunta = true;
        _Articulo.TextoDeNota = EntryNombreApunte.Text;

        //int NumIndex= _User.MisLeyes.IndexOf(_LeyCopia);
        //_User.MisLeyes.RemoveAt(NumIndex);
        //_User.MisLeyes.Insert(NumIndex,_Ley);
        if (!await App.Database.UpdateUserAsync(App.MyUser)) return;
    }

    await PopupNavigation.Instance.PopAsync(false);
}

private async void BtnEliminar_Clicked(object sender, EventArgs e)
{
    _Articulo.NotaAdjunta = false;
    _Articulo.TextoDeNota = null;

    if (!await App.Database.UpdateUserAsync(App.MyUser)) return;

    await PopupNavigation.Instance.PopAsync(false);
}
}
}

```

WindowOfEmergencyNombreDeApunte.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<Pages:PopupPage xmlns:Pages="clr-
namespace:Rg.Plugins.Popup.Pages;assembly=Rg.Plugins.Popup"
xmlns="http://xamarin.com/schemas/2014/forms"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:d="http://xamarin.com/schemas/2014/forms/design"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006" xmlns:Local="clr-namespace:IURIS.MOVIL.Modelos_y_clases"
    mc:Ignorable="d"

x:Class="IURIS.MOVIL.Views.ViewsVentanasEmergentes.WindowOfEmergencyNombreDeApu
nte">

    <Frame BackgroundColor="{StaticResource PrimaryColor}"
        Margin="20"
        Padding="12"
        HorizontalOptions="Center"
        VerticalOptions="Center"
        CornerRadius="5">
        <StackLayout>
            <Local:CustomEntry x:Name="EntryNombreApunte" Placeholder="Nombre
del apunte" HorizontalOptions="FillAndExpand" HorizontalTextAlignment="Start"
Style="{StaticResource EntryCrearCuenta}" Completed="BtnGuardar_Clicked"/>
            <Grid ColumnDefinitions="*,*" ColumnSpacing="-1.5" Margin="-5" >

```

```

                <Button Grid.Column="0" x:Name="BtnGuardar" Text="Guardar"
Style="{StaticResource buttonPrincipal}" Clicked="BtnGuardar_Clicked"/>
                <Button Grid.Column="1" x:Name="BtnCancelar" Text="Cancelar"
Style="{StaticResource buttonPrincipal}" Clicked="BtnCancelar_Clicked"/>
            </Grid>
        </StackLayout>
    </Frame>
</Pages:PopupPage>

```

WindowOfEmergencyNombreDeApunte.xaml.cs

```

using IURIS.BIZ;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion.ComponentesDeUsuario;
using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using IURIS.MOVIL.Views.ViewsApuntes;
using Rg.Plugins.Popup.Pages;
using Rg.Plugins.Popup.Services;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views.ViewsVentanasEmergentes
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class WindowOfEmergencyNombreDeApunte : PopupPage
    {
        readonly string _TextoApunte;
        public WindowOfEmergencyNombreDeApunte(string TextoApunte)
        {
            InitializeComponent();
            _TextoApunte = TextoApunte;
        }

        private void BtnCancelar_Clicked(object sender, EventArgs e)
        {
            PopupNavigation.Instance.PopAsync(false);
        }

        private async void BtnGuardar_Clicked(object sender, EventArgs e)
        {
            if (string.IsNullOrEmpty(EntryNombreApunte.Text)) return;

            Apunte apunte = new Apunte
            {
                MiApunte = _TextoApunte,
                Nombre = EntryNombreApunte.Text
            };

```

```

        App.MyUser.Apuntes.Add(apunte);
        if (!await App.Database.UpdateUserAsync(App.MyUser)) return;

        try
        {
            await PopupNavigation.Instance.PopAllAsync();
            await PopupNavigation.Instance.PushAsync(new
WindowAlert("Apunte agregado correctamente"), false);
            await Task.Delay(2000);
            await PopupNavigation.Instance.PopAsync(false);
            App.masterDetail.IsPresented = false;
            App.masterDetail.Detail = new NavigationPage(new
MisApuntes(null));
            return;
        }
        catch (Exception)
        {
            return;
        }
    }
}
}
}

```

WindowOfEmergencyNuevaClasificacion.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<Pages:PopupPage xmlns:Pages="clr-
namespace:Rg.Plugins.Popup.Pages;assembly=Rg.Plugins.Popup"
                xmlns="http://xamarin.com/schemas/2014/forms"
                xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
                xmlns:d="http://xamarin.com/schemas/2014/forms/design"
                xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006" xmlns:Local="clr-namespace:IURIS.MOVIL.Modelos_y_clases"
                mc:Ignorable="d"

x:Class="IURIS.MOVIL.Views.ViewsVentanasEmergentes.WindowOfEmergencyNuevaClasif
icacion">
    <Frame BackgroundColor="{StaticResource PrimaryColor}"
        Margin="20"
        Padding="5"
        HorizontalOptions="Center"
        VerticalOptions="Center"
        CornerRadius="5">
        <StackLayout>
            <Local:CustomEntry FontSize="Large"
x:Name="EntryNuevaClasificacion" Placeholder="Nueva clasificación ..."
HorizontalTextAlignment="Start" Style="{StaticResource EntryCrearCuenta}"
Completed="BtnAceptar_Clicked"></Local:CustomEntry>

            <Grid ColumnDefinitions="*,*" ColumnSpacing="-1.5" Margin="-5,0,-
5,-5">
                <Button Grid.Column="0" x:Name="BtnAceptar" Text="Aceptar"
Style="{StaticResource buttonPrincipal}" Clicked="BtnAceptar_Clicked"></Button>

```



```

        <Button Grid.Column="1" x:Name="BtnCancelar" Text="Cancelar"
Style="{StaticResource buttonPrincipal}" Clicked="BtnCancelar_Clicked"
></Button>
    </Grid>
</StackLayout>
</Frame>
</Pages:PopupPage>

```

WindowOfEmergencyNuevaClasificacion.xaml.cs

```

using IURIS.BIZ;
using IURIS.COMMON.Entidades.Ley;
using IURIS.COMMON.Entidades.Ley.ComponentesDeLey;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion.ComponentesDeUsuario;
using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using Rg.Plugins.Popup.Pages;
using Rg.Plugins.Popup.Services;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views.ViewsVentanasEmergentes
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class WindowOfEmergencyNuevaClasificacion : PopupPage
    {
        MyLey _MyLey;
        Artículo _Articulo;
        public WindowOfEmergencyNuevaClasificacion(MyLey leyes, Artículo
articulo)
        {
            InitializeComponent();
            _MyLey = leyes;
            _Articulo = articulo;
        }

        private async void BtnAceptar_Clicked(object sender, EventArgs e)
        {
            if (string.IsNullOrWhiteSpace(EntryNuevaClasificacion.Text))
return;

            ClasificacionPUusuario clasificacion = new ClasificacionPUusuario
            {
                Nombre = EntryNuevaClasificacion.Text,
                MisArticulos = new List<Artículo>()
            };

```

```

        if (string.IsNullOrEmpty(clasificacion.Nombre)) return;

        if (_Articulo != null)
            clasificacion.MisArticulos.Add(_Articulo);

        App.MyUser.MisLeyes.Find(w => w.CodigoLey ==
        _MyLey.CodigoLey).Clasificaciones.Add(clasificacion);

        try
        {

            if (!await App.Database.UpdateUserAsync(App.MyUser))
            {
                await showAlert("No se pudo completar la operación");
                return;
            }

            await showAlert("Agregada correctamente");
            if (_Articulo != null)
            {
                await PopupNavigation.Instance.PopAsync(false);
                return;
            }

            App.masterDetail.IsPresented = false;
            App.masterDetail.Detail = new NavigationPage(new
ViewsMisClasificaciones(null, _MyLey));
            await PopupNavigation.Instance.PopAllAsync(false);
        }
        catch (Exception ex)
        {

        }

    }

    private void BtnCancelar_Clicked(object sender, EventArgs e)
    {
        PopupNavigation.Instance.PopAsync(false);
    }

    private async Task showAlert(string mensaje)
    {
        try
        {
            await PopupNavigation.Instance.PushAsync(new
WindowAlert(mensaje), false);
            await Task.Delay(2000);
            await PopupNavigation.Instance.PopAsync(false);
            return;
        }
        catch (Exception ex)
        {

        }

    }
}

```

```
}
```

WindowOfMenuAccion.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<Pages:PopupPage xmlns:Pages="clr-
namespace:Rg.Plugins.Popup.Pages;assembly=Rg.Plugins.Popup"
xmlns="http://xamarin.com/schemas/2014/forms"
xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"

x:Class="IURIS.MOVIL.Views.ViewsVentanasEmergentes.WindowOfMenuAccion">

    <Frame BackgroundColor="{StaticResource PrimaryColor}"
        BorderColor="White"
        HasShadow="True"
        IsClippedToBounds="True"
        Margin="10"
        HorizontalOptions="CenterAndExpand"
        VerticalOptions="CenterAndExpand"
        CornerRadius="5" >

        <StackLayout Orientation="Vertical" HorizontalOptions="FillAndExpand"
            VerticalOptions="FillAndExpand" Padding="5" Spacing="10">
            <Label Text="Clasificación personalizada" Style="{StaticResource
                TextoAMostrarColorBlanco}" FontSize="Large" HorizontalTextAlignment="Center"
                VerticalTextAlignment="Center" VerticalOptions="Center">
                <Label.GestureRecognizers>
                <TapGestureRecognizer
                    Tapped="LblClasificacionPersonalizada"/>
                </Label.GestureRecognizers>
            </Label>
            <Frame BackgroundColor="{StaticResource SecondaryColor}"
                HorizontalOptions="FillAndExpand" HeightRequest="1" Margin="-50,0"
                Padding="0,1"/>
            <Label Text="Crear Nota" Style="{StaticResource
                TextoAMostrarColorBlanco}" FontSize="Large" HorizontalTextAlignment="Center"
                VerticalTextAlignment="Center" VerticalOptions="Center">
                <Label.GestureRecognizers >
                <TapGestureRecognizer Tapped="LblCrearNota"/>
                </Label.GestureRecognizers>
            </Label>
            <Frame BackgroundColor="{StaticResource SecondaryColor}"
                HorizontalOptions="FillAndExpand" HeightRequest="1" Margin="-50,0"
                Padding="0,1"/>
            <Label Text="Habilitar subrayado" Style="{StaticResource
                TextoAMostrarColorBlanco}" FontSize="Large" HorizontalTextAlignment="Center"
                VerticalTextAlignment="Center" VerticalOptions="Center">
                <Label.GestureRecognizers >
                <TapGestureRecognizer Tapped="GuardarSeleccion"/>
                </Label.GestureRecognizers>
            </Label>
        </StackLayout>
    </Frame>
```

</Pages:PopupPage>

WindowOfMenuAccion.xaml.cs

```
using Acr.UserDialogs;
using IURIS.BIZ;
using IURIS.COMMON.Entidades.Ley;
using IURIS.COMMON.Entidades.Ley.ComponentesDeLey;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;
using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using IURIS.MOVIL.Modelos_y_clases.DB_Local.COMMON;
using Plugin.Clipboard;
using Rg.Plugins.Popup.Pages;
using Rg.Plugins.Popup.Services;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;
using Xamarin.Essentials;
using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views.ViewsVentanasEmergentes
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class WindowOfMenuAccion : PopupPage
    {
        readonly Titulo _Titulo;
        readonly Artículo _Articulo;
        MyLey _MyLey;
        readonly Capitulo _Capitulo;

        public WindowOfMenuAccion(Titulo titulo, Artículo articulo, MyLey ley,
            Capitulo capitulo)
        {
            InitializeComponent();
            _Articulo = articulo;
            _Titulo = titulo;
            _MyLey = ley;
            _Capitulo = capitulo;
        }

        private async void LblCrearNota(object sender, EventArgs e)
        {
            await PopupNavigation.Instance.PopAsync(false);
            await PopupNavigation.Instance.PushAsync(new
            WindowOfEmergencyCrearNota(_Titulo, _Articulo, _MyLey, _Capitulo), false);
        }

        private async Task showAlert(string mensaje)
        {
            try
```

```

        {
            await PopupNavigation.Instance.PushAsync(new
WindowAlert(mensaje), false);
            await Task.Delay(2000);
            await PopupNavigation.Instance.PopAsync(false);
            return;
        }
        catch (Exception)
        {
            return;
        }
    }

    private async void LblClasificacionPersonalizada(object sender,
EventArgs e)
    {
        //await Task.Delay(2000);
        //await PopupNavigation.Instance.PopAsync(false);
        if (_MyLey.Clasificaciones.Count() > 0)
        {
            await PopupNavigation.Instance.PushAsync(new
WindowListadoDeClasificaciones(_MyLey, _Articulo), false);
            //List<String> name = new List<String>();
            //_MyLey.Clasificaciones.ForEach(d => name.Add(d.Nombre));
            //string[] Opciones = new[] { "Cancelar", "Crear nueva
clasificación" };

            //var Clasificaicon = await
UserDialogs.Instance.ActionSheetAsync("Elige una clasificación", Opciones[0],
Opciones[1], Cancellation.Token.None, name.ToArray());

            //if (!string.IsNullOrEmpty(Clasificaicon)&&
!Clasificaicon.Equals(Opciones[0]))
            //{

                //    if (Clasificaicon.Equals(Opciones[1]))
                //    {

                    //        await PopupNavigation.Instance.PushAsync(new
WindowOfEmergencyNuevaClasificacion(_MyLey, _Articulo));
                    //        //await Navigation.PushAsync(new
ViewsMisClasificaciones(_Articulo, _MyLey), false);
                    //        //await
PopupNavigation.Instance.PopAllAsync(IsAnimating);

                    //    }
                    //    else
                    //    {
                        //_MyLey.Clasificaciones.Find(clasi =>
clasi.Nombre.Equals(Clasificaicon)).MisArticulos.Add(_Articulo);

                        //        string[] datos = await
App.Database.UpdateUserAsync(App.MyUser) ? new[] { "Ok", "Artículo agregado" }
: new[] { "Error", "Intente más tarde" };

                        //        UserDialogs.Instance.Alert(datos[0], datos[1]);
                    }
                }
            }
        }
    }

```

```

        //      await PopupNavigation.Instance.PopAsync(false);
        //    }
        //}
    }
    else
    {
        MainThread.BeginInvokeOnMainThread(async () =>
        {
            await Task.Delay(100);

            await Navigation.PushAsync(new
ViewsMisClasificaciones(_Articulo, _MyLey), false);
            await PopupNavigation.Instance.PopAsync(false);
        });
    }

}

private void CrearNuevaClasificaicon()
{
    MainThread.BeginInvokeOnMainThread(async () =>
    {
        await Task.Delay(100);
        await PopupNavigation.Instance.PushAsync(new
WindowOfEmergencyNuevaClasificacion(_MyLey, _Articulo));
        //await Navigation.PushAsync(new
ViewsMisClasificaciones(_Articulo, _MyLey), false);
        await PopupNavigation.Instance.PopAsync(false);
    });
}

private async void GuardarSeleccion(object sender, EventArgs e)
{
    try
    {
        await PopupNavigation.Instance.PopAsync(false);
        await showAlert("Preciona el boton de +,\nselecciona el texto y
da en copiar");
    }
    catch (Exception)
    {

    }

    return;
}

}
}
}

```

WindowOpcionDeCompra.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"

```

```

x:Class="IURIS.MOVIL.Views.ViewsVentanasEmergentes.WindowOpcionDeCompra"
Title="Opciones de compra">

    <ContentPage.Content>
        <ContentView BackgroundColor="White">

            <FlexLayout Margin="0,0,0,15" >

                <StackLayout FlexLayout.Shrink="0" Margin="0,-20,0,0"
                BackgroundColor="Black" Padding="20">
                    <Image FlexLayout.Shrink="0" Source="Logo.png"
                    Aspect="Fill" HorizontalOptions="CenterAndExpand"
                    VerticalOptions="CenterAndExpand"/>
                </StackLayout>

                <Label FlexLayout.Shrink="0" Text="IURIS"
                HorizontalTextAlignment="Center" FontAttributes="Bold" FontSize="40"
                TextColor="{StaticResource PrimaryColor}"></Label>

                <Button x:Name="_Btn39Mensuales" FlexLayout.Shrink="0"
                Text="Cero comerciales, 1 espacio más para leyes por 39 pesos mensuales"
                Style="{StaticResource buttonPrincipal}" BorderColor="Transparent"
                CornerRadius="10" Margin="30,0" Padding="15"
                Clicked="_Btn39Mensuales_Clicked"></Button>

                <Button x:Name="_Btn49Mensuales" FlexLayout.Shrink="0"
                Text="Cero comerciales, espacio ilimitado para leyes por 49 pesos mensuales"
                Style="{StaticResource buttonPrincipal}" BorderColor="Transparent"
                CornerRadius="10" Margin="30,0" Padding="15"
                Clicked="_Btn49Mensuales_Clicked"></Button>

                <Button x:Name="_Btn479Anuales" FlexLayout.Shrink="0"
                Text="Cero comerciales, espacio ilimitado para leyes por 479 pesos al año"
                Style="{StaticResource buttonPrincipal}" BorderColor="Transparent"
                CornerRadius="10" Margin="30,0" Padding="15"
                Clicked="_Btn479Anuales_Clicked"></Button>

                <Button x:Name="_Btn19PorLey" FlexLayout.Shrink="0"
                Text="Comprar un espacio para una ley por 19 pesos de forma permanente"
                Style="{StaticResource buttonPrincipal}" BorderColor="Transparent"
                CornerRadius="10" Margin="30,0" Padding="15"
                Clicked="_Btn19PorLey_Clicked"></Button>

            </FlexLayout>

        </ContentView>
    </ContentPage.Content>
</ContentPage>

```

WindowOpcionDeCompra.xaml.cs

```

using Acr.UserDialogs;
using IURIS.BIZ;
using IURIS.COMMON.Entidades.UsuariosDeAplicacion;

```

```

using
IURIS.COMMON.Entidades.UsuariosDeAplicacion.ComponentesDeUsuario.DatosCriticos;
using IURIS.COMMON.Interfaces;
using IURIS.DAL;
using IURIS.MOVIL.Modelos_y_clases.DB_Local;
using IURIS.MOVIL.Views.ViewPayPal;
using PayPal.Forms;
using PayPal.Forms.Abstractions;
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Xamarin.Essentials;
using Xamarin.Forms;
using Xamarin.Forms.Xaml;

namespace IURIS.MOVIL.Views.ViewsVentanasEmergentes
{
    [XamlCompilation(XamlCompilationOptions.Compile)]
    public partial class WindowOpcionDeCompra : ContentPage
    {
        //IManejadorDeUsuarioAplicacion manejadorDeUsuarioAplicacion;

        protected override bool OnBackButtonPressed()
        {
            Navigation.PopAsync(false);
            Navigation.PushModalAsync(new FirtsView(), false);
            return true;
        }
        enum TipoDeCompra
        {
            _39Mensuales,
            _49Mensuales,
            _479Anuales,
            _ComprarEspacio1Ley
        }
        TipoDeCompra MiCompra;

        //Usuarios _User;
        public WindowOpcionDeCompra()
        {
            InitializeComponent();
            //MainThread.BeginInvokeOnMainThread(async () =>
            //{
            //    await Task.Delay(1000);
            //    manejadorDeUsuarioAplicacion = new
            ManejadorDeUsuarioAplicacion(new RepositorioGenerico<Usuarios>(true));
            //    _User =
            manejadorDeUsuarioAplicacion.EncontrarUsuarioID(App.MyUser.IdApp);
            //});
            //UserDialogs.Instance.ShowLoading("Validando datos\nEspere...",
            MaskType.Gradient);
            //while (_User != null) ;

            UserDialogs.Instance.HideLoading();
        }
    }

```



```

private void _Btn39Mensuales_Clicked(object sender, EventArgs e)
{
    MiCompra = TipoDeCompra._39Mensuales;
    SendPagePayPal(39);
}

private void _Btn49Mensuales_Clicked(object sender, EventArgs e)
{
    MiCompra = TipoDeCompra._49Mensuales;
    SendPagePayPal(49);
}

private void _Btn479Anuales_Clicked(object sender, EventArgs e)
{
    MiCompra = TipoDeCompra._479Anuales;
    SendPagePayPal(479);
}

private void _Btn19PorLey_Clicked(object sender, EventArgs e)
{
    MiCompra = TipoDeCompra._ComprarEspacio1Ley;
    SendPagePayPal(19);
}

private async void SendPagePayPal(int Monto)
{
    //await Navigation.PopAsync();
    //await Navigation.PushAsync(new PayPalPage(Monto), false);
    try
    {
        var result = await CrossPayPalManager.Current.Buy(new
PayPalItem("Compra en IURIS", new Decimal(Monto), "MXN"),
            new Decimal(0));
        if (result.Status == PayPalStatus.Cancelled)
        {
            Debug.WriteLine("Cancelled");
        }
        else if (result.Status == PayPalStatus.Error)
        {
            Debug.WriteLine(result.ErrorMessage);
        }
        else if (result.Status == PayPalStatus.Successful)
        {
            //App.MyUser.DatosSobreUsuario.FechaDeCompra =
DateTime.UtcNow;
            //if (App.MyUser.DatosSobreUsuario.TipoDeCompra == null)
App.MyUser.DatosSobreUsuario.TipoDeCompra = new List<string>();

            if (MiCompra == TipoDeCompra._ComprarEspacio1Ley)
            {

```

```

        App.MyUser.DatosSobreUsuario.NumLeyesPermitidas =
App.MyUser.DatosSobreUsuario.NumLeyesPermitidas == default ? 5 :
App.MyUser.DatosSobreUsuario.NumLeyesPermitidas + 1;

    }
    else
    {

        if (MiCompra == TipoDeCompra._39Mensuales)
        {
            //if
(!App.MyUser.DatosSobreUsuario.TipoDeCompra.Contains(MiCompra.ToString()))
App.MyUser.DatosSobreUsuario.NumLeyesPermitidas += 1;
            //App.MyUser.DatosSobreUsuario.NumDeMeses = 1;
        }
        if (MiCompra == TipoDeCompra._49Mensuales)
        {
            //App.MyUser.DatosSobreUsuario.TipoDeCompra = new
List<string>();
            //App.MyUser.DatosSobreUsuario.NumLeyesPermitidas
=default;
            //App.MyUser.DatosSobreUsuario.NumDeMeses = 1;
        }
        if (MiCompra == TipoDeCompra._479Anuales)
        {
            //App.MyUser.DatosSobreUsuario.TipoDeCompra = new
List<string>();
            //App.MyUser.DatosSobreUsuario.NumLeyesPermitidas=default;
            //App.MyUser.DatosSobreUsuario.NumDeMeses = 12;
        }

    }

    //if (App.MyUser.DatosSobreUsuario.TipoDeCompra.Where(e =>
e.ToString() == MiCompra.ToString()).Count() == 0)
App.MyUser.DatosSobreUsuario.TipoDeCompra.Add(MiCompra.ToString());

    LocalSaveUser saveUser = new LocalSaveUser();
    await saveUser.UpdateUser();

    }
}
catch (Exception exception)
{
    Console.WriteLine(exception);
}
}
}
}

```